

```

const express = require("express");
const router = express.Router();
const pool = require("../db");
const admin = require("../firebase-admin");

// POST /api/auth/register
router.post("/register", async (req, res) => {
  const { firebase_uid, email, display_name, role } = req.body;

  if (!firebase_uid || !email) {
    return res.status(400).json({ error: "firebase_uid and email are required" });
  }

  try {
    await pool.query(
      `INSERT INTO users (firebase_uid, email, display_name, role)
      VALUES (?, ?, ?, ?)
      ON DUPLICATE KEY UPDATE
        email = VALUES(email),
        display_name = VALUES(display_name)`,
      [firebase_uid, email, display_name || null, role || "student"]
    );

    const [rows] = await pool.query(
      `SELECT id, firebase_uid, email, display_name, role, created_at
      FROM users WHERE firebase_uid = ?`,
      [firebase_uid]
    );

    res.json({ success: true, user: rows[0] });
  } catch (err) {
    console.error("POST /api/auth/register:", err.message);
    res.status(500).json({ error: err.message });
  }
});

// GET /api/auth/me
router.get("/me", async (req, res) => {
  const token = req.headers.authorization?.split("Bearer ")[1];

```

```

    if (!token) return res.status(401).json({ error: "No token provided" });

    try {
      const decoded = await admin.auth().verifyIdToken(token);

      const [rows] = await pool.query(
        `SELECT id, firebase_uid, email, display_name, role, created_at
         FROM users WHERE firebase_uid = ?`,
        [decoded.uid]
      );

      if (!rows.length) return res.status(404).json({ error: "User not in DB yet" });
      res.json(rows[0]);
    } catch (err) {
      res.status(401).json({ error: "Token invalid: " + err.message });
    }
  });

// PATCH /api/auth/last-active
router.patch("/last-active", async (req, res) => {
  const { firebase_uid } = req.body;
  try {
    await pool.query(
      `UPDATE users SET last_active = NOW() WHERE firebase_uid = ?`,
      [firebase_uid]
    );
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
const express = require("express");
const router = express.Router();
const pool = require("../db");
const admin = require("../firebase-admin");

// POST /api/auth/register

```

```

router.post("/register", async (req, res) => {
  const { firebase_uid, email, display_name, role } = req.body;

  if (!firebase_uid || !email) {
    return res.status(400).json({ error: "firebase_uid and email are
required" });
  }

  try {
    await pool.query(
      `INSERT INTO users (firebase_uid, email, display_name, role)
VALUES (?, ?, ?, ?)
ON DUPLICATE KEY UPDATE
  email = VALUES(email),
  display_name = VALUES(display_name)`,
      [firebase_uid, email, display_name || null, role || "student"]
    );

    const [rows] = await pool.query(
      `SELECT id, firebase_uid, email, display_name, role, created_at
FROM users WHERE firebase_uid = ?`,
      [firebase_uid]
    );

    res.json({ success: true, user: rows[0] });
  } catch (err) {
    console.error("POST /api/auth/register:", err.message);
    res.status(500).json({ error: err.message });
  }
});

// GET /api/auth/me
router.get("/me", async (req, res) => {
  const token = req.headers.authorization?.split("Bearer ")[1];
  if (!token) return res.status(401).json({ error: "No token provided" });

  try {
    const decoded = await admin.auth().verifyIdToken(token);

    const [rows] = await pool.query(

```

```

        `SELECT id, firebase_uid, email, display_name, role, created_at
        FROM users WHERE firebase_uid = ?`,
        [decoded.uid]
    );

    if (!rows.length) return res.status(404).json({ error: "User not in DB yet" });
    res.json(rows[0]);
  } catch (err) {
    res.status(401).json({ error: "Token invalid: " + err.message });
  }
});

// PATCH /api/auth/last-active
router.patch("/last-active", async (req, res) => {
  const { firebase_uid } = req.body;
  try {
    await pool.query(
      `UPDATE users SET last_active = NOW() WHERE firebase_uid = ?`,
      [firebase_uid]
    );
    res.json({ success: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
// backend/routes/modules.js
const express = require("express");
const router = express.Router();
const pool = require("../db");
const multer = require("multer");
const path = require("path");
const fs = require("fs");

// — Multer —————
const uploadDir = path.join(__dirname, "../uploads");
if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir, { recursive: true });

```

```

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, uploadDir),
  filename: (req, file, cb) => {
    const unique = Date.now() + "-" + Math.round(Math.random() * 1e9);
    cb(null, unique + path.extname(file.originalname));
  },
});

const upload = multer({
  storage,
  limits: { fileSize: 5 * 1024 * 1024 },
  fileFilter: (req, file, cb) => {
    cb(null, /jpeg|jpg|png|gif|webp/.test(file.mimetype));
  },
});

const parseJSON = (v, fallback = []) => {
  if (!v) return fallback;
  try { return JSON.parse(v); } catch { return fallback; }
};

const toBool = (v) => v === "true" || v === true || v === 1 ? 1 : 0;

// =====
// IMPORTANT: specific named routes MUST come before /:id
// =====

// GET /api/modules/test
router.get("/test", (req, res) => res.json({ ok: true }));

// GET /api/modules/stats - viewer counts per module
router.get("/stats", async (req, res) => {
  try {
    const [rows] = await pool.query(`
      SELECT
        module_id          AS id,
        COUNT(*)            AS enrolled,
        ROUND(AVG(progress),1) AS avg_progress,
        SUM(completed=1)    AS completed_count
      FROM module_progress
    `);
  }
});

```

```

        GROUP BY module_id
    `);
    res.json(rows);
} catch (err) {
    console.error("GET /stats:", err.message);
    res.status(500).json({ error: err.message });
}
});

// GET /api/modules/progress/:userId
router.get("/progress/:userId", async (req, res) => {
    try {
        const [rows] = await pool.query(
            "SELECT module_id, progress, completed FROM module_progress WHERE user_id = ?",
            [req.params.userId]
        );
        res.json(rows);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

// POST /api/modules/progress
router.post("/progress", async (req, res) => {
    const { userId, userName, moduleId, progress, completed } = req.body;
    if (!userId || !moduleId)
        return res.status(400).json({ ok: false, error: "userId and moduleId required" });
    try {
        await pool.query(
            `INSERT INTO module_progress (user_id, user_name, module_id, progress, completed)
            VALUES (?, ?, ?, ?, ?)
            ON DUPLICATE KEY UPDATE
            user_name=VALUES(user_name), progress=VALUES(progress), completed=VALUES(completed)`,
            [userId, userName || "", moduleId, progress || 0, completed ? 1 : 0]
        );
        res.json({ ok: true });
    }
});

```

```

    } catch (err) {
      res.status(500).json({ ok: false, error: err.message });
    }
  });

// — GET /api/modules (list) —————
router.get("/", async (req, res) => {
  const { role } = req.query;
  try {
    let sql = `
      SELECT m.*,
        (SELECT COUNT(*) FROM module_sections WHERE module_id = m.id) AS
section_count,
        (SELECT COUNT(*) FROM module_quizzes WHERE module_id = m.id) AS
quiz_count,
        (SELECT COUNT(*) FROM module_progress WHERE module_id = m.id) AS
viewer_count,
        (SELECT ROUND(AVG(progress),1) FROM module_progress WHERE
module_id = m.id) AS avg_progress
      FROM modules m
    `;
    if (role !== "admin") sql += " WHERE m.is_published = 1";
    sql += " ORDER BY m.created_at DESC";
    const [rows] = await pool.query(sql);
    res.json(rows);
  } catch (err) {
    console.error("GET /:", err.message);
    res.status(500).json({ error: err.message });
  }
});

// — GET /api/modules/:id —————
router.get("/:id", async (req, res) => {
  try {
    const [[mod]] = await pool.query("SELECT * FROM modules WHERE id=?",
[req.params.id]);
    if (!mod) return res.status(404).json({ error: "Not found" });
    const [sections] = await pool.query(
      "SELECT * FROM module_sections WHERE module_id=? ORDER BY
sort_order", [req.params.id]);
  }
});

```

```

    const [quizzes] = await pool.query(
      "SELECT * FROM module_quizzes WHERE module_id=? ORDER BY
sort_order", [req.params.id]);
    res.json({ ...mod, sections, quizzes });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// — POST /api/modules (create) —————
router.post("/", upload.single("thumbnail"), async (req, res) => {
  const { title, description, category, icon, color,
    is_published, created_by, creator_name, sections, quizzes } =
req.body;

  if (!title) return res.status(400).json({ error: "Title required"
});
  if (!created_by) return res.status(400).json({ error: "created_by
required" });

  const thumbnail_url = req.file ? `/uploads/${req.file.filename}` : null;
  const parsedSections = parseJSON(sections, []);
  const parsedQuizzes = parseJSON(quizzes, []);
  const publishedVal = toBool(is_published);

  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const [r] = await conn.query(
      `INSERT INTO modules
(title,description,category,thumbnail_url,icon,color,is_published,created_
by,creator_name,lessons_count)
VALUES(?,?,?,?,?,?,?,?,?,?,?)`,
      [title, description||"", category||"General", thumbnail_url,
        icon||"

```



```

    for (let i = 0; i < parsedSections.length; i++) {
      const s = parsedSections[i];
      await conn.query(
        `INSERT INTO
module_sections(module_id,sort_order,tag,title,content,image_url,color,icon)
VALUES(?, ?, ?, ?, ?, ?, ?, ?)`,
        [mid, i, s.tag || "", s.title || "Slide " + (i + 1), s.content || "",
        s.image_url || "", s.color || color || "#3b82f6", s.icon || "📖"]
      );
    }

    for (let i = 0; i < parsedQuizzes.length; i++) {
      const q = parsedQuizzes[i];
      if (!q.question) continue;
      await conn.query(
        `INSERT INTO
module_quizzes(module_id,sort_order,question,option_a,option_b,option_c,option_d,correct_answer,explanation)
VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?)`,
        [mid, i, q.question, q.option_a || "", q.option_b || "",
q.option_c || "", q.option_d || "",
q.correct_answer || "A", q.explanation || ""]
      );
    }

    await conn.commit();
    res.status(201).json({ ok: true, moduleId: mid });
  } catch (err) {
    await conn.rollback();
    console.error("POST /:", err.message);
    res.status(500).json({ error: err.message });
  } finally {
    conn.release();
  }
});

// — PUT /api/modules/:id (update) —————
router.put("/:id", upload.single("thumbnail"), async (req, res) => {

```

```

const { id } = req.params;
const { title, description, category, icon, color,
      is_published, sections, quizzes } = req.body;

const parsedSections = parseJSON(sections, null);
const parsedQuizzes = parseJSON(quizzes, null);

const conn = await pool.getConnection();
try {
  await conn.beginTransaction();

  const fields = [], vals = [];
  if (title !== undefined) { fields.push("title=?");
vals.push(title); }
  if (description !== undefined) { fields.push("description=?");
vals.push(description); }
  if (category !== undefined) { fields.push("category=?");
vals.push(category); }
  if (icon !== undefined) { fields.push("icon=?");
vals.push(icon); }
  if (color !== undefined) { fields.push("color=?");
vals.push(color); }
  if (is_published !== undefined) { fields.push("is_published=?");
vals.push(toBool(is_published)); }
  if (req.file) {
fields.push("thumbnail_url=?");vals.push(`/uploads/${req.file.filename}`);
}

  if (fields.length) {
    vals.push(id);
    await conn.query(`UPDATE modules SET ${fields.join(",")} WHERE
id=?`, vals);
  }

  if (parsedSections !== null) {
    await conn.query("DELETE FROM module_sections WHERE module_id=?",
[id]);
    for (let i = 0; i < parsedSections.length; i++) {
      const s = parsedSections[i];
      await conn.query(

```

```

        `INSERT INTO
module_sections(module_id,sort_order,tag,title,content,image_url,color,icon)
        VALUES(?,?,?,?,?,?,?,?)`,
        [id,i, s.tag||"", s.title||"", s.content||"", s.image_url||"",
s.color||"#3b82f6", s.icon||"📖"]
    );
    }
    await conn.query("UPDATE modules SET lessons_count=? WHERE id=?",
[parsedSections.length, id]);
    }

    if (parsedQuizzes !== null) {
        await conn.query("DELETE FROM module_quizzes WHERE module_id=?",
[id]);
        for (let i = 0; i < parsedQuizzes.length; i++) {
            const q = parsedQuizzes[i];
            if (!q.question) continue;
            await conn.query(
                `INSERT INTO
module_quizzes(module_id,sort_order,question,option_a,option_b,option_c,option_d,correct_answer,explanation)
                VALUES(?,?,?,?,?,?,?,?)`,
                [id,i, q.question, q.option_a||"", q.option_b||"",
q.option_c||"", q.option_d||"",
                q.correct_answer||"A", q.explanation||""]
            );
        }
    }

    await conn.commit();
    res.json({ ok: true });
} catch (err) {
    await conn.rollback();
    res.status(500).json({ error: err.message });
} finally {
    conn.release();
}
});

```

```

// — PATCH /:id/publish —————
router.patch("/:id/publish", async (req, res) => {
  try {
    const val = toBool(req.body.is_published);
    await pool.query("UPDATE modules SET is_published=? WHERE id=?", [val,
req.params.id]);
    res.json({ ok: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// — DELETE /:id —————
router.delete("/:id", async (req, res) => {
  try {
    await pool.query("DELETE FROM modules WHERE id=?", [req.params.id]);
    res.json({ ok: true });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;

const express = require("express");
const router = express.Router();
const pool = require("../db");

// GET /api/students
router.get("/", async (req, res) => {
  try {
    const [rows] = await pool.query(
      `SELECT id, display_name AS name, email, created_at, role
      FROM users
      WHERE role = 'student'
      ORDER BY display_name ASC`
    );
    res.json(rows);
  } catch (err) {
    console.error("GET /api/students:", err.message);
  }
});

```

```
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
```

```
const express = require("express");
const router = express.Router();
const pool = require("../db");
const admin = require("../firebase-admin");

// GET /api/users/me
router.get("/me", async (req, res) => {
  const token = req.headers.authorization?.split("Bearer ")[1];

  if (!token) {
    return res.status(401).json({ error: "No token" });
  }

  try {
    const decoded = await admin.auth().verifyIdToken(token);

    const [rows] = await pool.query(
      `SELECT id, display_name AS name, email, role, created_at
      FROM users
      WHERE firebase_uid = ?`,
      [decoded.uid]
    );

    if (!rows.length) {
      return res.status(404).json({ error: "User not found" });
    }

    res.json(rows[0]);
  } catch (err) {
    console.error("AUTH ERROR:", err);
    res.status(401).json({
      error: "Invalid token: " + err.message
    });
  }
}
```

```
});
```

```
module.exports = router;
```

```
OPENROUTER_API_KEY=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
```

```
DB_HOST=mysql-1fdc0a77-disaster-project.i.aivencloud.com
```

```
DB_PORT=10480
```

```
DB_USER=avnadmin
```

```
DB_PASSWORD=AVNS_qxj7tZct8H1b55rn3EW # <-- use the rotated one
```

```
DB_NAME=defaultdb
```

```
DB_URL=mysql://avnadmin:AVNS_qxj7tZct8H1b55rn3EW@mysql-1fdc0a77-disaster-project.i.aivencloud.com:10480/defaultdb?ssl-mode=REQUIRED
```

```
PORT=5000
```

```
require("dotenv").config();
```

```
const mysql = require("mysql2/promise");
```

```
const pool = mysql.createPool({  
  host: process.env.DB_HOST,  
  port: Number(process.env.DB_PORT),  
  user: process.env.DB_USER,  
  password: process.env.DB_PASSWORD,  
  database: process.env.DB_NAME,
```

```
  waitForConnections: true,
```

```
  connectionLimit: 10,
```

```
  queueLimit: 0,
```

```
  ssl: {
```

```
    rejectUnauthorized: false,
```

```
  },
```

```
});
```

```
// Test connection
```

```

pool.getConnection()
  .then((connection) => {
    console.log("MySQL Connected (Aiven) ✅");
    connection.release();
  })
  .catch((err) => {
    console.error("MySQL Error ❌", err);
  });

module.exports = pool;

const admin = require('firebase-admin');

// Initialize Firebase Admin SDK
admin.initializeApp({
  projectId: "disasterready-e8597"
});

module.exports = admin;

const express = require("express");
const router = express.Router();

router.post("/generate-quiz", (req, res) => {
  const { role, classLevel, topic } = req.body;

  const questions = [
    {
      question: `What is ${topic}??`,
      options: ["A", "B", "C", "D"],
      answer: "A",
    },
  ];

  res.json({ questions });
});

module.exports = router;

const express = require("express");
const cors = require("cors");
const path = require("path");

```

```
require("dotenv").config();

// Import routes
const authRouter = require("./routes/auth");
const usersRouter = require("./routes/users");
const modulesRouter = require("./routes/modules");
const studentsRouter = require("./routes/students");
const assignmentsRouter = require("./routes/assignments");

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Static files for uploads
app.use("/uploads", express.static(path.join(__dirname, "uploads")));

// API Routes
app.use("/api/auth", authRouter);
app.use("/api/users", usersRouter);
app.use("/api/modules", modulesRouter);
app.use("/api/students", studentsRouter);
app.use("/api/assignments", assignmentsRouter);

// Health check
app.get("/api/health", (req, res) => {
  res.json({ status: "ok", message: "Server running" });
});

// Database check
app.get("/api/db-check", async (req, res) => {
  try {
    const pool = require("./db");
    const [tables] = await pool.query("SHOW TABLES");
    res.json({ message: "Connected", tables });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```



```

// Start server
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);

  // Test database connection
  const pool = require("./db");
  pool.query("SELECT 1")
    .then(() => console.log("MySQL Connected"))
    .catch((err) => console.error("MySQL Error:", err.message));
});

// src/components/pages/AdminDashboard.jsx
import React, { useState, useEffect, useCallback, useRef } from "react";
import { auth } from "../../firebase";
import { signOut } from "firebase/auth";
import { useNavigate } from "react-router-dom";
import axios from "axios";
import "./Dashboard.css";
import EarthquakeModule from "./EarthquakeModule";
import DesertificationModule from "./DesertificationModule";
import TsunamiModule from "./TsunamiModule";
import CreateModule from "./CreateModule";

const BASE = "http://localhost:5000";

const STATIC_MODULES = [
  { id:1, title:"Earthquake Preparedness", icon:"🌋", lessons:12,
color:"#f59e0b", isStatic:true },
  { id:5, title:"Tsunami Awareness", icon:"🌊", lessons:10,
color:"#06b6d4", isStatic:true, isNew:true },
  { id:7, title:"Desertification", icon:"🌵", lessons:8,
color:"#8b5cf6", isStatic:true },
];

export default function AdminDashboard() {
  const navigate = useNavigate();

  const [user, setUser] = useState(null);

```

```

const [activeTab, setActiveTab] = useState("overview");
const [search, setSearch] = useState("");
const [greeting, setGreeting] = useState("Good day");
const [quizHover, setQuizHover] = useState(false);

// DB data
const [dbModules, setDbModules] = useState([]);
const [moduleStats, setModuleStats] = useState([]);
const [students, setStudents] = useState([]); // ← real
registered students
const [assignments, setAssignments] = useState([]);

// Modals
const [earthquakeOpen, setEarthquakeOpen] = useState(false);
const [tsunamiOpen, setTsunamiOpen] = useState(false);
const [desertOpen, setDesertOpen] = useState(false);
const [createOpen, setCreateOpen] = useState(false);
const [editTarget, setEditTarget] = useState(null);
const [assignModal, setAssignModal] = useState(false);

// Assignment form state
const [asnTitle, setAsnTitle] = useState("");
const [asnDesc, setAsnDesc] = useState("");
const [asnModuleId, setAsnModuleId] = useState("");
const [asnDueDate, setAsnDueDate] = useState("");
const [asnStudentIds, setAsnStudentIds] = useState([]); // selected
student IDs
const [asnFile, setAsnFile] = useState(null);
const [asnLoading, setAsnLoading] = useState(false);
const [asnError, setAsnError] = useState("");
const fileInputRef = useRef(null);

// — fetch helpers —————
const fetchDbModules = useCallback(async () => {
  try {
    const res = await axios.get(`${BASE}/api/modules?role=admin`);
    setDbModules(res.data);
  } catch (e) { console.error("fetchDbModules:", e.message); }
}, []);

```

```

const fetchStats = useCallback(async () => {
  try {
    const res = await axios.get(`${BASE}/api/modules/stats`);
    setModuleStats(res.data);
  } catch (e) { console.error("fetchStats:", e.message); }
}, []);

// ← NEW: fetch real registered students
const fetchStudents = useCallback(async () => {
  try {
    const res = await axios.get(`${BASE}/api/students`);
    setStudents(res.data);
  } catch (e) { console.error("fetchStudents:", e.message); }
}, []);

// ← NEW: fetch assignments
const fetchAssignments = useCallback(async () => {
  try {
    const res = await axios.get(`${BASE}/api/assignments`);
    setAssignments(res.data);
  } catch (e) { console.error("fetchAssignments:", e.message); }
}, []);

useEffect(() => {
  const h = new Date().getHours();
  setGreeting(h < 12 ? "Good morning" : h < 17 ? "Good afternoon" :
"Good evening");
  const unsub = auth.onAuthStateChanged(setUser);
  fetchDbModules();
  fetchStats();
  fetchStudents();
  fetchAssignments();
  return unsub;
}, [fetchDbModules, fetchStats, fetchStudents, fetchAssignments]);

const handleModuleDone = () => {
  setCreateOpen(false);
  setEditTarget(null);
  fetchDbModules();
  fetchStats();

```

```

};

const togglePublish = async (mod, e) => {
  e.stopPropagation();
  const newVal = !mod.is_published;
  try {
    await axios.patch(`${BASE}/api/modules/${mod.id}/publish`, {
is_published: newVal });
    setDbModules(prev => prev.map(m => m.id === mod.id ? { ...m,
is_published: newVal } : m));
  } catch (err) {
    alert("Failed to update: " + (err.response?.data?.error ||
err.message));
  }
};

const deleteModule = async (mod, e) => {
  e.stopPropagation();
  if (!window.confirm(`Delete "${mod.title}"? This cannot be undone.`))
return;
  try {
    await axios.delete(`${BASE}/api/modules/${mod.id}`);
    setDbModules(prev => prev.filter(m => m.id !== mod.id));
  } catch (err) {
    alert("Delete failed: " + (err.response?.data?.error ||
err.message));
  }
};

// — Submit Assignment —————
const handleAssignSubmit = async () => {
  setAsnError("");
  if (!asnTitle.trim()) return setAsnError("Title is
required.");
  if (asnStudentIds.length === 0) return setAsnError("Select at least
one student.");
  if (!asnModuleId && !asnFile) return setAsnError("Attach a file or
select a module.");

  setAsnLoading(true);

```

```

try {
  const fd = new FormData();
  fd.append("title",      asnTitle.trim());
  fd.append("description", asnDesc.trim());
  fd.append("student_ids", JSON.stringify(asnStudentIds));
  if (asnModuleId) fd.append("module_id", asnModuleId);
  if (asnDueDate)   fd.append("due_date",  asnDueDate);
  if (asnFile)      fd.append("file",      asnFile);

  await axios.post(`${BASE}/api/assignments`, fd, {
    headers: { "Content-Type": "multipart/form-data" },
  });

  // Reset form
  setAsnTitle(""); setAsnDesc(""); setAsnModuleId("");
  setAsnDueDate(""); setAsnStudentIds([]); setAsnFile(null);
  if (fileInputRef.current) fileInputRef.current.value = "";
  setAssignModal(false);
  fetchAssignments();
} catch (err) {
  setAsnError(err.response?.data?.error || err.message);
} finally {
  setAsnLoading(false);
}
};

const deleteAssignment = async (id) => {
  if (!window.confirm("Delete this assignment?")) return;
  try {
    await axios.delete(`${BASE}/api/assignments/${id}`);
    setAssignments(prev => prev.filter(a => a.id !== id));
  } catch (err) {
    alert("Delete failed: " + err.message);
  }
};

const toggleStudent = (id) => {
  setAsnStudentIds(prev =>
    prev.includes(id) ? prev.filter(x => x !== id) : [...prev, id]
  );
};

```

```

};

// — Helpers —————
const statFor      = (id) => moduleStats.find(s => Number(s.id) ===
Number(id)) || { enrolled:0, avg_progress:0 };
const allModulesForOverview = () => {
  const statics = STATIC_MODULES.map(m => {
    const s = statFor(m.id);
    return { ...m, enrolled: s.enrolled, avgScore:
Math.round(s.avg_progress||0) };
  });
  const dynamics = dbModules
    .filter(dm => !STATIC_MODULES.find(sm => sm.id === dm.id))
    .map(dm => {
      const s = statFor(dm.id);
      return { ...dm, lessons: dm.lessons_count||0, enrolled:
Number(s.enrolled)||0, avgScore: Math.round(s.avg_progress||0), isDynamic:
true };
    });
  return [...statics, ...dynamics];
};

const overviewModules      = allModulesForOverview();
const publishedDynamic      = dbModules.filter(dm =>
!STATIC_MODULES.find(sm => sm.id === dm.id) && dm.is_published);
const draftDynamic          = dbModules.filter(dm =>
!STATIC_MODULES.find(sm => sm.id === dm.id) && !dm.is_published);
const allDynamic            = dbModules.filter(dm =>
!STATIC_MODULES.find(sm => sm.id === dm.id));
const firstName             = user?.displayName?.split(" ")[0] ||
user?.email?.split("@")[0] || "Admin";
const filteredStudents      = students.filter(s =>
  s.name?.toLowerCase().includes(search.toLowerCase()) ||
  s.email?.toLowerCase().includes(search.toLowerCase())
);

const activeToday = students.filter(s => {
  if (!s.last_active) return false;
  const d = new Date(s.last_active);
  const t = new Date();
  return d.toDateString() === t.toDateString();
}).length;

```

```

const allPublishedModules = [
  ...STATIC_MODULES,
  ...dbModules.filter(dm => !STATIC_MODULES.find(sm => sm.id === dm.id)
&& dm.is_published),
];

const NAV_ITEMS = [
  { id:"overview", label:"Dashboard", icon:"M3 13h8V3H3v10zm0
8h8v-6H3v6zm10 0h8V11h-8v10zm0-18v6h8V3h-8z" },
  { id:"modules", label:"Modules", icon:"M21
5c-1.11-.35-2.33-.5-3.5-.5-1.95 0-4.05.4-5.5
1.5-1.45-1.1-3.55-1.5-5.5-1.5S2.45 4.9 1 6v14.65c0 .25.25.5.5.1 0
.15-.05.25-.05C3.1 20.45 5.05 20 6.5 20c1.95 0 4.05.4 5.5 1.5 1.35-.85
3.8-1.5 5.5-1.5 1.65 0 3.35.3 4.75 1.05.1.05.15.05.25.05.25 0
.5-.25.5-.5V6c-.6-.45-1.25-.75-2-1z" },
  { id:"assignments", label:"Assignments", icon:"M19 3H5c-1.1 0-2 .9-2
2v14c0 1.1.9 2 2 2h14c1.1 0 2-.9 2-2V5c0-1.1-.9-2-2-2zm-7 3c1.93 0 3.5
1.57 3.5 3.5S13.93 13 12 13s-3.5-1.57-3.5-3.5S10.07 6 12 6zm7
13H5v-.23c0-.62.28-1.2.76-1.58C7.47 15.82 9.64 15 12 15s4.53.82 6.24
2.19c.48.38.76.97.76 1.58V19z" },
];

const openStatic = (id) => {
  if (id===1) setEarthquakeOpen(true);
  if (id===5) setTsunamiOpen(true);
  if (id===7) setDesertOpen(true);
};

const statusColor = (status) => {
  if (status === "completed") return { bg:"#10b98122", color:"#10b981"
};
  if (status === "in_progress") return { bg:"#3b82f622", color:"#3b82f6"
};
  return { bg:"#f59e0b22", color:"#f59e0b" };
};

return (
  <div className="dash-root">

```

```

        {earthquakeOpen && <EarthquakeModule      onClose={() =>
setEarthquakeOpen(false)} />}
        {tsunamiOpen      && <TsunamiModule      onClose={() =>
setTsunamiOpen(false)} />}
        {desertOpen      && <DesertificationModule  onClose={() =>
setDesertOpen(false)} />}
        {createOpen      && <CreateModule editData={editTarget}
onDone={handleModuleDone} />}

    { /* ===== ASSIGN MODAL ===== */ }
    {assignModal && (
      <div style={{
        position:"fixed",inset:0,zIndex:1000,

background:"rgba(0,0,0,0.7)",display:"flex",alignItems:"center",justifyCon
tent:"center",
        padding:20
      }}>
        <div style={{
          background:"#0f1117",border:"1px solid
#1e2535",borderRadius:16,

width:"100%",maxWidth:620,maxHeight:"90vh",overflowY:"auto",padding:32
        }}>
          <div
style={{display:"flex",justifyContent:"space-between",alignItems:"center",
marginBottom:24}}>
            <h2
style={{color:"#f1f5f9",fontSize:20,fontWeight:700,margin:0}}>📋 New
Assignment</h2>
            <button onClick={() => setAssignModal(false)}

style={{background:"none",border:"none",color:"#64748b",fontSize:22,cursor
:"pointer"}}>X</button>
          </div>

          {asnError && (
            <div style={{background:"#ef444422",border:"1px solid
#ef4444",borderRadius:8,

```



```

        padding:"10px
14px",color:"#ef4444",marginBottom:16,fontSize:13}}>
        {asnError}
    </div>
    )}

    {/* Title */}
    <label
style={{color:"#94a3b8",fontSize:12,fontWeight:600,display:"block",marginB
ottom:6}}>
        ASSIGNMENT TITLE *
    </label>
    <input value={asnTitle}
onChange={e=>setAsnTitle(e.target.value)}
        placeholder="e.g. Read Earthquake Safety Guide"
        style={{
            width:"100%",background:"#1e2535",border:"1px solid
#2d3748",borderRadius:8,
            padding:"10px
14px",color:"#f1f5f9",fontSize:14,marginBottom:16,boxSizing:"border-box"
        }}/>

    {/* Description */}
    <label
style={{color:"#94a3b8",fontSize:12,fontWeight:600,display:"block",marginB
ottom:6}}>
        DESCRIPTION
    </label>
    <textarea value={asnDesc}
onChange={e=>setAsnDesc(e.target.value)}
        placeholder="Instructions or notes for students..." rows={3}
        style={{
            width:"100%",background:"#1e2535",border:"1px solid
#2d3748",borderRadius:8,
            padding:"10px
14px",color:"#f1f5f9",fontSize:14,marginBottom:16,
            resize:"vertical",boxSizing:"border-box"
        }}/>

    {/* Link to existing module OR upload file */}

```

```

        <div style={{display:"grid",gridTemplateColumns:"1fr
1fr",gap:16,marginBottom:16}}>
            <div>
                <label
style={{color:"#94a3b8",fontSize:12,fontWeight:600,display:"block",marginB
ottom:6}}>
                    LINK MODULE (optional)
                </label>
                <select value={asnModuleId}
onChange={e=>setAsnModuleId(e.target.value)}
                    style={{
                        width:"100%",background:"#1e2535",border:"1px solid
#2d3748",borderRadius:8,
                        padding:"10px 14px",color:
asnModuleId?"#f1f5f9":"#64748b",fontSize:14,
                        boxSizing:"border-box"
                    }}>
                    <option value="">- none </option>
                    {allPublishedModules.map(m => (
                        <option key={m.id} value={m.id}>{m.icon}
{m.title}</option>
                    ))}
                </select>
            </div>
            <div>
                <label
style={{color:"#94a3b8",fontSize:12,fontWeight:600,display:"block",marginB
ottom:6}}>
                    DUE DATE (optional)
                </label>
                <input type="date" value={asnDueDate}
onChange={e=>setAsnDueDate(e.target.value)}
                    style={{
                        width:"100%",background:"#1e2535",border:"1px solid
#2d3748",borderRadius:8,
                        padding:"10px
14px",color:"#f1f5f9",fontSize:14,boxSizing:"border-box"
                    }}/>
            </div>
        </div>
    </div>

```

```

        {/* File upload */}
        <label
style={{color:"#94a3b8",fontSize:12,fontWeight:600,display:"block",marginBottom:6}}>
            UPLOAD FILE - PDF / PPT / DOC (optional)
        </label>
        <div
            onClick={() => fileInputRef.current?.click()}
            style={{
                border:"2px dashed #2d3748",borderRadius:8,padding:"20px",
                textAlign:"center",cursor:"pointer",marginBottom:16,
                background: asnFile?"#10b98111":"transparent",
                borderColor: asnFile?"#10b981":"#2d3748",
                transition:"all 0.2s"
            }}>
            {asnFile ? (
                <div>
                    <div style={{fontSize:28}}>
                        {asnFile.name.endsWith(".pdf") ? "📄" :
asnFile.name.match(/\.(pptx?$/i) ? "📊" : "📝"}
                    </div>
                    <p style={{color:"#10b981",fontSize:13;margin:"6px 0 0 0"}}>{asnFile.name}</p>
                    <p style={{color:"#64748b",fontSize:11;margin:"2px 0 0 0"}}>
                        {(asnFile.size / 1024).toFixed(1)} KB
                    </p>
                </div>
            ) : (
                <div>
                    <div style={{fontSize:28}}>📁</div>
                    <p style={{color:"#64748b",fontSize:13;margin:"6px 0 0 0"}}>
                        Click to upload PDF, PPT, or DOC
                    </p>
                    <p style={{color:"#475569",fontSize:11;margin:"2px 0 0 0"}}>Max 50 MB</p>
                </div>
            ) }
        </div>

```

```

        <input
          ref={fileInputRef} type="file"
          accept=".pdf,.ppt,.pptx,.doc,.docx"
          style={{display:"none"}}
          onChange={e => setAsnFile(e.target.files[0] || null)}
        />
      </div>

      { /* Select Students */ }
      <label
style={{color:"#94a3b8",fontSize:12,fontWeight:600,display:"block",marginBottom:8}}>
        ASSIGN TO STUDENTS * ({asnStudentIds.length} selected)
      </label>

      { /* Select All / Clear */ }
      <div style={{display:"flex",gap:8,marginBottom:8}}>
        <button onClick={() =>
setAsnStudentIds(students.map(s=>s.id)) }
          style={{background:"#1e2535",border:"1px solid
#2d3748",borderRadius:6,
            padding:"4px
12px",color:"#94a3b8",fontSize:12,cursor:"pointer"}}>
          Select All ({students.length})
        </button>
        <button onClick={() => setAsnStudentIds([])}
          style={{background:"#1e2535",border:"1px solid
#2d3748",borderRadius:6,
            padding:"4px
12px",color:"#94a3b8",fontSize:12,cursor:"pointer"}}>
          Clear
        </button>
      </div>

      <div style={{
        background:"#1e2535",border:"1px solid
#2d3748",borderRadius:8,
        maxHeight:200,overflowY:"auto",marginBottom:20
      }}>
        {students.length === 0 ? (

```

```

        <p
style={{color:"#475569",padding:16,textAlign:"center",fontSize:13}}>
        No registered students found.
    </p>
    ) : students.map(s => (
        <label key={s.id}
            style={{
display:"flex",alignItems:"center",gap:10,padding:"10px 14px",
            cursor:"pointer",borderBottom:"1px solid #0f1117",
            background:
asnStudentIds.includes(s.id)?"#3b82f611":"transparent",
            transition:"background 0.15s"
        }}>
            <input type="checkbox"
checked={asnStudentIds.includes(s.id)}
            onChange={() => toggleStudent(s.id)}
            style={{accentColor:"#3b82f6",width:15,height:15}}/>
            <div style={{
                width:30,height:30,borderRadius:"50%",
                background:"#3b82f622",color:"#3b82f6",
display:"flex",alignItems:"center",justifyContent:"center",
                fontSize:13,fontWeight:700,flexShrink:0
            }}>
                { (s.name||s.email||" ?") [0].toUpperCase() }
            </div>
            <div>
                <p
style={{color:"#f1f5f9",fontSize:13,fontWeight:600,margin:0}}>{s.name ||
                "—"}</p>
                <p
style={{color:"#64748b",fontSize:11,margin:0}}>{s.email}</p>
            </div>
            </label>
        ) ) }
    </div>

    { /* Actions */ }
    <div style={{display:"flex",gap:10}}>

```

```

        <button onClick={() => setAssignModal(false)}
            style={{
                flex:1,padding:"11px",background:"none",border:"1px
solid #2d3748",

borderRadius:8,color:"#64748b",cursor:"pointer",fontSize:14
            }}>
            Cancel
        </button>
        <button onClick={handleAssignSubmit} disabled={asnLoading}
            style={{
                flex:2,padding:"11px",
                background:
asnLoading?"#1e2535":"linear-gradient(135deg,#3b82f6,#8b5cf6)",
border:"none",borderRadius:8,color:"#fff",fontWeight:700,
                cursor: asnLoading?"not-allowed":"pointer",fontSize:14
            }}>
            {asnLoading ? "Assigning..." : `🚀 Assign to
${asnStudentIds.length} student${asnStudentIds.length!==1?"s":""}`}
        </button>
    </div>
</div>
</div>
)}

{/* — Sidebar — */}
<aside className="dash-sidebar admin-sidebar">
    <div className="dash-logo">
        <svg width="28" height="28" viewBox="0 0 32 32" fill="none">
            <path d="M16 2L28 9v14L16 30 4 23V9L16 2z"
fill="url(#hexGA)"/>
            <circle cx="16" cy="16" r="3" fill="white"/>
            <defs>
                <linearGradient id="hexGA" x1="4" y1="2" x2="28" y2="30"
gradientUnits="userSpaceOnUse">
                    <stop stopColor="#f472b6"/><stop offset="1"
stopColor="#8b5cf6"/>
                </linearGradient>
            </defs>

```

```

        </svg>
        <span>DisasterReady</span>
      </div>
      <div className="admin-badge-pill">Admin Panel</div>
      <nav className="dash-nav">
        {NAV_ITEMS.map(item => (
          <button key={item.id}
            className={`dash-nav-item ${activeTab===item.id?"active
admin-active":""}`}
            onClick={() => setActiveTab(item.id)}>
            <svg width="18" height="18" viewBox="0 0 24 24"
fill="currentColor"><path d={item.icon}/></svg>
            {item.label}
            {item.id==="assignments" && assignments.length > 0 && (
              <span style={{
                marginLeft:"auto",background:"#3b82f6",color:"#fff",
                fontSize:10,fontWeight:700,borderRadius:10,padding:"1px
7px"
              }}>{assignments.length}</span>
            )}
          </button>
        ))}
      </nav>
      <div className="dash-sidebar-footer">
        <div className="dash-user-pill">
          <div className="dash-avatar
admin-avatar">{firstName[0].toUpperCase()}</div>
          <div className="dash-user-info">
            <span className="dash-user-name">{firstName}</span>
            <span className="dash-user-role">Administrator</span>
          </div>
        </div>
        <button className="dash-logout" onClick={async () => { await
signOut(auth); localStorage.removeItem("role"); navigate("/login"); }}>
          <svg width="18" height="18" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M17 7l-1.41 1.41L18.17 11H8v2h10.17l-2.58 2.58L17
17l5-5zM4 5h8V3H4c-1.1 0-2 .9-2 2v14c0 1.1 9 2 2 2h8v-2H4V5z"/>
          </svg>
        </button>
      </div>
    </div>
  </div>
</div>

```

```

    </div>
  </aside>

  { /* — Main — */ }
  <main className="dash-main">
    { /* Topbar */ }
    <div className="dash-topbar">
      <div>
        <h2 className="dash-welcome">{greeting}, {firstName} 🛡️</h2>
        <p className="dash-date">{new
Date().toLocaleDateString("en-US", {weekday:"long", month:"long", day:"numeri
c"})}</p>
      </div>
      <div className="dash-topbar-actions">
        <div className="dash-search">
          <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M15.5 14h-.791-.28-.27A6.471 6.471 0 0016 9.5 6.5
6.5 0 109.5 16c1.61 0 3.09-.59 4.23-1.571.27.28v.7915 4.99L20.49
191-4.99-5zm-6 0C7.01 14 5 11.99 5 9.5S7.01 5 9.5 5 14 7.01 14 9.5 11.99
14 9.5 14z"/>
          </svg>
          <input placeholder="Search students..." value={search}
onChange={e=>setSearch(e.target.value)} />
        </div>
        {activeTab === "assignments" ? (
          <button className="btn-admin-action" onClick={() =>
setAssignModal(true)}>
            <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor"><path d="M19 13h-6v6h-2v-6H5v-2h6V5h2v6h6v2z"/></svg>
            New Assignment
          </button>
        ) : (
          <button className="btn-admin-action" onClick={() => {
setEditTarget(null); setCreateOpen(true); }}>
            <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor"><path d="M19 13h-6v6h-2v-6H5v-2h6V5h2v6h6v2z"/></svg>
            Add Module
          </button>
        ) }
      </div>
    </div>
  </main>

```



```

    </div>
  </div>

  { /* ===== OVERVIEW TAB ===== */
  {activeTab === "overview" && (
    <div className="dash-content">
      <div className="stat-grid">
        [
          { label:"Total Students",      value:students.length,
color:"#8b5cf6", icon:"M16 11c1.66 0 2.99-1.34 2.99-3S17.66 5 16 5c-1.66
0-3 1.34-3 3s1.34 3 3 3zm-8 0c1.66 0 2.99-1.34 2.99-3S9.66 5 8 5C6.34 5 5
6.34 5 8s1.34 3 3 3zm0 2c-2.33 0-7 1.17-7
3.5V19h14v-2.5c0-2.33-4.67-3.5-7-3.5zm8 0c-.29 0-.62.02-.97.05 1.16.84
1.97 1.97 1.97 3.45V19h6v-2.5c0-2.33-4.67-3.5-7-3.5z" },
          { label:"Active Today",        value:activeToday,
color:"#10b981", icon:"M12 2C6.48 2 2 6.48 2 12s4.48 10 10 10-4.48
10-10S17.52 2 12 2zm-2 14.5v-9l6 4.5-6 4.5z" },
          { label:"Total Modules",        value:overviewModules.length,
color:"#3b82f6", icon:"M21 5c-1.11-.35-2.33-.5-3.5-.5-1.95 0-4.05.4-5.5
1.5-1.45-1.1-3.55-1.5-5.5-1.5S2.45 4.9 1 6v14.65c0 .25.25.5.5.5.1 0
.15-.05.25-.05C3.1 20.45 5.05 20 6.5 20c1.95 0 4.05.4 5.5 1.5 1.35-.85
3.8-1.5 5.5-1.5 1.65 0 3.35.3 4.75 1.05.1.05.15.05.25.05.25 0
.5-.25.5-.5V6c-.6-.45-1.25-.75-2-1z" },
          { label:"Assignments Sent",    value:assignments.length,
color:"#f59e0b", icon:"M9 16.17L4.83 12l-1.42 1.41L9 19 21 7l-1.41-1.41L9
16.17z" },
        ].map(stat => (
          <div key={stat.label} className="stat-card">
            <div className="stat-icon"
style={{background:stat.color+"22",color:stat.color}}>
              <svg width="20" height="20" viewBox="0 0 24 24"
fill="currentColor"><path d={stat.icon}/></svg>
            </div>
            <div>
              <p className="stat-label">{stat.label}</p>
              <p className="stat-value">{stat.value}</p>
            </div>
          </div>
        )))
      </div>
    </div>
  )
}

```

```

    { /* AI Quiz banner */ }
    <div className={`quiz-banner-admin ${quizHover?"hovered":""}`}
      onClick={() => navigate("/quiz-generator")}
      onMouseEnter={() => setQuizHover(true)}
      onMouseLeave={() => setQuizHover(false)}>
      <div className="qba-glow"/><div className="qba-grid"/>
      <div className="qba-left">
        <div className="qba-icon-cluster">
          <div className="qba-icon-main">
            <svg width="28" height="28" viewBox="0 0 24 24"
fill="currentColor"><path d="M7 2v11h3v9l7-12h-4l4-8z"/></svg>
          </div>
          <div className="qba-icon-ring"/><div
className="qba-icon-ring qba-ring-2"/>
          </div>
          <div>
            <div className="qba-badge"><span
className="qba-badge-dot"/>ADMIN TOOL · AI POWERED</div>
            <h3 className="qba-title">Create AI Quiz for
Students</h3>
            <p className="qba-desc">Generate personalized
disaster-preparedness quizzes instantly</p>
          </div>
        </div>
        <button className="qba-btn" type="button">
          <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor"><path d="M19 13h-6v6h-2v-6H5v-2h6V5h2v6h6v2z"/></svg>
          Create Quiz
        </button>
      </div>

    { /* Student table - real registered students */ }
    <div className="section-card">
      <h3 className="section-title">Registered Students
({students.length})</h3>
      {students.length === 0 ? (
        <div style={{textAlign:"center",padding:"30px
20px",color:"#475569"}}>
          <div style={{fontSize:40}}><img alt="No students icon" data-bbox="580 885 605 900"/></div>

```

```

        <p style={{marginTop:10}}>No students registered
yet.</p>

    </div>
  ) : (
    <div className="student-table-wrap">
      <table className="student-table">
        <thead>

<tr><th>Student</th><th>Joined</th><th>Assignments</th><th>Status</th></tr>
<

        </thead>
        <tbody>
          {filteredStudents.map(s => {
            // Count assignments for this student
            const myAssignments = assignments.flatMap(a =>
              (a.students || []).filter(st => st.student_id
=== s.id)

            );
            const done = myAssignments.filter(x => x.status
=== "completed").length;
            return (
              <tr key={s.id}>
                <td>
                  <div className="student-cell">
                    <div
className="student-avatar">{(s.name||s.email||"?")[0].toUpperCase()}</div>
                    <div>
                      <p className="student-name">{s.name ||
"–"}</p>
                      <p
className="student-email">{s.email}</p>
                    </div>
                  </div>
                </td>
                <td>
                  <span className="last-active">
                    {s.created_at ? new
Date(s.created_at).toLocaleDateString() : "–"}
                  </span>
                </td>

```

```

        <td>
            <span style={{color:"#f1f5f9",fontSize:13}}>
                {done}/{myAssignments.length} done
            </span>
        </td>
        <td>
            <span className={`status-pill ${s.role ===
"student" ? "active" : "inactive"}}`>
                {s.role === "student" ? "Student" :
s.role}
            </span>
        </td>
    </tr>
    );
    })}
</tbody>
</table>
</div>
    )}
</div>
</div>
)}

{/* ===== MODULES TAB ===== */}
{activeTab === "modules" && (
    <div className="dash-content">
        <div className="section-card" style={{marginBottom:28}}>
            <div
style={{display:"flex",justifyContent:"space-between",alignItems:"center",
marginBottom:20}}>
                <div>
                    <h3 className="section-title" style={{margin:0}}>Your
Created Modules</h3>
                    <p style={{color:"#64748b",fontSize:13,marginTop:4}}>
                        {publishedDynamic.length} published ·
{draftDynamic.length} draft
                    </p>
                </div>
                <button className="btn-admin-action" onClick={() => {
setEditTarget(null); setCreateOpen(true); }}>

```

```

        <svg width="15" height="15" viewBox="0 0 24 24"
fill="currentColor"><path d="M19 13h-6v6h-2v-6H5v-2h6V5h2v6h6v2z"/></svg>
        New Module
      </button>
    </div>

    {allDynamic.length === 0 ? (
      <div style={{textAlign:"center",padding:"40px
20px",color:"#475569"}}>
        <div style={{fontSize:48,marginBottom:12}}><img alt="No modules icon" data-bbox="748 275 765 290"/></div>
        <p
style={{fontSize:16,fontWeight:600,color:"#64748b"}}>No modules created
yet</p>
        <p style={{fontSize:13,marginTop:4}}>Click "New Module"
to create your first one</p>
        <button className="btn-admin-action"
style={{margin:"16px auto 0"}}
onClick={() => { setEditTarget(null);
setCreateOpen(true); }}>
          + Create First Module
        </button>
      </div>
    ) : (
      <div className="modules-grid">
        {allDynamic.map(dm => {
          const s = statFor(dm.id);
          const viewers = Number(s.enrolled)||0;
          const avgScore = Math.round(s.avg_progress||0);
          return (
            <div key={dm.id} className="module-card"
              style={{border:`1px solid
${dm.color||"#3b82f6"}33`}}>
              <div className="module-card-header"
style={{background:(dm.color||"#3b82f6")+"22"}}>
                <span
style={{fontSize:36}}>{dm.icon||"<img alt='Module icon' data-bbox='455 805 475 820'"}/></span>
                <span className="module-badge" style={{
                  background:
dm.is_published?"#10b98122":"#ffffff11",

```



```

        <p
style={{fontSize:11,color:"#64748b",margin:0}}>Slides</p>
    </div>
</div>
<div
style={{display:"flex",gap:8,marginTop:10}}>
    <button className="module-cta"
style={{flex:1,borderColor:dm.is_published?"#ef4444":"#10b981",color:dm.is
_published?"#ef4444":"#10b981",fontWeight:700}}
onClick={e=>togglePublish(dm,e)}>
        {dm.is_published?"🛑 Unpublish":"🚀
Publish"}
    </button>
    <button className="module-cta"
style={{borderColor:dm.color||"#3b82f6",color:dm.color||"#3b82f6"}}
onClick={e=>{e.stopPropagation();setEditTarget(dm);setCreateOpen(true);}}>
🖋️</button>
    <button className="module-cta"
style={{borderColor:"#ef4444",color:"#ef4444"}}
onClick={e=>deleteModule(dm,e)}>🗑️</button>
    </div>
</div>
</div>
    );
    })}
</div>
    )}
</div>

    { /* Built-in modules */ }
    <div className="section-card">
        <h3 className="section-title"
style={{marginBottom:16}}>Built-in Modules</h3>
        <div className="modules-grid">
            {STATIC_MODULES.map(m => {
                const s = statFor(m.id);
                const viewers = Number(s.enrolled)||0;
                const avgScore = Math.round(s.avg_progress||0);
                return (

```

```

        <div key={m.id} className="module-card"
style={{cursor:"pointer",...(m.isNew?{border:`1px solid
${m.color}44`:{}})}} onClick={()=>openStatic(m.id)}>
        <div className="module-card-header"
style={{background:m.color+"22"}}>
            <span style={{fontSize:36}}>{m.icon}</span>
            <div
style={{display:"flex",flexDirection:"column",alignItems:"flex-end",gap:4}
}>
                {m.isNew&&<span
style={{fontSize:10,color:m.color,background:m.color+"22",border:`1px
solid ${m.color}44`,borderRadius:20,padding:"2px
8px",fontWeight:700}}>NEW</span>}
                <span className="module-badge"
style={{background:"#10b98122",color:"#10b981"}}>✔ Published</span>
            </div>
        </div>
        <div className="module-card-body">
            <h4 className="module-card-title">{m.title}</h4>
            <p className="module-card-meta">{m.lessons}
lessons · Built-in</p>
            <div style={{display:"flex",gap:16,margin:"12px 0
4px",padding:"10px 12px",background:"#0f1117",borderRadius:8,border:"1px
solid #1e2535"}}>
                <div style={{textAlign:"center"}}><p
style={{fontSize:18,fontWeight:700,color:m.color,margin:0}}>{viewers}</p><
p style={{fontSize:11,color:"#64748b",margin:0}}>Students</p></div>
                <div style={{width:1,background:"#1e2535"}}/>
                <div style={{textAlign:"center"}}><p
style={{fontSize:18,fontWeight:700,color:avgScore>=70?"#10b981":avgScore>0
?"#f59e0b":"#475569",margin:0}}>{avgScore>0?`${avgScore}%`:"-"}</p><p
style={{fontSize:11,color:"#64748b",margin:0}}>Avg Progress</p></div>
                <div style={{width:1,background:"#1e2535"}}/>
                <div style={{textAlign:"center"}}><p
style={{fontSize:18,fontWeight:700,color:"#94a3b8",margin:0}}>{m.lessons}<
/p><p style={{fontSize:11,color:"#64748b",margin:0}}>Lessons</p></div>
            </div>
            <button className="module-cta"
style={{borderColor:m.color,color:m.color,marginTop:10}}>

```



```

onClick={e=>{e.stopPropagation();openStatic(m.id);}}>View Module
</button>

        </div>
      </div>
    );
  })}
</div>
</div>
</div>
)}

{/* ===== ASSIGNMENTS TAB ===== */}
{activeTab === "assignments" && (
  <div className="dash-content">

    {/* Summary bar */}
    <div className="stat-grid" style={{marginBottom:24}}>
      [
        { label:"Total Assignments", value:assignments.length,
color:"#3b82f6" },
        { label:"Total Assigned",
value:assignments.reduce((acc,a)=>acc+Number(a.total_assigned||0),0),
color:"#8b5cf6" },
        { label:"Completed",
value:assignments.reduce((acc,a)=>acc+Number(a.completed_count||0),0),
color:"#10b981" },
        { label:"Avg Progress",      value:assignments.length ?
Math.round(assignments.reduce((acc,a)=>acc+Number(a.avg_progress||0),0)/as
signments.length)+"%":"-", color:"#f59e0b" },
      ].map(stat => (
        <div key={stat.label} className="stat-card">
          <div className="stat-icon"
style={{background:stat.color+"22",color:stat.color,fontSize:20,display:"f
lex",alignItems:"center",justifyContent:"center",width:40,height:40,border
Radius:10}}>

          </div>
          <div>
            <p className="stat-label">{stat.label}</p>
            <p className="stat-value">{stat.value}</p>
          </div>
        </div>
      ))
    </div>
  </div>
)
}

```

```

        </div>
      </div>
    )))
  </div>

  {assignments.length === 0 ? (
    <div className="section-card"
      style={{textAlign:"center",padding:"60px 20px"}}>
      <div style={{fontSize:56,marginBottom:16}}><img alt="Clipboard icon" data-bbox="728 254 743 269"/></div>
      <h3 style={{color:"#f1f5f9",margin:"0 0 8px"}}>No
Assignments Yet</h3>
      <p style={{color:"#64748b",fontSize:14}}>Create your first
assignment to track student progress.</p>
      <button className="btn-admin-action" style={{margin:"16px
auto 0"}} onClick={()=>setAssignModal(true)}>
        + Create First Assignment
      </button>
    </div>
  ) : (
    <div style={{display:"flex",flexDirection:"column",gap:20}}>
      {assignments.map(asn => {
        const total      = Number(asn.total_assigned) || 0;
        const done       = Number(asn.completed_count) || 0;
        const pct        = total > 0 ?
Math.round((done/total)*100) : 0;
        const avgProg    = Number(asn.avg_progress) || 0;
        return (
          <div key={asn.id} className="section-card"
style={{padding:24}}>
            {/* Header row */}
            <div
style={{display:"flex",justifyContent:"space-between",alignItems:"flex-sta
rt",marginBottom:16}}>
              <div style={{flex:1}}>
                <div
style={{display:"flex",alignItems:"center",gap:10,flexWrap:"wrap"}}>
                  <h3
style={{color:"#f1f5f9",fontWeight:700,fontSize:17,margin:0}}>{asn.title}<
/h3>
                  {asn.file_name && (

```

```

        <span
style={{background:"#3b82f622",color:"#3b82f6",fontSize:11,padding:"2px
8px",borderRadius:10,border:"1px solid #3b82f633"}}>
             {asn.file_name}
        </span>
    )}
    {asn.module_id && (
        <span
style={{background:"#8b5cf622",color:"#8b5cf6",fontSize:11,padding:"2px
8px",borderRadius:10,border:"1px solid #8b5cf633"}}>
             Module #{asn.module_id}
        </span>
    )}
    {asn.due_date && (
        <span
style={{background:"#f59e0b22",color:"#f59e0b",fontSize:11,padding:"2px
8px",borderRadius:10}}>
             Due {new
Date(asn.due_date).toLocaleDateString()}
        </span>
    )}
</div>
    {asn.description && (
        <p
style={{color:"#64748b",fontSize:13,margin:"6px 0
0"}}>{asn.description}</p>
    )}
</div>
    <button onClick={()=>deleteAssignment(asn.id)}
style={{background:"none",border:"1px solid
#ef444444",borderRadius:6,
color:"#ef4444",fontSize:12,padding:"4px
10px",cursor:"pointer",marginLeft:12,flexShrink:0}}>
         Delete
    </button>
</div>

    { /* Progress summary */ }
    <div
style={{display:"flex",gap:16,marginBottom:16,flexWrap:"wrap"}}>

```

```

<div style={{background:"#0f1117",border:"1px
solid #1e2535",borderRadius:8,padding:"10px 16px",flex:1,minWidth:100}}>
  <p
style={{fontSize:20,fontWeight:700,color:"#3b82f6",margin:0}}>{total}</p>
  <p
style={{fontSize:11,color:"#64748b",margin:0}}>Assigned</p>
</div>
<div style={{background:"#0f1117",border:"1px
solid #1e2535",borderRadius:8,padding:"10px 16px",flex:1,minWidth:100}}>
  <p
style={{fontSize:20,fontWeight:700,color:"#10b981",margin:0}}>{done}</p>
  <p
style={{fontSize:11,color:"#64748b",margin:0}}>Completed</p>
</div>
<div style={{background:"#0f1117",border:"1px
solid #1e2535",borderRadius:8,padding:"10px 16px",flex:1,minWidth:100}}>
  <p
style={{fontSize:20,fontWeight:700,color:"#f59e0b",margin:0}}>{total-done}
</p>
  <p
style={{fontSize:11,color:"#64748b",margin:0}}>Pending</p>
</div>
<div style={{background:"#0f1117",border:"1px
solid #1e2535",borderRadius:8,padding:"10px 16px",flex:2,minWidth:160}}>
  <div
style={{display:"flex",justifyContent:"space-between",marginBottom:4}}>
    <p
style={{fontSize:12,color:"#64748b",margin:0}}>Completion</p>
    <p
style={{fontSize:12,color:"#f1f5f9",fontWeight:700,margin:0}}>{pct}%</p>
  </div>
  <div
style={{background:"#1e2535",borderRadius:4,height:6}}>
    <div
style={{width:`${pct}%`,background:"linear-gradient(90deg,#3b82f6,#10b981)",borderRadius:4,height:6,transition:"width 0.4s"}}/>
    </div>
    <p
style={{fontSize:11,color:"#64748b",margin:"4px 0 0"}}>Avg progress:
{avgProg}%</p>

```

```

        </div>
    </div>

    { /* Per-student progress table */
    {asn.students && asn.students.filter(Boolean).length
> 0 && (
        <div style={{background:"#0f1117",border:"1px
solid #1e2535",borderRadius:8,overflow:"hidden"}}>
            <table
style={{width:"100%",borderCollapse:"collapse",fontSize:13}}>
                <thead>
                    <tr style={{borderBottom:"1px solid
#1e2535"}}>
                        <th style={{padding:"10px
14px",color:"#64748b",fontWeight:600,textAlign:"left"}}>Student</th>
                        <th style={{padding:"10px
14px",color:"#64748b",fontWeight:600,textAlign:"left"}}>Progress</th>
                        <th style={{padding:"10px
14px",color:"#64748b",fontWeight:600,textAlign:"left"}}>Status</th>
                        <th style={{padding:"10px
14px",color:"#64748b",fontWeight:600,textAlign:"left"}}>Completed</th>
                    </tr>
                </thead>
                <tbody>
                    {asn.students.filter(Boolean).map((st,i) =>
{
                        const sc = statusColor(st.status);
                        return (
                            <tr key={i} style={{borderBottom:"1px
solid #1e2535"}}>
                                <td style={{padding:"10px 14px"}}>
                                    <div
style={{display:"flex",alignItems:"center",gap:8}}>
                                        <div
style={{width:28,height:28,borderRadius:"50%",background:"#3b82f622",color
:"#3b82f6",display:"flex",alignItems:"center",justifyContent:"center",font
Size:12,fontWeight:700}}>
{ (st.name || st.email || "?") [0].toUpperCase() }
                                        </div>

```

```

</div>
<p
style={{color:"#f1f5f9",fontWeight:600,margin:0}}>{st.name||"-"}</p>
<p
style={{color:"#64748b",fontSize:11,margin:0}}>{st.email}</p>
</div>
</div>
</td>
<td style={{padding:"10px
14px",minWidth:140}}>
<div
style={{display:"flex",alignItems:"center",gap:8}}>
<div
style={{flex:1,background:"#1e2535",borderRadius:4,height:6}}>
<div
style={{width:`${st.progress||0}%`,background:"linear-gradient(90deg,#3b82
f6,#10b981)",borderRadius:4,height:6}}/>
</div>
<span
style={{color:"#f1f5f9",fontSize:12,fontWeight:600,minWidth:32}}>{st.progr
ess||0}%</span>
</div>
</td>
<td style={{padding:"10px 14px"}}>
<span
style={{background:sc.bg,color:sc.color,fontSize:11,padding:"3px
8px",borderRadius:10,fontWeight:600}}>
{st.status === "completed" ? "✅
Completed" : st.status === "in_progress" ? "⌚ In Progress" : "🔴
Assigned"}
</span>
</td>
<td style={{padding:"10px
14px",color:"#64748b",fontSize:12}}>
{st.completed_at ? new
Date(st.completed_at).toLocaleDateString() : "-"}
</td>
</tr>
);
}}

```

```

        </tbody>
      </table>
    </div>
  )}
</div>
);
}}
</div>
)}
</div>
)}
</main>
</div>
);
}

```

/* src/components/pages/CreateModule.css */

```

/* — Overlay (covers entire screen) ————— */
.cm-overlay {
  position: fixed;
  inset: 0;
  z-index: 9999;
  background: rgba(0,0,0,0.80);
  display: flex;
  align-items: flex-start;
  justify-content: center;
  padding: 24px 16px 40px;
  overflow-y: auto;
}

.cm-modal {
  width: 100%;
  max-width: 900px;
  background: #0f1117;
  border: 1px solid #1e2535;
  border-radius: 18px;
  display: flex;
  flex-direction: column;
  overflow: hidden;
  font-family: 'Inter', sans-serif;
}

```

```
    color: #e2e8f0;
}

/* — Header — */
.cm-header {
  display: flex;
  align-items: center;
  gap: 14px;
  padding: 18px 28px;
  background: #161b27;
  border-bottom: 1px solid #1e2535;
  position: sticky;
  top: 0;
  z-index: 10;
}

.cm-back {
  background: transparent;
  border: 1px solid #2d3748;
  color: #94a3b8;
  padding: 7px 14px;
  border-radius: 8px;
  cursor: pointer;
  font-size: 13px;
  white-space: nowrap;
  transition: all .2s;
}

.cm-back:hover:not(:disabled) { background:#1e2535; color:#e2e8f0; }
.cm-back:disabled { opacity:.4; cursor:not-allowed; }
.cm-title { flex:1; font-size:18px; font-weight:700; margin:0; }
.cm-actions { display:flex; gap:10px; }

.cm-btn-draft, .cm-btn-publish {
  padding: 9px 18px;
  border-radius: 9px;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
  border: none;
  display: flex;
  align-items: center;
```



```

    gap: 6px;
    transition: all .2s;
    white-space: nowrap;
}
.cm-btn-draft {
    background: #1e2535;
    color: #94a3b8;
    border: 1px solid #2d3748;
}
.cm-btn-draft:hover:not(:disabled) { background:#2d3748; color:#e2e8f0; }
.cm-btn-publish {
    background: linear-gradient(135deg,#3b82f6,#6366f1);
    color: #fff;
    min-width: 100px;
    justify-content: center;
}
.cm-btn-publish:hover:not(:disabled) { opacity:.88;
transform:translateY(-1px); }
.cm-btn-publish:disabled, .cm-btn-draft:disabled { opacity:.45;
cursor:not-allowed; transform:none; }

/* — Messages — */
.cm-msg {
    margin: 14px 28px 0;
    padding: 11px 16px;
    border-radius: 10px;
    font-size: 14px;
    font-weight: 500;
}
.cm-msg-error { background:rgba(239,68,68,.12); border:1px solid
rgba(239,68,68,.3); color:#f87171; }
.cm-msg-success { background:rgba(16,185,129,.12); border:1px solid
rgba(16,185,129,.3); color:#34d399; }

/* — Tabs — */
.cm-tabs {
    display: flex;
    gap: 4px;
    padding: 18px 28px 0;
    border-bottom: 1px solid #1e2535;

```

```
    background: #0f1117;
}

.cm-tab {
  padding: 9px 18px;
  border-radius: 8px 8px 0 0;
  border: 1px solid transparent;
  border-bottom: none;
  background: transparent;
  color: #64748b;
  cursor: pointer;
  font-size: 14px;
  font-weight: 500;
  transition: all .15s;
}

.cm-tab:hover { color:#94a3b8; background:#161b27; }
.cm-tab.active { background:#161b27; border-color:#1e2535; color:#e2e8f0;
font-weight:600; }

/* — Scrollable body — */
.cm-body {
  flex: 1;
  overflow-y: auto;
  padding-bottom: 20px;
}

/* — Panel — */
.cm-panel { padding: 24px 28px; }

/* — Info layout — */
.cm-row { display:flex; gap:24px; align-items:flex-start; }
.cm-thumb-col { width:190px; flex-shrink:0; }
.cm-fields-col { flex:1; display:flex; flex-direction:column; gap:12px; }

.cm-thumb-upload {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 190px;
  height: 152px;
  border: 2px dashed #2d3748;
```

```
border-radius: 14px;
cursor: pointer;
overflow: hidden;
background: #161b27;
transition: border-color .2s;
}
.cm-thumb-upload:hover { border-color:#3b82f6; }
.cm-thumb-img { width:100%; height:100%; object-fit:cover; }
.cm-thumb-ph { text-align:center; font-size:34px; color:#475569; }
.cm-thumb-ph small { font-size:11px; display:block; margin-top:4px; }

.cm-label {
  display: block;
  font-size: 11px;
  font-weight: 600;
  color: #64748b;
  text-transform: uppercase;
  letter-spacing: .05em;
  margin-bottom: 5px;
}

.cm-input {
  width: 100%;
  padding: 9px 13px;
  background: #161b27;
  border: 1px solid #1e2535;
  border-radius: 9px;
  color: #e2e8f0;
  font-size: 14px;
  box-sizing: border-box;
  transition: border-color .2s;
  font-family: inherit;
}

.cm-input:focus { outline:none; border-color:#3b82f6; }
.cm-input.sm { padding:7px 10px; font-size:13px; }
.cm-textarea {
  width: 100%;
  padding: 9px 13px;
  background: #161b27;
  border: 1px solid #1e2535;
  border-radius: 9px;
```

```

    color: #e2e8f0;
    font-size: 14px;
    resize: vertical;
    box-sizing: border-box;
    font-family: inherit;
    transition: border-color .2s;
}
.cm-textarea:focus { outline:none; border-color:#3b82f6; }
.cm-select {
    width: 100%;
    padding: 9px 13px;
    background: #161b27;
    border: 1px solid #1e2535;
    border-radius: 9px;
    color: #e2e8f0;
    font-size: 14px;
    cursor: pointer;
    font-family: inherit;
}
.cm-select:focus { outline:none; border-color:#3b82f6; }

.cm-row-inline { display:flex; gap:20px; flex-wrap:wrap;
align-items:flex-start; }
.cm-icon-grid { display:flex; gap:6px; flex-wrap:wrap; margin-top:6px; }
.cm-color-grid { display:flex; gap:6px; flex-wrap:wrap; margin-top:6px; }

.cm-icon-btn {
    width:36px; height:36px;
    background:#1e2535;
    border:2px solid transparent;
    border-radius:8px;
    cursor:pointer;
    font-size:16px;
    transition:all .15s;
}
.cm-icon-btn.selected { border-color:#3b82f6;
background:rgba(59,130,246,.15); }
.cm-icon-btn:hover { border-color:#3b82f6; }

.cm-color-btn {

```

```

width:28px; height:28px;
border:3px solid transparent;
border-radius:50%;
cursor:pointer;
transition:transform .15s;
}

.cm-color-btn.selected { border-color:#fff; transform:scale(1.2); }
.cm-color-btn:hover    { transform:scale(1.1); }

/* — Slide cards — */
.cm-sec-header { display:flex; justify-content:space-between;
align-items:center; margin-bottom:18px; }
.cm-hint { color:#64748b; font-size:13px; margin:0; }
.cm-add-btn {
padding:8px 16px;
background:rgba(59,130,246,.12);
border:1px solid rgba(59,130,246,.3);
color:#3b82f6;
border-radius:9px;
cursor:pointer;
font-size:13px;
font-weight:600;
transition:all .2s;
}
.cm-add-btn:hover { background:rgba(59,130,246,.22); }

.cm-sec-card {
background:#161b27;
border:1px solid #1e2535;
border-left:4px solid #3b82f6;
border-radius:12px;
padding:16px 18px;
margin-bottom:14px;
display:flex;
flex-direction:column;
gap:10px;
}
.cm-sec-top { display:flex; justify-content:space-between;
align-items:center; }

```

```
.cm-sec-num { font-size:11px; font-weight:700; color:#64748b;
text-transform:uppercase; letter-spacing:.04em; }
.cm-rm-btn {
  background:rgba(239,68,68,.12);
  border:1px solid rgba(239,68,68,.2);
  color:#f87171;
  width:28px; height:28px;
  border-radius:6px;
  cursor:pointer;
  font-size:12px;
  transition:all .2s;
}
.cm-rm-btn:hover { background:rgba(239,68,68,.25); }

/* — Quiz cards — */
.cm-quiz-card {
  background:#161b27;
  border:1px solid #1e2535;
  border-radius:12px;
  padding:16px 18px;
  margin-bottom:14px;
  display:flex;
  flex-direction:column;
  gap:10px;
}
.cm-opts-grid { display:grid; grid-template-columns:1fr 1fr; gap:8px; }
.cm-opt {
  display:flex; align-items:center; gap:7px;
  padding:7px 10px;
  border-radius:8px;
  border:1px solid #1e2535;
  background:#0f1117;
  transition:border-color .2s;
}
.cm-opt.correct { border-color:rgba(16,185,129,.45);
background:rgba(16,185,129,.07); }
.cm-opt-lbl { font-size:12px; font-weight:700; color:#64748b; width:16px;
flex-shrink:0; }
.cm-opt .cm-input { border:none; background:transparent; padding:4px 0; }
.cm-chk-btn {
```

```

width:26px; height:26px; flex-shrink:0;
background:#1e2535;
border:1px solid #2d3748;
border-radius:6px;
color:#64748b;
cursor:pointer;
font-size:12px;
transition:all .2s;
}
.cm-chk-btn.active { background:#10b981; border-color:#10b981;
color:#fff; }
.cm-chk-btn:hover { border-color:#10b981; color:#10b981; }

/* — Spinner — */
.cm-spinner {
  display:inline-block;
  width:14px; height:14px;
  border:2px solid rgba(255,255,255,.3);
  border-top-color:#fff;
  border-radius:50%;
  animation:cm-spin .55s linear infinite;
}
@keyframes cm-spin { to { transform:rotate(360deg); } }

/* — Responsive — */
@media(max-width:640px) {
  .cm-overlay { padding:0; align-items:stretch; }
  .cm-modal { border-radius:0; min-height:100dvh; }
  .cm-header { flex-wrap:wrap; padding:14px; gap:10px; }
  .cm-panel { padding:16px; }
  .cm-row { flex-direction:column; }
  .cm-thumb-col{ width:100%; }
  .cm-thumb-upload { width:100%; height:130px; }
  .cm-opts-grid{ grid-template-columns:1fr; }
}

// src/components/pages/CreateModule.jsx
import React, { useState } from "react";
import { auth } from "../../firebase";
import axios from "axios";
import "../CreateModule.css";

```

```

const BASE = "http://localhost:5000";

const ICON_OPTIONS =
["📖", "🌊", "🌋", "💣", "🌵", "🗿", "⚡", "🔥", "🌍", "🛡️", "🔪", "💧"];
const COLOR_OPTIONS = [
  "#f59e0b", "#06b6d4", "#8b5cf6", "#10b981", "#ef4444",
  "#3b82f6", "#ec4899", "#f97316", "#84cc16", "#6366f1",
];

const emptySection = () => ({ tag:"", title:"", content:"", icon:"📖",
color:"#3b82f6", image_url:"" });
const emptyQuiz = () => ({ question:"", option_a:"", option_b:"",
option_c:"", option_d:"", correct_answer:"A", explanation:"" });

export default function CreateModule({ editData = null, onDone }) {
  const isEdit = !!editData;

  const [form, setForm] = useState({
    title:      editData?.title      ?? "",
    description: editData?.description ?? "",
    category:    editData?.category    ?? "Natural Disasters",
    icon:        editData?.icon        ?? "📖",
    color:       editData?.color       ?? "#3b82f6",
  });

  const [thumbnail,      setThumbnail]      = useState(null);
  const [thumbPreview, setThumbPreview] = useState(editData?.thumbnail_url
?? "");

  const [sections,      setSections]      =
useState(editData?.sections?.length ? editData.sections :
[emptySection()]);

  const [quizzes,      setQuizzes]      =
useState(editData?.quizzes?.length ? editData.quizzes : [emptyQuiz()]);
  const [loading,      setLoading]      = useState(false);
  const [error,        setError]        = useState("");
  const [success,      setSuccess]      = useState("");
  const [activeTab,    setActiveTab]    = useState("info");

  const handleThumb = (e) => {
    const f = e.target.files[0];

```



```

    if (!f) return;
    setThumbnail(f);
    setThumbPreview(URL.createObjectURL(f));
  };

  const updSec = (i,k,v) => setSections(p => p.map((s,idx) => idx===i ?
{...s,[k]:v} : s));
  const updQz  = (i,k,v) => setQuizzes(p  => p.map((q,idx) => idx===i ?
{...q,[k]:v} : q));

  const handleSubmit = async (publish) => {
    setError(""); setSuccess("");
    if (!form.title.trim()) { setError("Title is required.");
setActiveTab("info"); return; }

    const user = auth.currentUser;
    if (!user) { setError("Not authenticated. Please log in again.");
return; }

    setLoading(true);
    const fd = new FormData();
    fd.append("title",      form.title.trim());
    fd.append("description", form.description);
    fd.append("category",   form.category);
    fd.append("icon",       form.icon);
    fd.append("color",      form.color);
    fd.append("is_published", String(publish)); // "true" or "false"
    fd.append("created_by",  user.uid);
    fd.append("creator_name", user.displayName || user.email || "");
    fd.append("sections",   JSON.stringify(sections));
    fd.append("quizzes",    JSON.stringify(quizzes));
    if (thumbnail) fd.append("thumbnail", thumbnail);

    try {
      if (isEdit) {
        await axios.put(`${BASE}/api/modules/${editData.id}`, fd);
      } else {
        await axios.post(`${BASE}/api/modules`, fd);
      }
    }
  }

```

```

        setSuccess(publish ? "✅ Module published! Students can now see it."
: "💾 Draft saved!");
        setTimeout(() => onDone && onDone(), 1000);
    } catch (err) {
        console.error(err);
        setError(err.response?.data?.error || err.message || "Something went
wrong.");
    } finally {
        setLoading(false);
    }
};

return (
    <div className="cm-overlay" onClick={ (e) => e.target ===
e.currentTarget && !loading && onDone?.() }>
        <div className="cm-modal">

            { /* — Header — */ }
            <div className="cm-header">
                <button className="cm-back" onClick={ () => !loading &&
onDone?.() } disabled={loading}>← Back</button>
                <h1 className="cm-title">{isEdit ? "✎ Edit Module" : "✨ Create
Module"}</h1>
                <div className="cm-actions">
                    <button className="cm-btn-draft" disabled={loading}
onClick={ () => handleSubmit(false) }>
                        💾 Save Draft
                    </button>
                    <button className="cm-btn-publish" disabled={loading}
onClick={ () => handleSubmit(true) }>
                        {loading ? <span className="cm-spinner"/> : "🚀 Publish"}
                    </button>
                </div>
            </div>

            { /* — Messages — */ }
            {error && <div className="cm-msg cm-msg-error">⚠️ {error}</div>}
            {success && <div className="cm-msg
cm-msg-success">{success}</div>}

```

```

    { /* — Tabs — */
    <div className="cm-tabs">
        {[["info", "📄 Info"], ["content", "📄 Slides"], ["quiz", " ?
Quiz"]].map(([t, label]) => (
            <button key={t} className={`cm-tab
${activeTab===t?"active":""}`} onClick={() => setActiveTab(t)}>
                {label}
            </button>
        ))}
    </div>

    { /* — Scrollable body — */
    <div className="cm-body">

        { /* INFO */
        {activeTab === "info" && (
            <div className="cm-panel">
                <div className="cm-row">
                    <div className="cm-thumb-col">
                        <span className="cm-label">Thumbnail</span>
                        <label className="cm-thumb-upload"
style={{borderColor:form.color}}>
                            {thumbPreview
                            ? <img src={thumbPreview} alt="thumb"
className="cm-thumb-img"/>
                            : <div
className="cm-thumb-ph">{form.icon}<br/><small>Click to
upload</small></div>
                            }
                            <input type="file" accept="image/*"
onChange={handleThumb} hidden/>
                        </label>
                    </div>
                    <div className="cm-fields-col">
                        <span className="cm-label">Title *</span>
                        <input className="cm-input" value={form.title}
placeholder="e.g. Flood Preparedness"
                        onChange={e =>
setForm(p=>({...p,title:e.target.value})) }/>

```

```

        <span className="cm-label">Description</span>
        <textarea className="cm-textarea" rows={3}
value={form.description}
        placeholder="What will students learn?"
        onChange={e =>
setForm(p=>({...p,description:e.target.value})) }/>

        <span className="cm-label">Category</span>
        <select className="cm-select" value={form.category}
        onChange={e =>
setForm(p=>({...p,category:e.target.value})) }>
            {["Natural Disasters","Climate Change","Emergency
Preparedness","Environmental Science","General"]
                .map(c => <option key={c}>{c}</option>)}
        </select>

        <div className="cm-row-inline">
            <div>
                <span className="cm-label">Icon</span>
                <div className="cm-icon-grid">
                    {ICON_OPTIONS.map(ic => (
                        <button key={ic} type="button"
                            className={`cm-icon-btn
${form.icon===ic?"selected":""}`}
                            onClick={() =>
setForm(p=>({...p,icon:ic})) }>{ic}</button>
                        )))}
                </div>
            </div>
            <div>
                <span className="cm-label">Accent Color</span>
                <div className="cm-color-grid">
                    {COLOR_OPTIONS.map(c => (
                        <button key={c} type="button"
                            className={`cm-color-btn
${form.color===c?"selected":""}`}
                            style={{background:c}}
                            onClick={() => setForm(p=>({...p,color:c})) }/>
                        )))}
                </div>
            </div>
        </div>

```

```

        </div>
      </div>
    </div>
  </div>
</div>
)}

{/* CONTENT SLIDES */}
{activeTab === "content" && (
  <div className="cm-panel">
    <div className="cm-sec-header">
      <p className="cm-hint">Each slide is one page students
will read through.</p>
      <button className="cm-add-btn" onClick={() =>
setSections(p=>[...p,emptySection()])}>+ Add Slide</button>
    </div>
    {sections.map((s,i) => (
      <div key={i} className="cm-sec-card"
style={{borderLeftColor:s.color||form.color}}>
        <div className="cm-sec-top">
          <span className="cm-sec-num">Slide {i+1}</span>
          <button className="cm-rm-btn" onClick={() =>
setSections(p=>p.filter((_,x)=>x!==i))}>X</button>
        </div>
        <input className="cm-input" placeholder="Tag (e.g.
Introduction)"
          value={s.tag}
onChange={e=>updSec(i,"tag",e.target.value)}/>
        <input className="cm-input" placeholder="Slide Title *"
          value={s.title}
onChange={e=>updSec(i,"title",e.target.value)}/>
        <textarea className="cm-textarea" rows={4}
placeholder="Content text..."
          value={s.content}
onChange={e=>updSec(i,"content",e.target.value)}/>
        <input className="cm-input" placeholder="Image URL
(optional)"
          value={s.image_url}
onChange={e=>updSec(i,"image_url",e.target.value)}/>
      </div>

```

```

    ))}
  </div>
)}

{/* QUIZ */}
{activeTab === "quiz" && (
  <div className="cm-panel">
    <div className="cm-sec-header">
      <p className="cm-hint">Quiz appears after students finish
all slides.</p>
      <button className="cm-add-btn" onClick={() =>
setQuizzes(p=>[...p,emptyQuiz()])}>+ Add Question</button>
    </div>
    {quizzes.map((q,i) => (
      <div key={i} className="cm-quiz-card">
        <div className="cm-sec-top">
          <span className="cm-sec-num">Q{i+1}</span>
          <button className="cm-rm-btn" onClick={() =>
setQuizzes(p=>p.filter((_,x)=>x!==i))>X</button>
        </div>
        <input className="cm-input" placeholder="Question *"
          value={q.question}
onChange={e=>updQz(i,"question",e.target.value)}/>
        <div className="cm-opts-grid">
          [{"a","b","c","d"].map(opt => (
            <div key={opt} className={`cm-opt
${q.correct_answer===opt.toUpperCase()?"correct":""}`>
              <span
className="cm-opt-lbl">{opt.toUpperCase()}</span>
              <input className="cm-input sm"
placeholder={`Option ${opt.toUpperCase()}`}
value={q[`option_${opt}`]}
onChange={e=>updQz(i,`option_${opt}`,e.target.value)}/>
              <button type="button"
className={`cm-chk-btn
${q.correct_answer===opt.toUpperCase()?"active":""}`}
onClick={() =>
updQz(i,"correct_answer",opt.toUpperCase())}
title="Mark correct">✓</button>

```

```

        </div>
    ))}
</div>
<input className="cm-input" placeholder="Explanation
(optional)"
        value={q.explanation}
onChange={e=>updQz(i,"explanation",e.target.value)}/>
    </div>
    ))}
</div>
    )}
</div>
</div>
</div>
);
}
/* =====
Dashboard.css - DisasterReady Dashboard Shared Styles
===== */

@import
url('https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600;70
0&display=swap');

/* ----- Root Layout ----- */
.dash-root {
    display: flex;
    min-height: 100vh;
    background: #0a0f1e;
    font-family: 'Sora', sans-serif;
    color: #e2e8f0;
}

/* ----- Sidebar ----- */
.dash-sidebar {
    width: 240px;
    min-height: 100vh;
    background: rgba(255,255,255,0.03);
    border-right: 1px solid rgba(255,255,255,0.07);
    display: flex;

```

```

    flex-direction: column;
    padding: 24px 16px;
    position: sticky;
    top: 0;
    height: 100vh;
    overflow-y: auto;
    flex-shrink: 0;
}

/* ----- Logo ----- */
.dash-logo {
    display: flex;
    align-items: center;
    gap: 10px;
    padding: 4px 8px 28px;
    font-size: 15px;
    font-weight: 700;
    color: #f1f5f9;
    letter-spacing: -0.3px;
}

/* ----- Admin Badge ----- */
.admin-badge-pill {
    display: inline-block;
    background: linear-gradient(135deg, rgba(244,114,182,0.2),
    rgba(139,92,246,0.2));
    border: 1px solid rgba(139,92,246,0.3);
    border-radius: 20px;
    padding: 4px 12px;
    font-size: 11px;
    font-weight: 600;
    color: #c4b5fd;
    text-transform: uppercase;
    letter-spacing: 0.8px;
    margin-bottom: 20px;
    margin-left: 8px;
}

/* ----- Nav ----- */
.dash-nav {

```



```
display: flex;
flex-direction: column;
gap: 4px;
flex: 1;
}

.dash-nav-item {
display: flex;
align-items: center;
gap: 10px;
padding: 10px 12px;
border-radius: 10px;
border: none;
background: transparent;
color: rgba(255,255,255,0.42);
font-family: 'Sora', sans-serif;
font-size: 14px;
font-weight: 500;
cursor: pointer;
transition: all 0.2s ease;
text-align: left;
}

.dash-nav-item:hover {
background: rgba(255,255,255,0.06);
color: rgba(255,255,255,0.8);
}

.dash-nav-item.active {
background: rgba(59,130,246,0.15);
color: #93c5fd;
border: 1px solid rgba(59,130,246,0.2);
}

.dash-nav-item.admin-active {
background: rgba(139,92,246,0.15);
color: #c4b5fd;
border: 1px solid rgba(139,92,246,0.2);
}
```

```
/* ----- Sidebar Footer ----- */
.dash-sidebar-footer {
  display: flex;
  align-items: center;
  gap: 8px;
  padding-top: 20px;
  border-top: 1px solid rgba(255,255,255,0.07);
}

.dash-user-pill {
  display: flex;
  align-items: center;
  gap: 10px;
  flex: 1;
  min-width: 0;
}

.dash-avatar {
  width: 34px;
  height: 34px;
  border-radius: 10px;
  background: linear-gradient(135deg, #6ee7b7, #3b82f6);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 14px;
  font-weight: 700;
  color: #0a0f1e;
  flex-shrink: 0;
}

.admin-avatar {
  background: linear-gradient(135deg, #f472b6, #8b5cf6);
}

.dash-user-info {
  display: flex;
  flex-direction: column;
  min-width: 0;
}
```

```
.dash-user-name {
  font-size: 13px;
  font-weight: 600;
  color: #e2e8f0;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

.dash-user-role {
  font-size: 11px;
  color: rgba(255,255,255,0.35);
}

.dash-logout {
  padding: 8px;
  border-radius: 8px;
  border: none;
  background: transparent;
  color: rgba(255,255,255,0.3);
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  transition: all 0.2s ease;
  flex-shrink: 0;
}

.dash-logout:hover {
  background: rgba(239,68,68,0.15);
  color: #f87171;
}

/* ----- Main Content ----- */
.dash-main {
  flex: 1;
  min-width: 0;
  display: flex;
  flex-direction: column;
```

```
    overflow: hidden;
}

/* ----- Top Bar ----- */
.dash-topbar {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 24px 32px 20px;
    border-bottom: 1px solid rgba(255,255,255,0.07);
    gap: 20px;
    flex-wrap: wrap;
}

.dash-welcome {
    font-size: 22px;
    font-weight: 700;
    color: #f1f5f9;
    margin: 0 0 4px;
    letter-spacing: -0.3px;
}

.dash-date {
    font-size: 13px;
    color: rgba(255,255,255,0.35);
    margin: 0;
}

.dash-topbar-actions {
    display: flex;
    align-items: center;
    gap: 12px;
}

.dash-search {
    display: flex;
    align-items: center;
    gap: 8px;
    background: rgba(255,255,255,0.05);
    border: 1px solid rgba(255,255,255,0.09);
}
```

```
border-radius: 10px;
padding: 8px 14px;
}

.dash-search svg {
  color: rgba(255,255,255,0.3);
  flex-shrink: 0;
}

.dash-search input {
  background: none;
  border: none;
  outline: none;
  color: #e2e8f0;
  font-family: 'Sora', sans-serif;
  font-size: 14px;
  width: 180px;
}

.dash-search input::placeholder {
  color: rgba(255,255,255,0.25);
}

/* ----- Admin Action Button ----- */
.btn-admin-action {
  display: flex;
  align-items: center;
  gap: 6px;
  padding: 9px 16px;
  background: linear-gradient(135deg, #f472b6, #8b5cf6);
  border: none;
  border-radius: 10px;
  color: white;
  font-family: 'Sora', sans-serif;
  font-size: 13.5px;
  font-weight: 600;
  cursor: pointer;
  transition: opacity 0.2s ease, transform 0.15s ease;
  white-space: nowrap;
}
```

```
.btn-admin-action:hover {
  opacity: 0.88;
  transform: translateY(-1px);
}

/* ----- Content Area ----- */
.dash-content {
  padding: 28px 32px;
  display: flex;
  flex-direction: column;
  gap: 24px;
  overflow-y: auto;
  flex: 1;
}

/* ----- Stat Cards ----- */
.stat-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px;
}

.stat-card {
  background: rgba(255,255,255,0.04);
  border: 1px solid rgba(255,255,255,0.08);
  border-radius: 16px;
  padding: 20px;
  display: flex;
  align-items: center;
  gap: 16px;
  transition: border-color 0.2s ease, transform 0.2s ease;
}

.stat-card:hover {
  border-color: rgba(255,255,255,0.16);
  transform: translateY(-2px);
}

.stat-icon {
```

```
width: 44px;
height: 44px;
border-radius: 12px;
display: flex;
align-items: center;
justify-content: center;
flex-shrink: 0;
}

.stat-label {
  font-size: 12px;
  color: rgba(255,255,255,0.38);
  margin: 0 0 4px;
  font-weight: 500;
  letter-spacing: 0.3px;
}

.stat-value {
  font-size: 22px;
  font-weight: 700;
  color: #f1f5f9;
  margin: 0;
  letter-spacing: -0.5px;
}

/* ----- Section Card ----- */
.section-card {
  background: rgba(255,255,255,0.04);
  border: 1px solid rgba(255,255,255,0.08);
  border-radius: 16px;
  padding: 24px;
}

.section-title {
  font-size: 16px;
  font-weight: 600;
  color: #e2e8f0;
  margin: 0 0 20px;
  letter-spacing: -0.2px;
}
```

```
/* ----- Module List (overview) ----- */
.module-list {
  display: flex;
  flex-direction: column;
  gap: 16px;
}

.module-row {
  display: flex;
  align-items: center;
  gap: 14px;
}

.module-emoji {
  font-size: 24px;
  width: 36px;
  text-align: center;
  flex-shrink: 0;
}

.module-info {
  flex: 1;
  min-width: 0;
}

.module-name {
  font-size: 14px;
  font-weight: 500;
  color: #cbd5e1;
  margin: 0 0 6px;
}

.module-pct {
  font-size: 13px;
  font-weight: 700;
  min-width: 38px;
  text-align: right;
}
```



```
/* ----- Progress Bar ----- */
.progress-bar-bg {
  height: 5px;
  background: rgba(255,255,255,0.08);
  border-radius: 4px;
  overflow: hidden;
}

.progress-bar-fill {
  height: 100%;
  border-radius: 4px;
  transition: width 0.5s ease;
}

/* ----- Quiz Rows ----- */
.quiz-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 14px 0;
  border-bottom: 1px solid rgba(255,255,255,0.06);
  gap: 12px;
}

.quiz-row:last-child {
  border-bottom: none;
}

.quiz-name {
  font-size: 14px;
  font-weight: 500;
  color: #cbd5e1;
  margin: 0 0 3px;
}

.quiz-date {
  font-size: 12px;
  color: rgba(255,255,255,0.28);
  margin: 0;
}
```

```
.quiz-score {
  font-size: 16px;
  font-weight: 700;
  min-width: 44px;
  text-align: right;
}

.quiz-score-bar {
  display: flex;
  align-items: center;
  gap: 12px;
}

.score-track {
  height: 6px;
  background: rgba(255,255,255,0.08);
  border-radius: 4px;
  overflow: hidden;
  width: 120px;
}

.score-fill {
  height: 100%;
  border-radius: 4px;
  transition: width 0.5s ease;
}

/* ----- Alert Rows ----- */
.alert-row {
  display: flex;
  align-items: flex-start;
  gap: 12px;
  padding: 14px 0;
  border-bottom: 1px solid rgba(255,255,255,0.06);
}

.alert-row:last-child { border-bottom: none; }

.alert-dot {
```

```
width: 8px;
height: 8px;
border-radius: 50%;
margin-top: 4px;
flex-shrink: 0;
}

.alert-info .alert-dot { background: #60a5fa; }
.alert-success .alert-dot { background: #34d399; }
.alert-warning .alert-dot { background: #fbbf24; }

.alert-msg {
  font-size: 14px;
  color: #cbd5e1;
  margin: 0 0 3px;
}

.alert-time {
  font-size: 12px;
  color: rgba(255,255,255,0.28);
  margin: 0;
}

/* ----- Module Cards Grid ----- */
.modules-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(240px, 1fr));
  gap: 20px;
}

.module-card {
  background: rgba(255,255,255,0.04);
  border: 1px solid rgba(255,255,255,0.08);
  border-radius: 16px;
  overflow: hidden;
  transition: border-color 0.2s ease, transform 0.2s ease;
}

.module-card:hover {
  border-color: rgba(255,255,255,0.18);
}
```

```
    transform: translateY(-3px);
}

.module-card-header {
  padding: 22px;
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.module-badge {
  font-size: 11px;
  font-weight: 600;
  padding: 4px 10px;
  border-radius: 20px;
  letter-spacing: 0.3px;
}

.module-card-body {
  padding: 18px 22px;
}

.module-card-title {
  font-size: 15px;
  font-weight: 600;
  color: #e2e8f0;
  margin: 0 0 6px;
}

.module-card-meta {
  font-size: 12px;
  color: rgba(255,255,255,0.35);
  margin: 0 0 12px;
}

.module-meta-row {
  display: flex;
  justify-content: space-between;
  margin-bottom: 10px;
}
```

```
.module-cta {
  margin-top: 14px;
  background: transparent;
  border: 1px solid;
  border-radius: 8px;
  padding: 8px 14px;
  font-family: 'Sora', sans-serif;
  font-size: 13px;
  font-weight: 600;
  cursor: pointer;
  transition: opacity 0.2s ease;
  width: 100%;
}

.module-cta:hover { opacity: 0.75; }

/* ----- Student Table ----- */
.student-table-wrap {
  overflow-x: auto;
}

.student-table {
  width: 100%;
  border-collapse: collapse;
  font-size: 14px;
}

.student-table th {
  text-align: left;
  padding: 0 12px 14px;
  font-size: 11px;
  font-weight: 600;
  color: rgba(255,255,255,0.3);
  text-transform: uppercase;
  letter-spacing: 0.6px;
  border-bottom: 1px solid rgba(255,255,255,0.07);
}

.student-table td {
```

```
padding: 14px 12px;
border-bottom: 1px solid rgba(255,255,255,0.05);
vertical-align: middle;
}

.student-table tr:last-child td {
  border-bottom: none;
}

.student-cell {
  display: flex;
  align-items: center;
  gap: 12px;
}

.student-avatar {
  width: 32px;
  height: 32px;
  border-radius: 9px;
  background: linear-gradient(135deg, #8b5cf6, #3b82f6);
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 13px;
  font-weight: 700;
  color: white;
  flex-shrink: 0;
}

.student-name {
  font-size: 14px;
  font-weight: 500;
  color: #e2e8f0;
  margin: 0 0 2px;
}

.student-email {
  font-size: 12px;
  color: rgba(255,255,255,0.28);
  margin: 0;
```

```
}

.table-progress {
  display: flex;
  align-items: center;
  gap: 10px;
}

.table-pct {
  font-size: 12px;
  font-weight: 600;
  color: rgba(255,255,255,0.5);
  min-width: 30px;
}

.last-active {
  font-size: 13px;
  color: rgba(255,255,255,0.38);
}

.status-pill {
  padding: 4px 10px;
  border-radius: 20px;
  font-size: 11px;
  font-weight: 600;
  letter-spacing: 0.3px;
}

.status-pill.active {
  background: rgba(16,185,129,0.15);
  color: #34d399;
  border: 1px solid rgba(16,185,129,0.25);
}

.status-pill.inactive {
  background: rgba(239,68,68,0.1);
  color: #f87171;
  border: 1px solid rgba(239,68,68,0.2);
}
```

```
/* ----- Admin Module Card ----- */
.admin-module-card .module-card-header {
  background: rgba(139,92,246,0.08) !important;
}

.btn-edit {
  background: rgba(139,92,246,0.2);
  border: 1px solid rgba(139,92,246,0.3);
  border-radius: 8px;
  color: #c4b5fd;
  font-family: 'Sora', sans-serif;
  font-size: 12px;
  font-weight: 600;
  padding: 5px 10px;
  cursor: pointer;
  transition: all 0.2s ease;
}

.btn-edit:hover {
  background: rgba(139,92,246,0.3);
}

.btn-view {
  background: transparent;
  border: 1px solid rgba(255,255,255,0.12);
  border-radius: 8px;
  color: rgba(255,255,255,0.5);
  font-family: 'Sora', sans-serif;
  font-size: 12px;
  padding: 5px 10px;
  cursor: pointer;
  transition: all 0.2s ease;
}

.btn-view:hover {
  border-color: rgba(255,255,255,0.25);
  color: #e2e8f0;
}

/* ----- Scrollbar ----- */
```



```

::-webkit-scrollbar {
    width: 5px;
    height: 5px;
}

::-webkit-scrollbar-track {
    background: transparent;
}

::-webkit-scrollbar-thumb {
    background: rgba(255,255,255,0.1);
    border-radius: 4px;
}

::-webkit-scrollbar-thumb:hover {
    background: rgba(255,255,255,0.2);
}

/* ----- Responsive ----- */
@media (max-width: 768px) {
    .dash-sidebar {
        display: none;
    }
    .dash-topbar {
        padding: 16px 20px;
    }
    .dash-content {
        padding: 20px;
    }
    .stat-grid {
        grid-template-columns: 1fr 1fr;
    }
}

/* =====
ADD THESE STYLES TO THE BOTTOM OF YOUR EXISTING Dashboard.css
Quiz Banner – Student & Admin variants
===== */

```

```
/* =====  
STUDENT QUIZ BANNER  
===== */  
  
.quiz-banner-student {  
  position: relative;  
  border-radius: 18px;  
  padding: 26px 28px;  
  background: linear-gradient(135deg, rgba(110,231,183,0.08) 0%,  
rgba(59,130,246,0.12) 100%);  
  border: 1px solid rgba(110,231,183,0.2);  
  cursor: pointer;  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  gap: 20px;  
  overflow: hidden;  
  transition: border-color 0.3s ease, transform 0.2s ease, box-shadow 0.3s  
ease;  
}  
  
.quiz-banner-student.hovered {  
  border-color: rgba(110,231,183,0.45);  
  transform: translateY(-2px);  
  box-shadow: 0 16px 48px rgba(59,130,246,0.2);  
}  
  
/* Animated orbs */  
.qb-orb {  
  position: absolute;  
  border-radius: 50%;  
  filter: blur(40px);  
  pointer-events: none;  
  animation: orbFloat 8s ease-in-out infinite;  
}  
  
.qb-orb-1 {  
  width: 120px; height: 120px;  
  background: rgba(110,231,183,0.18);  
  top: -30px; left: -20px;  
  animation-delay: 0s;
```

```
}

.qb-orb-2 {
  width: 80px; height: 80px;
  background: rgba(59,130,246,0.18);
  bottom: -20px; right: 200px;
  animation-delay: 2.5s;
}

.qb-orb-3 {
  width: 60px; height: 60px;
  background: rgba(139,92,246,0.15);
  top: 10px; right: 120px;
  animation-delay: 5s;
}

@keyframes orbFloat {
  0%, 100% { transform: translateY(0px) scale(1); }
  50%      { transform: translateY(-12px) scale(1.1); }
}

.qb-left {
  display: flex;
  align-items: center;
  gap: 20px;
  position: relative;
  z-index: 2;
  flex: 1;
  min-width: 0;
}

.qb-icon-wrap {
  position: relative;
  width: 56px;
  height: 56px;
  flex-shrink: 0;
  display: flex;
  align-items: center;
  justify-content: center;
```

```
    background: linear-gradient(135deg, rgba(110,231,183,0.25),
    rgba(59,130,246,0.25));
    border: 1px solid rgba(110,231,183,0.3);
    border-radius: 16px;
    color: #6ee7b7;
}

.qb-icon-ping {
    position: absolute;
    inset: -3px;
    border-radius: 18px;
    border: 1.5px solid rgba(110,231,183,0.4);
    animation: ping 2s cubic-bezier(0,0,0.2,1) infinite;
}

@keyframes ping {
    0%      { transform: scale(1); opacity: 0.6; }
    80%, 100% { transform: scale(1.25); opacity: 0; }
}

.qb-text { min-width: 0; }

.qb-badge {
    display: inline-block;
    font-size: 10px;
    font-weight: 700;
    letter-spacing: 0.8px;
    text-transform: uppercase;
    color: #6ee7b7;
    background: rgba(110,231,183,0.12);
    border: 1px solid rgba(110,231,183,0.2);
    border-radius: 20px;
    padding: 3px 9px;
    margin-bottom: 8px;
}

.qb-title {
    font-size: 18px;
    font-weight: 700;
    color: #f1f5f9;
```

```
margin: 0 0 5px;
letter-spacing: -0.2px;
}

.qb-desc {
  font-size: 13px;
  color: rgba(255,255,255,0.42);
  margin: 0;
  line-height: 1.5;
}

.qb-right {
  display: flex;
  flex-direction: column;
  align-items: flex-end;
  gap: 14px;
  position: relative;
  z-index: 2;
  flex-shrink: 0;
}

.qb-stats {
  display: flex;
  align-items: center;
  gap: 16px;
}

.qb-stat {
  display: flex;
  flex-direction: column;
  align-items: center;
}

.qb-stat-num {
  font-size: 20px;
  font-weight: 700;
  color: #f1f5f9;
  line-height: 1;
}
```

```

.qb-stat-label {
  font-size: 11px;
  color: rgba(255,255,255,0.35);
  margin-top: 2px;
}

.qb-stat-divider {
  width: 1px;
  height: 28px;
  background: rgba(255,255,255,0.1);
}

.qb-cta-btn {
  display: flex;
  align-items: center;
  gap: 7px;
  padding: 10px 20px;
  background: linear-gradient(135deg, #6ee7b7, #3b82f6);
  border-radius: 10px;
  color: #0a0f1e;
  font-size: 14px;
  font-weight: 700;
  letter-spacing: -0.1px;
  transition: opacity 0.2s ease, transform 0.15s ease;
  box-shadow: 0 4px 16px rgba(59,130,246,0.35);
}

.quiz-banner-student.hovered .qb-cta-btn {
  opacity: 0.88;
  transform: translateY(-1px);
}

/* =====
ADMIN QUIZ BANNER
===== */

.quiz-banner-admin {
  position: relative;
  border-radius: 18px;
  padding: 26px 28px;

```

```

    background: linear-gradient(135deg, rgba(99,102,241,0.12) 0%,
    rgba(139,92,246,0.14) 100%);
    border: 1px solid rgba(139,92,246,0.25);
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 20px;
    overflow: hidden;
    transition: border-color 0.3s ease, transform 0.2s ease, box-shadow 0.3s
    ease;
}

.quiz-banner-admin.hovered {
    border-color: rgba(139,92,246,0.5);
    transform: translateY(-2px);
    box-shadow: 0 16px 48px rgba(139,92,246,0.2);
}

/* Decorative grid overlay */
.qba-grid {
    position: absolute;
    inset: 0;
    background-image:
        linear-gradient(rgba(255,255,255,0.03) 1px, transparent 1px),
        linear-gradient(90deg, rgba(255,255,255,0.03) 1px, transparent 1px);
    background-size: 28px 28px;
    pointer-events: none;
}

/* Glow orb */
.qba-glow {
    position: absolute;
    width: 180px;
    height: 180px;
    border-radius: 50%;
    background: rgba(139,92,246,0.2);
    filter: blur(60px);
    top: -40px;
    left: -30px;

```

```
    pointer-events: none;
    animation: adminGlow 6s ease-in-out infinite;
}

@keyframes adminGlow {
    0%, 100% { transform: scale(1); opacity: 0.7; }
    50%      { transform: scale(1.15); opacity: 1; }
}

.qba-left {
    display: flex;
    align-items: center;
    gap: 20px;
    position: relative;
    z-index: 2;
    flex: 1;
}

.qba-icon-cluster {
    position: relative;
    width: 58px;
    height: 58px;
    flex-shrink: 0;
}

.qba-icon-main {
    width: 58px;
    height: 58px;
    border-radius: 16px;
    background: linear-gradient(135deg, #6366f1, #8b5cf6);
    display: flex;
    align-items: center;
    justify-content: center;
    color: white;
    box-shadow: 0 6px 20px rgba(139,92,246,0.4);
}

.qba-icon-ring {
    position: absolute;
    inset: -5px;
```



```
border-radius: 20px;
border: 1.5px solid rgba(139,92,246,0.35);
animation: adminRing 3s ease-in-out infinite;
}

.qba-ring-2 {
  inset: -10px;
  border-color: rgba(139,92,246,0.15);
  animation-delay: 0.75s;
}

@keyframes adminRing {
  0%, 100% { opacity: 0.6; transform: scale(1); }
  50%      { opacity: 0; transform: scale(1.1); }
}

.qba-badge {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  font-size: 10px;
  font-weight: 700;
  letter-spacing: 0.7px;
  text-transform: uppercase;
  color: #c4b5fd;
  background: rgba(139,92,246,0.15);
  border: 1px solid rgba(139,92,246,0.25);
  border-radius: 20px;
  padding: 3px 10px;
  margin-bottom: 8px;
}

.qba-badge-dot {
  width: 6px;
  height: 6px;
  border-radius: 50%;
  background: #a78bfa;
  animation: blink 1.4s ease-in-out infinite;
}
```

```
@keyframes blink {
  0%, 100% { opacity: 1; }
  50%      { opacity: 0.3; }
}

.qba-title {
  font-size: 18px;
  font-weight: 700;
  color: #f1f5f9;
  margin: 0 0 5px;
  letter-spacing: -0.2px;
}

.qba-desc {
  font-size: 13px;
  color: rgba(255,255,255,0.4);
  margin: 0;
  line-height: 1.5;
}

.qba-btn {
  position: relative;
  z-index: 2;
  flex-shrink: 0;
  display: flex;
  align-items: center;
  gap: 8px;
  padding: 12px 22px;
  background: linear-gradient(135deg, #6366f1, #8b5cf6);
  border: none;
  border-radius: 12px;
  color: white;
  font-family: 'Sora', sans-serif;
  font-size: 14px;
  font-weight: 700;
  cursor: pointer;
  transition: opacity 0.2s ease, transform 0.15s ease;
  box-shadow: 0 6px 20px rgba(139,92,246,0.4);
}
```

```

.quiz-banner-admin.hovered .qba-btn {
  opacity: 0.9;
  transform: translateY(-1px);
}

/* =====
   QUIZ TAB CTA (Student Quizzes Tab)
   ===== */

.quiz-tab-cta {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 14px 18px;
  background: rgba(110,231,183,0.07);
  border: 1px dashed rgba(110,231,183,0.3);
  border-radius: 12px;
  color: #6ee7b7;
  font-family: 'Sora', sans-serif;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.2s ease;
}

.quiz-tab-cta:hover {
  background: rgba(110,231,183,0.12);
  border-color: rgba(110,231,183,0.5);
}

/* — Learning Module Cards (Student Overview) — */
.mod-section-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 18px;
}

.mod-count-pill {

```

```
font-size: 12px;
color: rgba(255,255,255,0.3);
background: rgba(255,255,255,0.06);
border: 1px solid rgba(255,255,255,0.08);
border-radius: 20px;
padding: 3px 10px;
}

.mod-cards-list {
display: flex;
flex-direction: column;
gap: 10px;
}

.mod-list-card {
background: rgba(255,255,255,0.03);
border: 1px solid rgba(255,255,255,0.07);
border-radius: 14px;
padding: 16px 18px;
display: flex;
align-items: center;
gap: 14px;
cursor: pointer;
position: relative;
overflow: hidden;
transition: all 0.22s ease;
}

.mod-list-card::before {
content: '';
position: absolute;
left: 0; top: 0; bottom: 0;
width: 3px;
border-radius: 4px 0 0 4px;
background: var(--accent);
opacity: 0.65;
transition: opacity 0.2s ease;
}

.mod-list-card:hover {
```

```
border-color: rgba(255,255,255,0.14);
transform: translateX(3px);
background: rgba(255,255,255,0.05);
}

.mod-list-card:hover::before { opacity: 1; }

.mod-list-icon {
  width: 50px; height: 50px;
  border-radius: 13px;
  background: var(--icon-bg);
  display: flex; align-items: center; justify-content: center;
  font-size: 24px; flex-shrink: 0;
}

.mod-list-body { flex: 1; min-width: 0; }

.mod-list-top {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 4px;
}

.mod-list-title {
  font-size: 14px;
  font-weight: 600;
  color: #e2e8f0;
  margin: 0;
}

.mod-list-tag {
  font-size: 10px;
  font-weight: 700;
  letter-spacing: 0.5px;
  padding: 2px 8px;
  border-radius: 20px;
  border: 1px solid;
  margin-left: auto;
  white-space: nowrap;
}
```

```
}

.mod-list-tag.tag-new      { color:#06b6d4; background:rgba(6,182,212,0.1);
border-color:rgba(6,182,212,0.25); }
.mod-list-tag.tag-essential { color:#f59e0b;
background:rgba(245,158,11,0.1); border-color:rgba(245,158,11,0.25); }
.mod-list-tag.tag-env      { color:#10b981;
background:rgba(16,185,129,0.1); border-color:rgba(16,185,129,0.25); }

.mod-list-meta {
  font-size: 11px;
  color: rgba(255,255,255,0.28);
  margin: 0 0 9px;
}

.mod-list-progress-row {
  display: flex;
  align-items: center;
  gap: 10px;
}

.mod-list-pct {
  font-size: 12px;
  font-weight: 700;
  min-width: 30px;
  text-align: right;
  flex-shrink: 0;
}

.mod-list-badge {
  font-size: 10px;
  font-weight: 600;
  padding: 2px 8px;
  border-radius: 20px;
  white-space: nowrap;
  flex-shrink: 0;
}

.mod-list-badge.completed { background:rgba(16,185,129,0.15);
color:#34d399; border:1px solid rgba(16,185,129,0.2); }
```

```

.mod-list-badge.inprogress { background:rgba(245,158,11,0.12);
color:#fbbf24; border:1px solid rgba(245,158,11,0.2); }
.mod-list-badge.notstarted { background:rgba(255,255,255,0.05);
color:#64748b; border:1px solid rgba(255,255,255,0.08); }

.mod-list-arrow {
  font-size: 20px;
  color: rgba(255,255,255,0.18);
  flex-shrink: 0;
  transition: color 0.2s ease, transform 0.2s ease;
  line-height: 1;
}

.mod-list-card:hover .mod-list-arrow {
  color: var(--accent);
  transform: translateX(2px);
}

/* =====
DesertificationModule.css – Desertification Learning Slides
Matches dark glassmorphism theme from TsunamiModule.css
===== */

@import
url('https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600;700&display=swap');

/* — Overlay — */
.dsm-overlay {
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.75);
  backdrop-filter: blur(8px);
  -webkit-backdrop-filter: blur(8px);
  z-index: 1000;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 20px;
  animation: dsFadeOverlay 0.3s ease;

```

```

}

@keyframes dsFadeOverlay {
  from { opacity: 0; }
  to   { opacity: 1; }
}

/* — Modal — */
.dsm-modal {
  width: 100%;
  max-width: 760px;
  max-height: 92vh;
  background: rgba(10, 15, 30, 0.97);
  border: 1px solid rgba(255, 255, 255, 0.10);
  border-radius: 28px;
  display: flex;
  flex-direction: column;
  overflow: hidden;
  box-shadow: 0 40px 80px rgba(0, 0, 0, 0.7);
  animation: dsModalIn 0.4s cubic-bezier(0.22, 1, 0.36, 1);
  font-family: 'Sora', sans-serif;
}

@keyframes dsModalIn {
  from { opacity: 0; transform: scale(0.92) translateY(20px); }
  to   { opacity: 1; transform: none; }
}

/* — Header — */
.dsm-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 18px 24px 14px;
  border-bottom: 1px solid rgba(255, 255, 255, 0.07);
  flex-shrink: 0;
}

.dsm-header-left { display: flex; align-items: center; gap: 12px; }

```



```
.dsm-module-badge {
  font-size: 13px;
  font-weight: 600;
  color: #fb923c;
  background: rgba(251, 146, 60, 0.12);
  border: 1px solid rgba(251, 146, 60, 0.25);
  border-radius: 20px;
  padding: 4px 12px;
}

.dsm-slide-count { font-size: 13px; color: rgba(255, 255, 255, 0.35); }

.dsm-header-right { display: flex; align-items: center; gap: 14px; }
.dsm-progress-label { font-size: 12px; color: rgba(255, 255, 255, 0.35); }

.dsm-close-btn {
  width: 30px;
  height: 30px;
  border-radius: 50%;
  border: 1px solid rgba(255, 255, 255, 0.15);
  background: transparent;
  color: rgba(255, 255, 255, 0.5);
  cursor: pointer;
  font-size: 14px;
  display: flex;
  align-items: center;
  justify-content: center;
  transition: all 0.2s;
}

.dsm-close-btn:hover { background: rgba(255,255,255,0.1); color: #fff; }

/* — Progress Track — */
.dsm-progress-track { height: 3px; background: rgba(255, 255, 255, 0.07);
flex-shrink: 0; }
.dsm-progress-fill { height: 100%; border-radius: 3px; transition: width
0.5s ease, background 0.5s ease; }

/* — Slide Dots — */
.dsm-dots {
  display: flex;
```

```
align-items: center;
justify-content: center;
gap: 7px;
padding: 12px 24px 8px;
flex-shrink: 0;
flex-wrap: wrap;
}

.dsm-dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  border: 1px solid rgba(255, 255, 255, 0.2);
  background: rgba(255, 255, 255, 0.08);
  cursor: pointer;
  transition: all 0.2s ease;
  padding: 0;
}

.dsm-dot:hover { border-color: rgba(255,255,255,0.5); transform:
scale(1.2); }
.dsm-dot.active { width: 20px; border-radius: 4px; }
.dsm-dot.done { background: rgba(255,255,255,0.2); border-color:
rgba(255,255,255,0.3); }

/* — Slide — */
.dsm-slide {
  flex: 1;
  overflow-y: auto;
  padding: 20px 32px 16px;
  border: 1px solid transparent;
  margin: 0 16px;
  border-radius: 20px;
  transition: background 0.4s ease, border-color 0.4s ease;
}

.dsm-slide::-webkit-scrollbar { width: 4px; }
.dsm-slide::-webkit-scrollbar-track { background: transparent; }
.dsm-slide::-webkit-scrollbar-thumb { background: rgba(255,255,255,0.1);
border-radius: 4px; }
```

```

@keyframes dsSlideInForward {
  from { opacity: 0; transform: translateX(40px); }
  to   { opacity: 1; transform: none; }
}

@keyframes dsSlideInBackward {
  from { opacity: 0; transform: translateX(-40px); }
  to   { opacity: 1; transform: none; }
}

.dsm-anim-forward { animation: dsSlideInForward 0.35s
cubic-bezier(0.22,1,0.36,1); }
.dsm-anim-backward { animation: dsSlideInBackward 0.35s
cubic-bezier(0.22,1,0.36,1); }

/* — Slide Content — */
.dsm-slide-tag {
  font-size: 10.5px;
  font-weight: 700;
  letter-spacing: 1px;
  text-transform: uppercase;
  border: 1px solid;
  border-radius: 20px;
  padding: 3px 10px;
  display: inline-block;
  margin-bottom: 12px;
}

.dsm-slide-icon { font-size: 42px; margin-bottom: 8px; line-height: 1; }
.dsm-slide-title { font-size: 22px; font-weight: 700; color: #f1f5f9;
margin: 0 0 6px; letter-spacing: -0.4px; }
.dsm-slide-subtitle { font-size: 13.5px; color: rgba(255,255,255,0.45);
margin: 0 0 18px; font-weight: 400; }
.dsm-body-text { font-size: 15px; color: #cbd5e1; line-height: 1.7;
margin: 0 0 16px; }

/* — Causes Grid — */
.dsm-causes-grid { display: grid; grid-template-columns: repeat(3, 1fr);
gap: 12px; margin-bottom: 16px; }

```

```

.dsm-cause-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid;
  border-radius: 14px;
  padding: 16px 14px;
  text-align: center;
  animation: dsCardPop 0.4s ease both;
}

@keyframes dsCardPop {
  from { opacity: 0; transform: scale(0.9) translateY(10px); }
  to { opacity: 1; transform: none; }
}

.dsm-cause-icon { font-size: 30px; display: block; margin-bottom: 8px; }
.dsm-cause-label { font-size: 13px; font-weight: 600; display: block;
margin-bottom: 6px; }
.dsm-cause-desc { font-size: 12px; color: rgba(255,255,255,0.45); margin:
0; line-height: 1.5; }

/* — Effects List — */
.dsm-effects-list { display: flex; flex-direction: column; gap: 10px;
margin-bottom: 16px; }

.dsm-effect-row {
  display: flex;
  align-items: flex-start;
  gap: 14px;
  background: rgba(255,255,255,0.03);
  border: 1px solid rgba(255,255,255,0.07);
  border-radius: 12px;
  padding: 12px 14px;
  animation: dsCardPop 0.4s ease both;
}

.dsm-effect-icon { font-size: 24px; flex-shrink: 0; }
.dsm-effect-row strong { font-size: 13.5px; display: block; margin-bottom:
3px; }

```

```
.dsm-effect-row p { font-size: 12.5px; color: rgba(255,255,255,0.42);
margin: 0; line-height: 1.5; }

/* — Tips Grid — */
.dsm-tips-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 10px;
margin-bottom: 16px; }

.dsm-tip-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid rgba(255,255,255,0.07);
  border-radius: 12px;
  padding: 14px;
  display: flex;
  align-items: flex-start;
  gap: 10px;
  animation: dsCardPop 0.4s ease both;
}

.dsm-tip-icon { font-size: 22px; flex-shrink: 0; }
.dsm-tip-card p { font-size: 12.5px; color: rgba(255,255,255,0.55);
margin: 0; line-height: 1.5; }

/* — During Cols — */
.dsm-during-cols { display: grid; grid-template-columns: 1fr 1fr; gap:
14px; margin-bottom: 16px; }
.dsm-do-header { font-size: 13px; font-weight: 700; margin-bottom: 10px;
letter-spacing: 0.3px; }
.dsm-do-item {
  font-size: 12.5px;
  color: rgba(255,255,255,0.5);
  line-height: 1.5;
  margin-bottom: 8px;
  display: flex;
  gap: 8px;
  animation: dsCardPop 0.4s ease both;
}

/* — Measures Grid — */
.dsm-measures-grid { display: grid; grid-template-columns: repeat(3, 1fr);
gap: 10px; margin-bottom: 16px; }
```

```

.dsm-measure-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid;
  border-radius: 14px;
  padding: 14px 12px;
  text-align: center;
  animation: dsCardPop 0.4s ease both;
}

.dsm-measure-icon { font-size: 26px; display: block; margin-bottom: 8px; }
.dsm-measure-card strong { font-size: 12px; display: block; margin-bottom: 5px; }
.dsm-measure-card p { font-size: 11.5px; color: rgba(255,255,255,0.4); margin: 0; line-height: 1.5; }

/* — Stats Row — */
.dsm-stats-row { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; margin-bottom: 16px; }

.dsm-stat-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid;
  border-radius: 12px;
  padding: 14px;
  text-align: center;
  animation: dsCardPop 0.4s ease both;
}

.dsm-stat-value { font-size: 22px; font-weight: 700; display: block; margin-bottom: 4px; }
.dsm-stat-label { font-size: 11.5px; color: rgba(255,255,255,0.45); line-height: 1.4; }

/* — Fact Badge — */
.dsm-fact-badge {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  border: 1px solid;

```

```
border-radius: 30px;
padding: 7px 16px;
font-size: 13px;
font-weight: 500;
margin-top: 4px;
}

/* — Navigation — */
.dsm-nav {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 14px 24px 20px;
  flex-shrink: 0;
  border-top: 1px solid rgba(255,255,255,0.07);
}

.dsm-nav-btn {
  padding: 10px 22px;
  border-radius: 10px;
  border: 1px solid rgba(255,255,255,0.15);
  background: transparent;
  color: rgba(255,255,255,0.6);
  font-family: 'Sora', sans-serif;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.2s ease;
}

.dsm-nav-btn:disabled { opacity: 0.3; cursor: not-allowed; }
.dsm-nav-btn:hover:not(:disabled) { background: rgba(255,255,255,0.08);
color: #fff; }

.dsm-nav-btn.dsm-next { color: #0a0f1e; }
.dsm-nav-btn.dsm-next:hover:not(:disabled) { opacity: 0.88; transform:
translateY(-1px); }

.dsm-nav-center { flex: 1; display: flex; align-items: center;
justify-content: center; }
```

```

.dsm-nav-hint { font-size: 11.5px; color: rgba(255,255,255,0.25);
text-transform: uppercase; letter-spacing: 0.5px; }

.dsm-finish-btn {
  padding: 10px 24px;
  background: linear-gradient(135deg, #fb923c, #f97316);
  border: none;
  border-radius: 10px;
  color: #0a0f1e;
  font-family: 'Sora', sans-serif;
  font-size: 14px;
  font-weight: 700;
  cursor: pointer;
  transition: all 0.2s ease;
  box-shadow: 0 4px 16px rgba(249, 115, 22, 0.3);
}

.dsm-finish-btn:hover { opacity: 0.9; transform: translateY(-2px); }

/* — Responsive — */
@media (max-width: 600px) {
  .dsm-causes-grid { grid-template-columns: 1fr; }
  .dsm-tips-grid { grid-template-columns: 1fr; }
  .dsm-during-cols { grid-template-columns: 1fr; }
  .dsm-measures-grid { grid-template-columns: 1fr 1fr; }
  .dsm-stats-row { grid-template-columns: 1fr; }
  .dsm-slide { padding: 16px; }
}

import React, { useState, useEffect, useRef } from "react";
import { useNavigate } from "react-router-dom";
import axios from "axios";
import { auth } from "../../firebase";
import "../DesertificationModule.css";

const SLIDES = [
  {
    id: 1,
    type: "intro",
    icon: "🌵",
    tag: "DEFINITION",
  }

```



```
title: "What is Desertification?",
subtitle: "Also known as Land Degradation",
body: "Desertification is the process by which fertile land becomes
desert. It is a type of land degradation in which a relatively dry area of
land becomes increasingly arid, typically losing its bodies of water as
well as vegetation and wildlife. It means irreversible decline in the
biological potential of the land – the soil loses its productivity and
becomes infertile. Onset type: Gradual – occurs over a significant period
of time with proper indications well before the area desertifies.",
fact: { label: "India's Affected Land", value: "82.18 Mha undergoing
desertification", icon: "📊" },
color: "#fb923c",
bg: "rgba(251,146,60,0.08)",
},
{
  id: 2,
  type: "warning",
  icon: "🔔",
  tag: "ONSET & WARNING",
  title: "Warning Signs & Onset Type",
  subtitle: "No specific warning systems – but early indicators exist",
  signs: [
    { icon: "☁️", label: "Scarce Rainfall", desc: "Prolonged periods of
reduced or deficit rainfall signal onset of desertification." },
    { icon: "💧", label: "Surface Water Drying", desc: "Rivers, ponds,
and lakes begin to dry up as the region loses water retention." },
    { icon: "🕒", label: "Groundwater Decline", desc: "Declination of
groundwater levels is a key slow-onset indicator." },
    { icon: "🌾", label: "Vegetation Loss", desc: "Progressive loss of
plant cover, degraded grasslands and scrublands." },
  ],
  fact: { label: "Onset Speed", value: "Gradual – years to decades",
icon: "⌚" },
  color: "#f59e0b",
  bg: "rgba(245,158,11,0.08)",
},
{
  id: 3,
  type: "causes",
  icon: "⚙️",
```

```

tag: "CAUSES",
title: "Major Causes of Desertification",
subtitle: "Human and natural factors combining to destroy fertile
land",
causes: [
  { icon: "🐄", label: "Overgrazing", desc: "Excessive animal grazing
prevents plant regrowth, destroying biome vegetation." },
  { icon: "🪓", label: "Deforestation", desc: "Tree removal for
settlements destroys the foundation of dryland biomes." },
  { icon: "🌾", label: "Poor Farming", desc: "Unsustainable farming
strips soil nutrients, causing progressive land degradation." },
  { icon: "🌡️", label: "Climate Change", desc: "Warmer temperatures
and frequent droughts accelerate the desertification process." },
  { icon: "🏙️", label: "Urbanization", desc: "Development kills plant
life and introduces chemicals that harm the ground." },
  { icon: "⛏️", label: "Mining", desc: "Extracting natural resources
strips soil of nutrients and kills vegetation." },
],
fact: { label: "Biggest Cause in India", value: "Water Erosion
(10.98%)", icon: "💧" },
color: "#fb923c",
bg: "rgba(251,146,60,0.08)",
},
{
  id: 4,
  type: "india_causes",
  icon: "🇮🇳",
  tag: "INDIA – CAUSES",
  title: "Major Causes in India",
  subtitle: "ISRO's Desertification & Land Degradation Atlas 2016",
  effects: [
    { icon: "💧", label: "Water Erosion – 10.98%", desc: "Loss of soil
cover mainly due to rainfall and surface runoff; seen in both hot and cold
desert areas." },
    { icon: "💨", label: "Wind Erosion – 5.55%", desc: "Spread of sand;
removes nutrient-rich topsoil even up to the Himalayan altitudes." },
    { icon: "🌿", label: "Vegetation Degradation – 8.91%", desc:
"Deforestation, shifting cultivation, and degradation in grazing and
grassland areas." },
  ],

```

```

    { icon: "🏠", label: "Salinity – 1.12%", desc: "Soil salinity in
cultivated, irrigated lands; can occur naturally or be human-induced." },
    { icon: "🏗️", label: "Human Settlement – 0.69%", desc: "Mining,
urbanisation, and other human developmental activities." },
    { icon: "🌊", label: "Others – 2.07%", desc: "Water logging, frost
shattering, mass movement, barren and rocky land types." },
  ],
  fact: { label: "Total Degraded Area", value: "120.40 Mha across
India", icon: "📍" },
  color: "#ef4444",
  bg: "rgba(239,68,68,0.08)",
},
{
  id: 5,
  type: "stats",
  icon: "📊",
  tag: "INDIAN CONTEXT",
  title: "Desertification in India",
  subtitle: "Scale and severity across Indian states",
  stats: [
    { value: "32%", label: "of India's land is degraded (UNCCD)", color:
"#ef4444" },
    { value: "81.4 Mha", label: "area affected by desertification",
color: "#fb923c" },
    { value: "105.19 Mha", label: "of total geographical area degraded",
color: "#f59e0b" },
  ],
  effects: [
    { icon: "🌵", label: "Rajasthan", desc: "Worst affected – 204.25
lakh ha degraded; mostly water and wind erosion." },
    { icon: "🌾", label: "Andhra Pradesh", desc: "91.94 lakh ha total
degraded; water erosion (88.64) is the dominant cause." },
    { icon: "🪨", label: "Gujarat", desc: "31.29 lakh ha degraded;
significant saline soil issues (15.59 lakh ha)." },
  ],
  fact: { label: "Most Affected State", value: "Rajasthan", icon: "🌵"
},
  color: "#f59e0b",
  bg: "rgba(245,158,11,0.08)",
},

```

```

{
  id: 6,
  type: "hazard",
  icon: "⚠️",
  tag: "HAZARD ZONES",
  title: "Desertification Hazard Zones",
  subtitle: "Types and categories of land degradation",
  causes: [
    { icon: "💧", label: "Soil Erosion", desc: "Water and wind erode topsoil, permanently reducing agricultural productivity." },
    { icon: "🌿", label: "Land Degradation", desc: "Progressive deterioration of land quality through multiple combined processes." },
    { icon: "🌾", label: "Loss of Soil Fertility", desc: "Nutrients stripped from soil make farming impossible without expensive intervention." },
    { icon: "🌵", label: "Arid Zones", desc: "Hot deserts (Rajasthan) and cold deserts (J&K, Himachal) are highest-risk." },
    { icon: "🌧️", label: "Semi-Arid Zones", desc: "Maharashtra, Karnataka, AP – moderate-risk zones prone to periodic drought." },
    { icon: "🌱", label: "Sub-Humid Dry", desc: "Parts of UP, MP, Bihar – at risk from overuse of agricultural land." },
  ],
  fact: { label: "India's High-Risk Region", value: "Rajasthan, UP, MP, Maharashtra", icon: "🔥" },
  color: "#ef4444",
  bg: "rgba(239,68,68,0.08)",
},
{
  id: 7,
  type: "effects",
  icon: "💥",
  tag: "EFFECTS",
  title: "Effects of Desertification",
  subtitle: "Environmental, social, economic, and human impact",
  effects: [
    { icon: "🌾", label: "Infertile Soil", desc: "Soil loses all biological potential – farming becomes nearly impossible without costly technology." },
    { icon: "🌿", label: "Vegetation Destroyed", desc: "Plant life and biodiversity is damaged or completely wiped out across vast areas." },
  ],

```

```

    { icon: "💧", label: "Polluted Water", desc: "Without vegetation,
water quality worsens; sources become contaminated." },
    { icon: "🍲", label: "Famine & Poverty", desc: "Food scarcity leads
to hunger, poverty, and economic collapse in affected regions." },
    { icon: "🌊", label: "Flooding Risk", desc: "Without plant life,
rainfall causes flooding – nothing stops water from spreading." },
    { icon: "🐦", label: "Extinctions", desc: "Habitat destruction
causes mass migration and species extinction in degraded zones." },
  ],
  fact: { label: "Global Impact", value: "250 million people affected
worldwide", icon: "🌍" },
  color: "#f87171",
  bg: "rgba(248,113,113,0.08)",
},
{
  id: 8,
  type: "unccd",
  icon: "🌐",
  tag: "UNCCD",
  title: "United Nations Convention to Combat Desertification",
  subtitle: "The international framework for tackling land degradation",
  effects: [
    { icon: "📋", label: "UNCCD Framework", desc: "The UN Convention to
Combat Desertification coordinates global action on land degradation and
drought." },
    { icon: "🎯", label: "Land Degradation Neutrality", desc: "Goal:
ensure the amount of degraded land remains stable or improves by 2030." },
    { icon: "🙌", label: "India's Commitment", desc: "India committed to
restore 26 million hectares of degraded land by 2030." },
    { icon: "💰", label: "Financial Mechanisms", desc: "Global
Environment Facility (GEF) provides funding to developing countries for
land restoration." },
    { icon: "🔬", label: "Research & Monitoring", desc: "ISRO's Land
Degradation Atlas maps and monitors India's desertification on a periodic
basis." },
    { icon: "🌱", label: "Great Green Wall", desc: "An African
initiative to restore 100M hectares across the Sahel – a model for India."
  ],
},
],

```

```

    fact: { label: "India's 2030 Goal", value: "Restore 26 million
hectares", icon: "🌳" },
    color: "#3b82f6",
    bg: "rgba(59,130,246,0.08)",
  },
  {
    id: 9,
    type: "during",
    icon: "✅",
    tag: "DO'S & DON'TS",
    title: "What To Do & Avoid",
    subtitle: "Before, during, and after desertification – sustainable
land practices",
    doList: [
      "Agroforestry: integrate trees into farming to improve soil and
reduce erosion",
      "Rainwater Harvesting: collect and store rainwater for later
agricultural use",
      "Sustainable Grazing: rotational grazing to prevent overgrazing and
allow recovery",
      "Conservation Agriculture: no-till farming and cover cropping to
protect topsoil",
      "Reforestation: plant drought-resistant native species to restore
degraded land",
    ],
    dontList: [
      "Avoid cutting trees or clearing vegetation unnecessarily
(deforestation)",
      "Prevent livestock from overgrazing a single area",
      "Avoid monoculture farming and overuse of chemical fertilizers",
      "Don't waste water through inefficient irrigation practices",
      "Don't ignore early warning signs like declining groundwater or
drying surface water",
    ],
    fact: { label: "Best Practice", value: "Sustainable Land Management",
icon: "🌱" },
    color: "#10b981",
    bg: "rgba(16,185,129,0.08)",
  },
  {

```

```

    id: 10,
    type: "mitigation",
    icon: "🛡️",
    tag: "MITIGATION",
    title: "Combating Desertification",
    subtitle: "Sustainable land management, water conservation & policy",
    measures: [
      { icon: "🌳", label: "Reforestation", desc: "Plant drought-resistant species to restore degraded land and improve soil quality." },
      { icon: "💧", label: "Drip Irrigation", desc: "Use efficient irrigation to minimise water waste in arid and semi-arid zones." },
      { icon: "🌾", label: "Soil Erosion Control", desc: "Terracing, contour plowing, and windbreaks prevent topsoil from being washed away." },
      { icon: "🐄", label: "Sustainable Grazing", desc: "Rotational grazing practices prevent overgrazing and allow vegetation to recover." },
      { icon: "🗣️", label: "Community Empowerment", desc: "Educate local communities on sustainable land and water management practices." },
      { icon: "📋", label: "Policy Enforcement", desc: "Implement land-use regulations; restrict deforestation, overgrazing, and mining." },
    ],
    fact: { label: "India's Strategy", value: "UNCCD Land Degradation Neutrality", icon: "🏠" },
    color: "#10b981",
    bg: "rgba(16,185,129,0.08)",
  },
];

```

```

function DesertificationModule({ onClose, moduleId = 7 }) {
  const [current, setCurrent] = useState(0);
  const [animDir, setAnimDir] = useState("forward");
  const [animKey, setAnimKey] = useState(0);
  const [completed, setCompleted] = useState(new Set());
  const [isSaving, setIsSaving] = useState(false);
  const slideRef = useRef(null);
  const navigate = useNavigate();

  const slide = SLIDES[current];
  const progress = ((current + 1) / SLIDES.length) * 100;

```

```

// Function to save progress to backend
const saveProgressToBackend = async (currentProgress, completedSlides)
=> {
  const user = auth.currentUser;
  if (!user || !user.uid) {
    console.warn("No user found, skipping progress save");
    return;
  }

  setIsSaving(true);
  try {
    const completedArray = Array.from(completedSlides);

    const response = await axios.post(
      "http://localhost:5000/api/modules/progress",
      {
        userId: user.uid,
        userName: user.email || "Guest User",
        moduleId: moduleId || 7,
        progress: Math.round(currentProgress),
        completed: completedArray,
        lastSlide: current,
        timestamp: new Date().toISOString(),
      }
    );

    console.log("Progress saved successfully:", response.data);

    // Save to localStorage as backup
    localStorage.setItem(`desert_progress_${user.uid}`, JSON.stringify({
      progress: Math.round(currentProgress),
      completed: completedArray,
      lastSlide: current,
      timestamp: new Date().toISOString()
    }));

  } catch (error) {
    console.error("Error saving progress:", error);
    // Save to localStorage as fallback

```



```

        localStorage.setItem(`desert_progress_backup_${user.uid}`,
JSON.stringify({
    progress: Math.round(currentProgress),
    completed: Array.from(completedSlides),
    lastSlide: current,
    timestamp: new Date().toISOString()
})));
    } finally {
        setIsSaving(false);
    }
};

// Load saved progress when component mounts
useEffect(() => {
    const loadSavedProgress = async () => {
        const user = auth.currentUser;
        if (user && user.uid) {
            try {
                // Try to fetch from backend first
                const response = await
axios.get(`http://localhost:5000/api/modules/progress/${user.uid}/${module
Id}`);

                if (response.data && response.data.progress) {
                    const savedProgress = response.data;
                    if (savedProgress.lastSlide && savedProgress.lastSlide <
SLIDES.length) {
                        setCurrent(savedProgress.lastSlide);
                    }
                    if (savedProgress.completed &&
Array.isArray(savedProgress.completed)) {
                        setCompleted(new Set(savedProgress.completed));
                    }
                    console.log("Loaded progress from backend");
                }
            } catch (error) {
                // If backend fails, try localStorage
                console.log("Loading from localStorage backup");
                const localBackup =
localStorage.getItem(`desert_progress_backup_${user.uid}`);
                if (localBackup) {

```

```

        const saved = JSON.parse(localBackup);
        if (saved.lastSlide && saved.lastSlide < SLIDES.length) {
            setCurrent(saved.lastSlide);
        }
        if (saved.completed && Array.isArray(saved.completed)) {
            setCompleted(new Set(saved.completed));
        }
    }
}

};

loadSavedProgress();
}, [moduleId]);

// Save progress whenever slide changes
useEffect(() => {
    const updatedCompleted = new Set([...completed, current]);
    setCompleted(updatedCompleted);

    // Save to backend
    saveProgressToBackend(progress, updatedCompleted);

    // Also save to localStorage for quick access
    localStorage.setItem(`desert_progress`, Math.round(progress));

}, [current]);

const goNext = () => {
    if (current < SLIDES.length - 1) {
        setAnimDir("forward");
        setAnimKey(k => k + 1);
        setCurrent(c => c + 1);
    }
};

const goPrev = () => {
    if (current > 0) {
        setAnimDir("backward");
        setAnimKey(k => k + 1);
    }
};

```

```

        setCurrent(c => c - 1);
    }
};

const goTo = (idx) => {
    setAnimDir(idx > current ? "forward" : "backward");
    setAnimKey(k => k + 1);
    setCurrent(idx);
};

const handleFinish = () => {
    // Save final progress before navigating
    saveProgressToBackend(100, new Set(Array.from({ length: SLIDES.length
}, (_, i) => i)));
    navigate("/quiz-generator");
};

return (
    <div className="dsm-overlay">
        <div className="dsm-modal">
            { /* Header */ }
            <div className="dsm-header">
                <div className="dsm-header-left">
                    <span className="dsm-module-badge">🌵 Desertification
Module</span>
                    <span className="dsm-slide-count">{current + 1} /
{SLIDES.length}</span>
                </div>
                <div className="dsm-header-right">
                    <span className="dsm-progress-label">{Math.round(progress)}%
complete</span>
                    {isSaving && <span className="dsm-saving-indicator">💾
Saving...</span>}
                    <button className="dsm-close-btn"
onClick={onClose}>✕</button>
                </div>
            </div>

            { /* Progress Bar */ }
            <div className="dsm-progress-track">

```

```

        <div className="dsm-progress-fill" style={{ width:
`${progress}%`, background: slide.color }} />
    </div>

    { /* Slide Dots */ }
    <div className="dsm-dots">
        {SLIDES.map((s, i) => (
            <button
                key={s.id}
                className={`dsm-dot ${i === current ? "active" : ""}
${completed.has(i) ? "done" : ""}`}
                onClick={() => goTo(i)}
                style={i === current ? { background: slide.color,
borderColor: slide.color } : {}}
                title={s.title}
            />
        ))}
    </div>

    { /* Slide Content */ }
    <div
        key={animKey}
        className={`dsm-slide dsm-anim-${animDir}`}
        ref={slideRef}
        style={{ background: slide.bg, borderColor: slide.color + "44"
}}
    >
        <div className="dsm-slide-tag" style={{ color: slide.color,
borderColor: slide.color + "44", background: slide.color + "18" }}>
            {slide.tag}
        </div>
        <div className="dsm-slide-icon">{slide.icon}</div>
        <h2 className="dsm-slide-title">{slide.title}</h2>
        <p className="dsm-slide-subtitle">{slide.subtitle}</p>

        { /* INTRO */ }
        {slide.type === "intro" && <p
className="dsm-body-text">{slide.body}</p>}

        { /* WARNING SIGNS */ }

```

```

    {slide.type === "warning" && (
      <div className="dsm-causes-grid" style={{ gridTemplateColumns:
"1fr 1fr" }}>
        {slide.signs.map((s, i) => (
          <div key={i} className="dsm-cause-card" style={{
borderColor: slide.color + "44", animationDelay: `${i * 0.1}s` }}>
            <span className="dsm-cause-icon">{s.icon}</span>
            <strong className="dsm-cause-label" style={{ color:
slide.color }}>{s.label}</strong>
            <p className="dsm-cause-desc">{s.desc}</p>
          </div>
        ))}
      </div>
    )}

    {/* CAUSES */}
    {(slide.type === "causes" || slide.type === "hazard") && (
      <div className="dsm-causes-grid">
        {slide.causes.map((c, i) => (
          <div key={i} className="dsm-cause-card" style={{
borderColor: slide.color + "44", animationDelay: `${i * 0.1}s` }}>
            <span className="dsm-cause-icon">{c.icon}</span>
            <strong className="dsm-cause-label" style={{ color:
slide.color }}>{c.label}</strong>
            <p className="dsm-cause-desc">{c.desc}</p>
          </div>
        ))}
      </div>
    )}

    {/* EFFECTS LIST (india_causes, effects, unccd) */}
    {(slide.type === "india_causes" || slide.type === "effects" ||
slide.type === "unccd") && (
      <div className="dsm-effects-list">
        {slide.effects.map((e, i) => (
          <div key={i} className="dsm-effect-row" style={{
animationDelay: `${i * 0.08}s` }}>
            <span className="dsm-effect-icon">{e.icon}</span>
            <div>

```

```

        <strong style={{ color: slide.color
}}>{e.label}</strong>
        <p>{e.desc}</p>
    </div>
</div>
    )})
</div>
)}

{/* STATS */}
{slide.type === "stats" && (
    <>
        <div className="dsm-stats-row">
            {slide.stats.map((s, i) => (
                <div key={i} className="dsm-stat-card" style={{
borderColor: s.color + "44", animationDelay: `${i * 0.1}s` }}>
                    <span className="dsm-stat-value" style={{ color:
s.color }}>{s.value}</span>
                    <span className="dsm-stat-label">{s.label}</span>
                </div>
            ))}
        </div>
        <div className="dsm-effects-list">
            {slide.effects.map((e, i) => (
                <div key={i} className="dsm-effect-row" style={{
animationDelay: `${i * 0.08}s` }}>
                    <span className="dsm-effect-icon">{e.icon}</span>
                    <div>
                        <strong style={{ color: slide.color
}}>{e.label}</strong>
                        <p>{e.desc}</p>
                    </div>
                </div>
            ))}
        </div>
    </>
)}

{/* DURING / DO & DONT */}
{slide.type === "during" && (

```

```

        <div className="dsm-during-cols">
            <div>
                <div className="dsm-do-header" style={{ color: "#10b981"
}}>✔ DO</div>
                {slide.doList.map((item, i) => (
                    <div key={i} className="dsm-do-item" style={{
animationDelay: `${i * 0.08}s` }}>
                        <span style={{ color: "#10b981" }}>•</span> {item}
                    </div>
                ))}
            </div>
            <div>
                <div className="dsm-do-header" style={{ color: "#ef4444"
}}>✗ DON'T</div>
                {slide.dontList.map((item, i) => (
                    <div key={i} className="dsm-do-item" style={{
animationDelay: `${i * 0.08}s` }}>
                        <span style={{ color: "#ef4444" }}>•</span> {item}
                    </div>
                ))}
            </div>
        </div>

        {/* MITIGATION */}
        {slide.type === "mitigation" && (
            <div className="dsm-measures-grid">
                {slide.measures.map((m, i) => (
                    <div key={i} className="dsm-measure-card" style={{
borderColor: slide.color + "44", animationDelay: `${i * 0.08}s` }}>
                        <span className="dsm-measure-icon">{m.icon}</span>
                        <strong style={{ color: slide.color
}}>{m.label}</strong>
                        <p>{m.desc}</p>
                    </div>
                ))}
            </div>
        )}

        {/* Fact Badge */}

```

```

        {slide.fact && (
            <div className="dsm-fact-badge" style={{ borderColor:
slide.color, color: slide.color, background: slide.color + "14" }}>
                <span>{slide.fact.icon}</span>
                <span>{slide.fact.label}</span>
                <strong>{slide.fact.value}</strong>
            </div>
        )}
    </div>

    { /* Navigation */
    <div className="dsm-nav">
        <button className="dsm-nav-btn dsm-prev" onClick={goPrev}
disabled={current === 0}>
            ← Previous
        </button>

        <div className="dsm-nav-center">
            {current === SLIDES.length - 1 ? (
                <button className="dsm-finish-btn" onClick={handleFinish}>
                    🎓 Take the Quiz!
                </button>
            ) : (
                <span className="dsm-nav-hint">{SLIDES[current + 1]?.tag}
→</span>
            )}
        </div>

        <button
            className="dsm-nav-btn dsm-next"
            onClick={goNext}
            disabled={current === SLIDES.length - 1}
            style={{ background: slide.color, borderColor: slide.color }}
        >
            Next →
        </button>
    </div>
</div>
);

```



```

}

export default DesertificationModule;

/* =====
   EarthquakeModule.css - Earthquake Learning Slides
   Matches dark glassmorphism theme from TsunamiModule.css
   ===== */

@import
url('https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600;700&display=swap');

/* — Overlay — */
.eqm-overlay {
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.75);
  backdrop-filter: blur(8px);
  -webkit-backdrop-filter: blur(8px);
  z-index: 1000;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 20px;
  animation: eqFadeOverlay 0.3s ease;
}

@keyframes eqFadeOverlay {
  from { opacity: 0; }
  to   { opacity: 1; }
}

/* — Modal — */
.eqm-modal {
  width: 100%;
  max-width: 760px;
  max-height: 92vh;
  background: rgba(10, 15, 30, 0.97);
  border: 1px solid rgba(255, 255, 255, 0.10);
  border-radius: 28px;

```

```

display: flex;
flex-direction: column;
overflow: hidden;
box-shadow: 0 40px 80px rgba(0, 0, 0, 0.7);
animation: eqModalIn 0.4s cubic-bezier(0.22, 1, 0.36, 1);
font-family: 'Sora', sans-serif;
}

@keyframes eqModalIn {
  from { opacity: 0; transform: scale(0.92) translateY(20px); }
  to   { opacity: 1; transform: none; }
}

/* — Header — */
.eqm-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 18px 24px 14px;
  border-bottom: 1px solid rgba(255, 255, 255, 0.07);
  flex-shrink: 0;
}

.eqm-header-left {
  display: flex;
  align-items: center;
  gap: 12px;
}

.eqm-module-badge {
  font-size: 13px;
  font-weight: 600;
  color: #fbbf24;
  background: rgba(251, 191, 36, 0.12);
  border: 1px solid rgba(251, 191, 36, 0.25);
  border-radius: 20px;
  padding: 4px 12px;
}

.eqm-slide-count {

```

```
    font-size: 13px;
    color: rgba(255, 255, 255, 0.35);
}

.eqm-header-right {
    display: flex;
    align-items: center;
    gap: 14px;
}

.eqm-progress-label {
    font-size: 12px;
    color: rgba(255, 255, 255, 0.35);
}

.eqm-close-btn {
    width: 30px;
    height: 30px;
    border-radius: 50%;
    border: 1px solid rgba(255, 255, 255, 0.15);
    background: transparent;
    color: rgba(255, 255, 255, 0.5);
    cursor: pointer;
    font-size: 14px;
    display: flex;
    align-items: center;
    justify-content: center;
    transition: all 0.2s;
}

.eqm-close-btn:hover { background: rgba(255,255,255,0.1); color: #fff; }

/* — Progress Track — */
.eqm-progress-track {
    height: 3px;
    background: rgba(255, 255, 255, 0.07);
    flex-shrink: 0;
}

.eqm-progress-fill {
    height: 100%;
```

```
border-radius: 3px;
transition: width 0.5s ease, background 0.5s ease;
}

/* — Slide Dots — */
.eqm-dots {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 7px;
  padding: 12px 24px 8px;
  flex-shrink: 0;
  flex-wrap: wrap;
}

.eqm-dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  border: 1px solid rgba(255, 255, 255, 0.2);
  background: rgba(255, 255, 255, 0.08);
  cursor: pointer;
  transition: all 0.2s ease;
  padding: 0;
}

.eqm-dot:hover { border-color: rgba(255,255,255,0.5); transform:
scale(1.2); }
.eqm-dot.active { width: 20px; border-radius: 4px; }
.eqm-dot.done { background: rgba(255,255,255,0.2); border-color:
rgba(255,255,255,0.3); }

/* — Slide — */
.eqm-slide {
  flex: 1;
  overflow-y: auto;
  padding: 20px 32px 16px;
  border: 1px solid transparent;
  margin: 0 16px;
  border-radius: 20px;
```

```
    transition: background 0.4s ease, border-color 0.4s ease;
}

.eqm-slide::-webkit-scrollbar { width: 4px; }
.eqm-slide::-webkit-scrollbar-track { background: transparent; }
.eqm-slide::-webkit-scrollbar-thumb { background: rgba(255,255,255,0.1);
border-radius: 4px; }

@keyframes eqSlideInForward {
    from { opacity: 0; transform: translateX(40px); }
    to   { opacity: 1; transform: none; }
}

@keyframes eqSlideInBackward {
    from { opacity: 0; transform: translateX(-40px); }
    to   { opacity: 1; transform: none; }
}

.eqm-anim-forward { animation: eqSlideInForward 0.35s
cubic-bezier(0.22,1,0.36,1); }
.eqm-anim-backward { animation: eqSlideInBackward 0.35s
cubic-bezier(0.22,1,0.36,1); }

/* — Slide Content — */
.eqm-slide-tag {
    font-size: 10.5px;
    font-weight: 700;
    letter-spacing: 1px;
    text-transform: uppercase;
    border: 1px solid;
    border-radius: 20px;
    padding: 3px 10px;
    display: inline-block;
    margin-bottom: 12px;
}

.eqm-slide-icon { font-size: 42px; margin-bottom: 8px; line-height: 1; }
.eqm-slide-title { font-size: 22px; font-weight: 700; color: #f1f5f9;
margin: 0 0 6px; letter-spacing: -0.4px; }
```

```

.eqm-slide-subtitle { font-size: 13.5px; color: rgba(255,255,255,0.45);
margin: 0 0 18px; font-weight: 400; }
.eqm-body-text { font-size: 15px; color: #cbd5e1; line-height: 1.7;
margin: 0 0 16px; }

/* — Causes Grid — */
.eqm-causes-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 12px;
  margin-bottom: 16px;
}

.eqm-cause-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid;
  border-radius: 14px;
  padding: 16px 14px;
  text-align: center;
  animation: eqCardPop 0.4s ease both;
}

@keyframes eqCardPop {
  from { opacity: 0; transform: scale(0.9) translateY(10px); }
  to   { opacity: 1; transform: none; }
}

.eqm-cause-icon { font-size: 30px; display: block; margin-bottom: 8px; }
.eqm-cause-label { font-size: 13px; font-weight: 600; display: block;
margin-bottom: 6px; }
.eqm-cause-desc { font-size: 12px; color: rgba(255,255,255,0.45); margin:
0; line-height: 1.5; }

/* — Steps — */
.eqm-steps { display: flex; flex-direction: column; gap: 0; margin-bottom:
16px; }

.eqm-step { display: flex; flex-direction: column; animation: eqCardPop
0.4s ease both; }

```

```
.eqm-step-body {
  display: flex;
  align-items: flex-start;
  gap: 14px;
  background: rgba(255,255,255,0.03);
  border: 1px solid rgba(255,255,255,0.07);
  border-radius: 12px;
  padding: 14px;
  margin-bottom: 4px;
}

.eqm-step-num {
  width: 28px;
  height: 28px;
  border-radius: 50%;
  color: #0a0f1e;
  font-weight: 700;
  font-size: 13px;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
}

.eqm-step-icon { font-size: 24px; flex-shrink: 0; }
.eqm-step-body strong { font-size: 14px; display: block; margin-bottom: 3px; }
.eqm-step-body p { font-size: 12.5px; color: rgba(255,255,255,0.45); margin: 0; line-height: 1.5; }
.eqm-step-arrow { text-align: center; color: rgba(255,255,255,0.25); font-size: 16px; line-height: 1; margin-bottom: 4px; }

/* — Signs Grid — */
.eqm-signs-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; margin-bottom: 16px; }

.eqm-sign-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid;
  border-radius: 14px;
}
```

```

padding: 16px;
text-align: center;
animation: eqCardPop 0.4s ease both;
}

.eqm-sign-icon { font-size: 28px; display: block; margin-bottom: 6px; }
.eqm-sign-card strong { font-size: 13px; display: block; margin-bottom: 6px; }
.eqm-sign-card p { font-size: 12px; color: rgba(255,255,255,0.4); margin: 0; line-height: 1.5; }

/* — Timeline — */
.eqm-timeline { display: flex; flex-direction: column; gap: 10px; margin-bottom: 16px; }

.eqm-event { display: flex; align-items: flex-start; gap: 14px; animation: eqCardPop 0.4s ease both; }

.eqm-event-year {
  font-size: 12px;
  font-weight: 700;
  border: 1px solid;
  border-radius: 8px;
  padding: 4px 10px;
  white-space: nowrap;
  flex-shrink: 0;
}

.eqm-event-body {
  background: rgba(255,255,255,0.03);
  border: 1px solid rgba(255,255,255,0.07);
  border-radius: 10px;
  padding: 10px 14px;
  flex: 1;
}

.eqm-event-body strong { font-size: 13px; color: #f1f5f9; display: block; margin-bottom: 3px; }
.eqm-event-body p { font-size: 12px; color: rgba(255,255,255,0.45); margin: 0; }

```



```
/* — Effects List — */
.eqm-effects-list { display: flex; flex-direction: column; gap: 10px;
margin-bottom: 16px; }

.eqm-effect-row {
  display: flex;
  align-items: flex-start;
  gap: 14px;
  background: rgba(255,255,255,0.03);
  border: 1px solid rgba(255,255,255,0.07);
  border-radius: 12px;
  padding: 12px 14px;
  animation: eqCardPop 0.4s ease both;
}

.eqm-effect-icon { font-size: 24px; flex-shrink: 0; }
.eqm-effect-row strong { font-size: 13.5px; display: block; margin-bottom:
3px; }
.eqm-effect-row p { font-size: 12.5px; color: rgba(255,255,255,0.42);
margin: 0; line-height: 1.5; }

/* — Tips Grid — */
.eqm-tips-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 10px;
margin-bottom: 16px; }

.eqm-tip-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid rgba(255,255,255,0.07);
  border-radius: 12px;
  padding: 14px;
  display: flex;
  align-items: flex-start;
  gap: 10px;
  animation: eqCardPop 0.4s ease both;
}

.eqm-tip-icon { font-size: 22px; flex-shrink: 0; }
.eqm-tip-card p { font-size: 12.5px; color: rgba(255,255,255,0.55);
margin: 0; line-height: 1.5; }
```

```

/* — During Cols — */
.eqm-during-cols { display: grid; grid-template-columns: 1fr 1fr; gap:
14px; margin-bottom: 16px; }

.eqm-do-header { font-size: 13px; font-weight: 700; margin-bottom: 10px;
letter-spacing: 0.3px; }

.eqm-do-item {
  font-size: 12.5px;
  color: rgba(255,255,255,0.5);
  line-height: 1.5;
  margin-bottom: 8px;
  display: flex;
  gap: 8px;
  animation: eqCardPop 0.4s ease both;
}

/* — Measures Grid — */
.eqm-measures-grid { display: grid; grid-template-columns: repeat(3, 1fr);
gap: 10px; margin-bottom: 16px; }

.eqm-measure-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid;
  border-radius: 14px;
  padding: 14px 12px;
  text-align: center;
  animation: eqCardPop 0.4s ease both;
}

.eqm-measure-icon { font-size: 26px; display: block; margin-bottom: 8px; }
.eqm-measure-card strong { font-size: 12px; display: block; margin-bottom:
5px; }
.eqm-measure-card p { font-size: 11.5px; color: rgba(255,255,255,0.4);
margin: 0; line-height: 1.5; }

/* — Richter Table — */
.eqm-richter-table { width: 100%; border-collapse: collapse;
margin-bottom: 16px; font-size: 12.5px; }

```

```

.eqm-richter-table th { padding: 8px 12px; text-align: left; font-weight:
600; border-bottom: 1px solid rgba(255,255,255,0.1); color:
rgba(255,255,255,0.6); }
.eqm-richter-table td { padding: 8px 12px; border-bottom: 1px solid
rgba(255,255,255,0.05); color: rgba(255,255,255,0.7); }
.eqm-richter-table tr:hover td { background: rgba(255,255,255,0.03); }

/* — Zones Grid — */
.eqm-zones-grid { display: grid; grid-template-columns: repeat(2, 1fr);
gap: 10px; margin-bottom: 16px; }

.eqm-zone-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid;
  border-radius: 12px;
  padding: 14px;
  animation: eqCardPop 0.4s ease both;
}

.eqm-zone-card strong { font-size: 13px; display: block; margin-bottom:
5px; }
.eqm-zone-card p { font-size: 12px; color: rgba(255,255,255,0.45); margin:
0; }

/* — Fact Badge — */
.eqm-fact-badge {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  border: 1px solid;
  border-radius: 30px;
  padding: 7px 16px;
  font-size: 13px;
  font-weight: 500;
  margin-top: 4px;
}

/* — Navigation — */
.eqm-nav {
  display: flex;

```

```
align-items: center;
justify-content: space-between;
padding: 14px 24px 20px;
flex-shrink: 0;
border-top: 1px solid rgba(255,255,255,0.07);
}

.eqm-nav-btn {
padding: 10px 22px;
border-radius: 10px;
border: 1px solid rgba(255,255,255,0.15);
background: transparent;
color: rgba(255,255,255,0.6);
font-family: 'Sora', sans-serif;
font-size: 14px;
font-weight: 600;
cursor: pointer;
transition: all 0.2s ease;
}

.eqm-nav-btn:disabled { opacity: 0.3; cursor: not-allowed; }
.eqm-nav-btn:hover:not(:disabled) { background: rgba(255,255,255,0.08);
color: #fff; }

.eqm-nav-btn.eqm-next { color: #0a0f1e; }
.eqm-nav-btn.eqm-next:hover:not(:disabled) { opacity: 0.88; transform:
translateY(-1px); }

.eqm-nav-center { flex: 1; display: flex; align-items: center;
justify-content: center; }

.eqm-nav-hint { font-size: 11.5px; color: rgba(255,255,255,0.25);
text-transform: uppercase; letter-spacing: 0.5px; }

.eqm-finish-btn {
padding: 10px 24px;
background: linear-gradient(135deg, #fbbf24, #f59e0b);
border: none;
border-radius: 10px;
color: #0a0f1e;
```

```

    font-family: 'Sora', sans-serif;
    font-size: 14px;
    font-weight: 700;
    cursor: pointer;
    transition: all 0.2s ease;
    box-shadow: 0 4px 16px rgba(245, 158, 11, 0.3);
  }

.eqm-finish-btn:hover { opacity: 0.9; transform: translateY(-2px); }

/* — Responsive — */
@media (max-width: 600px) {
  .eqm-causes-grid { grid-template-columns: 1fr; }
  .eqm-signs-grid { grid-template-columns: 1fr; }
  .eqm-tips-grid { grid-template-columns: 1fr; }
  .eqm-during-cols { grid-template-columns: 1fr; }
  .eqm-measures-grid { grid-template-columns: 1fr 1fr; }
  .eqm-zones-grid { grid-template-columns: 1fr; }
  .eqm-slide { padding: 16px; }
}

import React, { useState, useEffect, useRef } from "react";
import { useNavigate } from "react-router-dom";
import axios from "axios"; // Make sure axios is installed
import "./EarthquakeModule.css";
import { auth } from "../../firebase";

const SLIDES = [
  {
    id: 1,
    type: "intro",
    icon: "🌋",
    tag: "DEFINITION",
    title: "What is an Earthquake?",
    subtitle: "A sudden release of energy in the Earth's lithosphere",
    body: "An earthquake (also known as a quake, tremor or temblor) is the shaking of the surface of the Earth resulting from a sudden release of energy in the Earth's lithosphere that creates seismic waves. The focus is the point where the earthquake's motion starts, and the epicenter is the"
  }

```

```

point on the Earth's surface directly above the focus. Onset type: Sudden
- no sufficient time to react and escape.",
  fact: { label: "Richter Scale Created", value: "1935 by Charles
Richter", icon: "🔪" },
  color: "#f59e0b",
  bg: "rgba(245,158,11,0.08)",
},
{
  id: 2,
  type: "causes",
  icon: "🌍",
  tag: "CAUSES",
  title: "Causes of Earthquakes",
  subtitle: "Six primary mechanisms that trigger seismic activity",
  causes: [
    { icon: "🔗", label: "Tectonic Plates", desc: "Convergent, Divergent
& Transform (slip) movement of tectonic plates." },
    { icon: "🌋", label: "Volcanic Eruption", desc: "Convective currents
in mantle & volcanic eruptions release energy." },
    { icon: "🔄", label: "Elastic Rebound", desc: "Stress builds in rock
until it snaps - energy radiates as seismic waves." },
    { icon: "🪨", label: "Faults & Folds", desc: "Dip-slip and
strike-slip fault movements along rupture surfaces." },
    { icon: "🏗️", label: "Dam Construction", desc: "Reservoir-induced
seismicity from huge water storage impoundment." },
    { icon: "💣", label: "Other Causes", desc: "Mining, underground
explosions, and other human-induced triggers." },
  ],
  fact: { label: "Primary Cause", value: "Tectonic Plate Movement",
icon: "🔗" },
  color: "#f59e0b",
  bg: "rgba(245,158,11,0.08)",
},
{
  id: 3,
  type: "types",
  icon: "📊",
  tag: "TYPES & MEASUREMENT",
  title: "Types & How We Measure",
  subtitle: "Classification by focal depth and the Richter Scale",

```

```

steps: [
  { num: "1", icon: "🌋", title: "Shallow (< 60 km)", desc: "Most destructive type – closest to surface, causes maximum ground shaking." },
  { num: "2", icon: "🌐", title: "Medium (60–300 km)", desc: "Moderate impact; tremors widely felt but damage less concentrated." },
  { num: "3", icon: "🌊", title: "Deep (300–700 km)", desc: "Less surface damage but can still trigger tsunamis if undersea." },
  { num: "4", icon: "📡", title: "Seismometer", desc: "Instrument that detects vibrations via seismic waves through the crust." },
],
fact: { label: "Largest Recorded", value: "M 9.25 – Chile 1960", icon: "📊" },
color: "#ef4444",
bg: "rgba(239,68,68,0.08)",
},
{
  id: 4,
  type: "richter",
  icon: "📏",
  tag: "RICHTER SCALE",
  title: "Richter Scale of Magnitude",
  subtitle: "Logarithmic scale – each step is 10× stronger",
  table: [
    { mag: "< 3.0", cat: "Micro", effect: "Not felt by people; recorded on instruments" },
    { mag: "3.0–3.9", cat: "Minor", effect: "Felt by many people; no damage" },
    { mag: "4.0–4.9", cat: "Light", effect: "Felt by all; minor breakage of objects" },
    { mag: "5.0–5.9", cat: "Moderate", effect: "Some damage to weak structures" },
    { mag: "6.0–6.9", cat: "Strong", effect: "Moderate damage in populated areas" },
    { mag: "7.0–7.9", cat: "Major", effect: "Serious damage over large areas; loss of life" },
    { mag: "8.0+", cat: "Great", effect: "Severe destruction over large areas; mass casualties" },
  ],
  fact: { label: "Scale Type", value: "Logarithmic (base 10)", icon: "🔢" },
}

```

```

    color: "#8b5cf6",
    bg: "rgba(139,92,246,0.08)",
  },
  {
    id: 5,
    type: "hazard",
    icon: "⚠️",
    tag: "HAZARD ZONES",
    title: "Earthquake Hazard Zones",
    subtitle: "Direct and indirect hazards from seismic events",
    effects: [
      { icon: "🌍", label: "Ground Shaking", desc: "The primary direct hazard – responsible for most earthquake-related deaths and damage." },
      { icon: "🪨", label: "Soil Liquefaction", desc: "Saturated soil loses strength and behaves like liquid during shaking." },
      { icon: "🏔️", label: "Landslides & Avalanches", desc: "Slope instability triggered by strong shaking; rocks can roll considerable distances." },
      { icon: "🌊", label: "Tsunamis", desc: "Floods from tidal waves, sea surges; undersea quakes displace massive water volumes." },
      { icon: "🏰", label: "Dam Failures", desc: "Indirect hazard – compromised dams release catastrophic downstream flooding." },
      { icon: "☢️", label: "Industrial Pollution", desc: "Damage to industrial plants causes chemical spills and environmental contamination." }
    ],
    fact: { label: "Most Damage From", value: "Strong Ground Shaking", icon: "🌍" },
    color: "#ef4444",
    bg: "rgba(239,68,68,0.08)",
  },
  {
    id: 6,
    type: "india",
    icon: "🇮🇳",
    tag: "INDIAN CONTEXT",
    title: "Earthquakes in India",
    subtitle: "India lies on the Alpine-Himalayan Belt – highly seismically active",
    events: [

```



```

    { year: "1819", loc: "Kutch, Gujarat", note: "M8.0 – one of India's
most powerful historic quakes" },
    { year: "1897", loc: "Shillong Plateau", note: "M8.7 – catastrophic
damage across Assam region" },
    { year: "1950", loc: "Arunachal Pradesh", note: "M8.5 – massive
quake near the China border" },
    { year: "1993", loc: "Latur, Maharashtra", note: "M6.3 – 10,000+
lives lost due to poor construction" },
    { year: "2001", loc: "Bhuj, Gujarat", note: "M6.9 – 13,805 deaths;
1.2 million homes damaged" },
    { year: "2025", loc: "Myanmar (felt in India)", note: "M7.7 – 5,360+
fatalities; felt across NE India" },
  ],
  fact: { label: "Indian Plate Speed", value: "5 cm northward per year",
icon: "📍" },
  color: "#8b5cf6",
  bg: "rgba(139,92,246,0.08)",
},
{
  id: 7,
  type: "zones",
  icon: "🗺️",
  tag: "SEISMIC ZONES",
  title: "India's Seismic Zone Map",
  subtitle: "Bureau of Indian Standards classifies India into 4 seismic
zones",
  zones: [
    { label: "Zone II – Low Risk", desc: "Seismic disturbances up to
Magnitude 4.9. Peninsular India, stable Deccan plateau.", color: "#3b82f6"
},
    { label: "Zone III – Moderate Risk", desc: "Quakes up to Magnitude
6.9. Parts of Maharashtra, West Bengal, UP, and Kerala.", color: "#10b981"
},
    { label: "Zone IV – High Risk", desc: "Quakes up to Magnitude 7.9.
J&K, Delhi, parts of Himachal and UP-Nepal border.", color: "#f59e0b" },
    { label: "Zone V – Very High Risk", desc: "Magnitude 8+ quakes
likely. NE India, J&K, Himachal Pradesh, Andaman & Nicobar.", color:
"#ef4444" },
  ],
},

```

```
    fact: { label: "Highest Risk Zone", value: "Zone V – NE India & J&K",
icon: "🔥" },
    color: "#f59e0b",
    bg: "rgba(245,158,11,0.08)",
  },
  {
    id: 8,
    type: "effects",
    icon: "💥",
    tag: "EFFECTS",
    title: "Typical Effects of Earthquakes",
    subtitle: "Physical, social, environmental, and infrastructural
impact",
    effects: [
      { icon: "🏠", label: "Physical Damage", desc: "Buildings collapse,
roads crack, bridges fail – especially non-engineered structures." },
      { icon: "⚡", label: "Electricity & Communication", desc:
"Transmission towers, transponders, transformers all collapse; blackouts
ensue." },
      { icon: "💧", label: "Water Supply Failure", desc: "Water
distribution networks rupture; reservoirs and fire hydrant lines fail." },
      { icon: "🚗", label: "Transport Network", desc: "Roads, bridges,
railway tracks, and airport runways suffer severe damage." },
      { icon: "🏥", label: "Public Health Crisis", desc: "95% of deaths
come from building collapse; epidemics follow sanitation breakdown." },
      { icon: "🔥", label: "Secondary Hazards", desc: "Fires, landslides,
chemical spills, and dam failures compound primary destruction." },
    ],
    fact: { label: "Deaths from Collapse", value: "~95% of all
casualties", icon: "💀" },
    color: "#f87171",
    bg: "rgba(248,113,113,0.08)",
  },
  {
    id: 9,
    type: "during",
    icon: "🧑",
    tag: "DURING AN EARTHQUAKE",
    title: "What To Do During",
    subtitle: "Every second matters – act immediately and calmly",
```

```

doList: [
  "DROP, COVER, and HOLD ON – the most effective survival action",
  "Indoors: get under a table or against an interior wall",
  "Keep away from windows, chimneys, and heavy shelves",
  "If in bed: stay in bed, cover your head with a pillow",
  "If driving: pull over away from flyovers, slopes, and electric
poles",
],
dontList: [
  "Don't rush outside during shaking – most injuries happen in
doorways",
  "Don't use elevators",
  "Don't re-enter damaged buildings after shaking stops",
  "Don't touch downed electric wires or metal objects near them",
],
fact: { label: "Best Action", value: "DROP · COVER · HOLD ON", icon:
"🛡️" },
color: "#f59e0b",
bg: "rgba(245,158,11,0.08)",
},
{
  id: 10,
  type: "before",
  icon: "✅",
  tag: "BEFORE AN EARTHQUAKE",
  title: "How to Prepare",
  subtitle: "Preparedness begins long before the ground shakes",
  tips: [
    { icon: "🔦", text: "Keep a torch light and a working transistor
radio with spare batteries." },
    { icon: "☎️", text: "Keep updated emergency numbers: Doctor, Fire,
Police, Ambulance." },
    { icon: "🏠", text: "Arrange your home so you can move around
easily; secure heavy objects." },
    { icon: "⚡", text: "Teach all family members how to turn off
electricity and gas supply." },
    { icon: "🏠", text: "Place heavy objects on the floor or lower
shelves to prevent injury." },
    { icon: "📖", text: "Learn about causes and effects; practice DROP,
COVER, HOLD ON drills." },

```

```

    ],
    fact: { label: "Golden Rule", value: "DROP · COVER · HOLD ON", icon:
"🏆" },
    color: "#10b981",
    bg: "rgba(16,185,129,0.08)",
  },
  {
    id: 11,
    type: "after",
    icon: "🔧",
    tag: "AFTER AN EARTHQUAKE",
    title: "Be Safe After",
    subtitle: "Aftershocks can collapse already-weakened structures",
    tips: [
      { icon: "📻", text: "Switch on transistor radio and obey
instructions. Expect aftershocks." },
      { icon: "🔦", text: "Use a torch – do NOT turn on electric switches
if wired inside." },
      { icon: "🚫", text: "Do not drink water from open containers without
filtering or purification." },
      { icon: "🧯", text: "If there is a fire, try to put it out with help
of people around you." },
      { icon: "🏠", text: "Keep roads clear for relief and rescue teams;
avoid damaged structures." },
      { icon: "📷", text: "Document damage; call rescue teams if people
are buried under debris." },
    ],
    fact: { label: "Hidden Danger", value: "Aftershocks & Gas Leaks",
icon: "⚠️" },
    color: "#3b82f6",
    bg: "rgba(59,130,246,0.08)",
  },
  {
    id: 12,
    type: "mitigation",
    icon: "🛡️",
    tag: "MITIGATION",
    title: "Mitigation & Safety Measures",
    subtitle: "Engineering, community, and policy approaches",
    measures: [

```

```

    { icon: "🏗️", label: "Engineered Structures", desc: "Design buildings to withstand ground shaking; avoid construction on soft soil." },
    { icon: "📋", label: "Indian Standard Codes", desc: "IS 1893 – Bureau of Indian Standards criteria for earthquake-resistant design." },
    { icon: "🏠", label: "Retrofitting", desc: "Upgrade existing lifeline buildings (hospitals, schools) with earthquake safety." },
    { icon: "📢", label: "Public Awareness", desc: "Training for architects, engineers, builders, masons, and government officials." },
    { icon: "🗺️", label: "Land Use Planning", desc: "Enforce byelaws restricting construction on fault lines and soft alluvial soils." },
    { icon: "👥", label: "Community Drills", desc: "Earthquake drills and community-based risk management programmes save lives." },
  ],
  fact: { label: "India's Standard", value: "IS 1893 (BIS)", icon: "🏢" },
},
color: "#10b981",
bg: "rgba(16,185,129,0.08)",
},
];

```

```

function EarthquakeModule({ onClose, moduleId, user }) { // 👉 Add user prop
  const [current, setCurrent] = useState(0);
  const [animDir, setAnimDir] = useState("forward");
  const [animKey, setAnimKey] = useState(0);
  const [completed, setCompleted] = useState(new Set());
  const [isSaving, setIsSaving] = useState(false); // Optional: for loading state
  const slideRef = useRef(null);
  const navigate = useNavigate();

  const slide = SLIDES[current];
  const progress = ((current + 1) / SLIDES.length) * 100;

  // Function to save progress to backend
  const saveProgressToBackend = async (currentProgress, completedSlides) => {
    if (!user || !user.uid) {
      console.warn("No user found, skipping progress save");
    }
  }

```

```

        return;
    }

    setIsSaving(true);
    try {
        // Convert completed Set to array for storage
        const completedArray = Array.from(completedSlides);

        const currentUser = auth.currentUser;

        console.log("Firebase User:", currentUser);

        const response = await axios.post(
            "http://localhost:5000/api/modules/progress",
            {
                userId: currentUser?.uid || "unknown_user",

                // save email as username
                userName: currentUser?.email || "Guest User",

                moduleId: moduleId || 1,

                progress: Math.round(currentProgress),

                completed: completedArray,

                lastSlide: current,

                timestamp: new Date().toISOString(),
            }
        );

        console.log("Progress saved successfully:", response.data); // Also save
        // to localStorage as backup
        localStorage.setItem(`earthquake_progress_${user.uid}`,
            JSON.stringify({
                progress: Math.round(currentProgress),
                completed: completedArray,
                lastSlide: current,
                timestamp: new Date().toISOString()
            }));
    } catch (error) {
        console.error("Error saving progress:", error);
    }
}

```

```

    }));

    } catch (error) {
      console.error("Error saving progress:", error);
      // Save to localStorage as fallback
      localStorage.setItem(`earthquake_progress_backup_${user.uid}`,
JSON.stringify({
      progress: Math.round(currentProgress),
      completed: Array.from(completedSlides),
      lastSlide: current,
      timestamp: new Date().toISOString()
    }));
    } finally {
      setIsSaving(false);
    }
  };

  // Load saved progress when component mounts
  useEffect(() => {
    const loadSavedProgress = async () => {
      if (user && user.uid) {
        try {
          // Try to fetch from backend first
          const response = await
axios.get(`http://localhost:5000/api/modules/progress/${user.uid}/${module
Id || 1}`);
          if (response.data && response.data.progress) {
            const savedProgress = response.data;
            if (savedProgress.lastSlide && savedProgress.lastSlide <
SLIDES.length) {
              setCurrent(savedProgress.lastSlide);
            }
            if (savedProgress.completed &&
Array.isArray(savedProgress.completed)) {
              setCompleted(new Set(savedProgress.completed));
            }
            console.log("Loaded progress from backend");
          }
        } catch (error) {
          // If backend fails, try localStorage

```

```

        console.log("Loading from localStorage backup");
        const localBackup =
localStorage.getItem(`earthquake_progress_backup_${user.uid}`);
        if (localBackup) {
            const saved = JSON.parse(localBackup);
            if (saved.lastSlide && saved.lastSlide < SLIDES.length) {
                setCurrent(saved.lastSlide);
            }
            if (saved.completed && Array.isArray(saved.completed)) {
                setCompleted(new Set(saved.completed));
            }
        }
    }
}

loadSavedProgress();
}, [user, moduleId]);

// Save progress whenever slide changes
useEffect(() => {
    const updatedCompleted = new Set([...completed, current]);
    setCompleted(updatedCompleted);

    // Save to backend
    saveProgressToBackend(progress, updatedCompleted);

    // Also save to localStorage for quick access
    localStorage.setItem(`earthquake_progress`, Math.round(progress));

}, [current]);

const goNext = () => {
    if (current < SLIDES.length - 1) {
        setAnimDir("forward");
        setAnimKey(k => k + 1);
        setCurrent(c => c + 1);
    }
};

```



```

const goPrev = () => {
  if (current > 0) {
    setAnimDir("backward");
    setAnimKey(k => k + 1);
    setCurrent(c => c - 1);
  }
};

const goTo = (idx) => {
  setAnimDir(idx > current ? "forward" : "backward");
  setAnimKey(k => k + 1);
  setCurrent(idx);
};

const handleFinish = () => {
  // Save final progress before navigating
  saveProgressToBackend(100, new Set(Array.from({ length: SLIDES.length
}, (_, i) => i)));
  navigate("/quiz-generator");
};

return (
  <div className="eqm-overlay">
    <div className="eqm-modal">
      {/* Header */}
      <div className="eqm-header">
        <div className="eqm-header-left">
          <span className="eqm-module-badge">🌋 Earthquake Module</span>
          <span className="eqm-slide-count">{current + 1} /
{SLIDES.length}</span>
        </div>
        <div className="eqm-header-right">
          <span className="eqm-progress-label">{Math.round(progress)}%
complete</span>
          {isSaving && <span className="eqm-saving-indicator">💾
Saving...</span>}
          <button className="eqm-close-btn"
onClick={onClose}>✕</button>
        </div>
      </div>
    </div>
  )

```

```

    {/* Progress Bar */}
    <div className="eqm-progress-track">
      <div className="eqm-progress-fill" style={{ width:
`${progress}%`, background: slide.color }} />
    </div>

    {/* Slide Dots */}
    <div className="eqm-dots">
      {SLIDES.map((s, i) => (
        <button
          key={s.id}
          className={`eqm-dot ${i === current ? "active" : ""}
${completed.has(i) ? "done" : ""}`}
          onClick={() => goTo(i)}
          style={i === current ? { background: slide.color,
borderColor: slide.color } : {}}
          title={s.title}
        />
      ))}
    </div>

    {/* Slide Content */}
    <div
      key={animKey}
      className={`eqm-slide eqm-anim-${animDir}`}
      ref={slideRef}
      style={{ background: slide.bg, borderColor: slide.color + "44"
}}
    >
      <div className="eqm-slide-tag" style={{ color: slide.color,
borderColor: slide.color + "44", background: slide.color + "18" }}>
        {slide.tag}
      </div>
      <div className="eqm-slide-icon">{slide.icon}</div>
      <h2 className="eqm-slide-title">{slide.title}</h2>
      <p className="eqm-slide-subtitle">{slide.subtitle}</p>

    {/* INTRO */}

```

```

    {slide.type === "intro" && <p
className="eqm-body-text">{slide.body}</p>}

    {/* CAUSES */}
    {slide.type === "causes" && (
      <div className="eqm-causes-grid">
        {slide.causes.map((c, i) => (
          <div key={i} className="eqm-cause-card" style={{
borderColor: slide.color + "44", animationDelay: `${i * 0.1}s` }}>
            <span className="eqm-cause-icon">{c.icon}</span>
            <strong className="eqm-cause-label" style={{ color:
slide.color }}>{c.label}</strong>
            <p className="eqm-cause-desc">{c.desc}</p>
          </div>
        ))}
      </div>
    )}

    {/* TYPES STEPS */}
    {slide.type === "types" && (
      <div className="eqm-steps">
        {slide.steps.map((s, i) => (
          <div key={i} className="eqm-step" style={{ animationDelay:
`${i * 0.12}s` }}>
            <div className="eqm-step-body">
              <div className="eqm-step-num" style={{ background:
slide.color }}>{s.num}</div>
              <span className="eqm-step-icon">{s.icon}</span>
              <div>
                <strong style={{ color: slide.color
}}>{s.title}</strong>
                <p>{s.desc}</p>
              </div>
            </div>
            {i < slide.steps.length - 1 && <div
className="eqm-step-arrow">↓</div>}
          </div>
        ))}
      </div>
    )}

```

```

    { /* RICHTER TABLE */
    {slide.type === "richter" && (
      <table className="eqm-richter-table">
        <thead>
          <tr>
            <th>Magnitude</th>
            <th>Category</th>
            <th>Effects</th>
          </tr>
        </thead>
        <tbody>
          {slide.table.map((row, i) => (
            <tr key={i}>
              <td style={{ color: slide.color, fontWeight: 600
}}>{row.mag}</td>
              <td>{row.cat}</td>
              <td>{row.effect}</td>
            </tr>
          ))}
        </tbody>
      </table>
    )}

    { /* HAZARD / EFFECTS */
    {(slide.type === "hazard" || slide.type === "effects") && (
      <div className="eqm-effects-list">
        {slide.effects.map((e, i) => (
          <div key={i} className="eqm-effect-row" style={{
animationDelay: `${i * 0.08}s` }}>
            <span className="eqm-effect-icon">{e.icon}</span>
            <div>
              <strong style={{ color: slide.color
}}>{e.label}</strong>
              <p>{e.desc}</p>
            </div>
          </div>
        ))}
      </div>
    )}

```

```

    { /* INDIA TIMELINE */
    {slide.type === "india" && (
      <div className="eqm-timeline">
        {slide.events.map((e, i) => (
          <div key={i} className="eqm-event" style={{
animationDelay: `${i * 0.1}s` }}>
            <div className="eqm-event-year" style={{ color:
slide.color, borderColor: slide.color }}>{e.year}</div>
            <div className="eqm-event-body">
              <strong>{e.loc}</strong>
              <p>{e.note}</p>
            </div>
          </div>
        ))}
      </div>
    )}

    { /* ZONES */
    {slide.type === "zones" && (
      <div className="eqm-zones-grid">
        {slide.zones.map((z, i) => (
          <div key={i} className="eqm-zone-card" style={{
borderColor: z.color + "55", animationDelay: `${i * 0.1}s` }}>
            <strong style={{ color: z.color }}>{z.label}</strong>
            <p>{z.desc}</p>
          </div>
        ))}
      </div>
    )}

    { /* BEFORE / AFTER TIPS */
    {(slide.type === "before" || slide.type === "after") && (
      <div className="eqm-tips-grid">
        {slide.tips.map((t, i) => (
          <div key={i} className="eqm-tip-card" style={{
animationDelay: `${i * 0.08}s` }}>
            <span className="eqm-tip-icon">{t.icon}</span>
            <p>{t.text}</p>
          </div>
        ))}
      </div>
    )}

```

```

    )))
  </div>
})

{/* DURING */}
{slide.type === "during" && (
  <div className="eqm-during-cols">
    <div>
      <div className="eqm-do-header" style={{ color: "#10b981"
}}>✔ DO</div>
      {slide.doList.map((item, i) => (
        <div key={i} className="eqm-do-item" style={{
animationDelay: `${i * 0.08}s` }}>
          <span style={{ color: "#10b981" }}>•</span> {item}
        </div>
      )))
    </div>
    <div>
      <div className="eqm-do-header" style={{ color: "#ef4444"
}}>✗ DON'T</div>
      {slide.dontList.map((item, i) => (
        <div key={i} className="eqm-do-item" style={{
animationDelay: `${i * 0.08}s` }}>
          <span style={{ color: "#ef4444" }}>•</span> {item}
        </div>
      )))
    </div>
  </div>
})

{/* MITIGATION */}
{slide.type === "mitigation" && (
  <div className="eqm-measures-grid">
    {slide.measures.map((m, i) => (
      <div key={i} className="eqm-measure-card" style={{
borderColor: slide.color + "44", animationDelay: `${i * 0.08}s` }}>
        <span className="eqm-measure-icon">{m.icon}</span>
        <strong style={{ color: slide.color
}}>{m.label}</strong>
        <p>{m.desc}</p>
      </div>
    ))}
  </div>
})

```

```

        </div>
    )))
</div>
))

{/* Fact Badge */}
{slide.fact && (
    <div className="eqm-fact-badge" style={{ borderColor:
slide.color, color: slide.color, background: slide.color + "14" }}>
        <span>{slide.fact.icon}</span>
        <span>{slide.fact.label}</span>
        <strong>{slide.fact.value}</strong>
    </div>
)}
</div>

{/* Navigation */}
<div className="eqm-nav">
    <button className="eqm-nav-btn eqm-prev" onClick={goPrev}
disabled={current === 0}>
        ← Previous
    </button>

    <div className="eqm-nav-center">
        {current === SLIDES.length - 1 ? (
            <button className="eqm-finish-btn" onClick={handleFinish}>
                🎓 Take the Quiz!
            </button>
        ) : (
            <span className="eqm-nav-hint">{SLIDES[current + 1]?.tag}
→</span>
        )}
    </div>

    <button
        className="eqm-nav-btn eqm-next"
        onClick={goNext}
        disabled={current === SLIDES.length - 1}
        style={{ background: slide.color, borderColor: slide.color }}
    >

```

```

        Next →
      </button>
    </div>
  </div>
</div>
);
}

export default EarthquakeModule;
import React, { useState, useEffect, useRef } from "react";
import { auth } from "../../firebase";
import {
  signInWithEmailAndPassword,
  GoogleAuthProvider,
  signInWithPopup,
} from "firebase/auth";
import { useNavigate, Link } from "react-router-dom";
import { db } from "../../firebase";
import { doc, getDoc } from "firebase/firestore";
import "../../Auth.css";

const PARTICLES = Array.from({ length: 18 }, (_, i) => ({
  id: i,
  x: Math.random() * 100,
  y: Math.random() * 100,
  size: 2 + Math.random() * 4,
  duration: 6 + Math.random() * 10,
  delay: Math.random() * 5,
})));

const FEATURES = [
  {
    icon: (
      <svg width="22" height="22" viewBox="0 0 24 24" fill="currentColor">
        <path d="M21 5c-1.11-.35-2.33-.5-3.5-.5-1.95 0-4.05.4-5.5
1.5-1.45-1.1-3.55-1.5-5.5-1.5S2.45 4.9 1 6v14.65c0 .25.25.5.5.1 0
.15-.05.25-.05C3.1 20.45 5.05 20 6.5 20c1.95 0 4.05.4 5.5 1.5 1.35-.85
3.8-1.5 5.5-1.5 1.65 0 3.35.3 4.75 1.05.1.05.15.05.25.05.25 0
.5-.25.5-.5V6c-.6-.45-1.25-.75-2-1z" />
      </svg>
    )
  }
]

```



```

    ),
    color: "#f59e0b",
    title: "Interactive Learning",
    desc: "Modules & Quizzes",
  },
  {
    icon: (
      <svg width="22" height="22" viewBox="0 0 24 24" fill="currentColor">
        <path d="M12 1L3 5v6c0 5.55 3.84 10.74 9 12 5.16-1.26 9-6.45
9-12V5l-9-4z" />
      </svg>
    ),
    color: "#6ee7b7",
    title: "Virtual Drills",
    desc: "Practice Scenarios",
  },
  {
    icon: (
      <svg width="22" height="22" viewBox="0 0 24 24" fill="currentColor">
        <path d="M19 3H5c-1.1 0-2 .9-2 2v14c0 1.1.9 2 2 2h14c1.1 0 2-.9
2-2V5c0-1.1-.9-2-2-2zM9 17H7v-7h2v7zm4 0h-2V7h2v10zm4 0h-2v-4h2v4z" />
      </svg>
    ),
    color: "#3b82f6",
    title: "Real-time Analytics",
    desc: "Track Progress",
  },
  {
    icon: (
      <svg width="22" height="22" viewBox="0 0 24 24" fill="currentColor">
        <path d="M12 22c1.1 0 2-.9 2-2h-4c0 1.1.9 2 2 2
2zm6-6v-5c0-3.07-1.64-5.64-4.5-6.32V4c0-.83-.67-1.5-1.5-1.5s-1.5-.67-1.5
1.5v.68C7.63 5.36 6 7.92 6 11v5l-2 2v1h16v-11-2-2z" />
      </svg>
    ),
    color: "#f87171",
    title: "SOS Alerts",
    desc: "Emergency Response",
  },
];

```

```

function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [role, setRole] = useState("student");
  const [loading, setLoading] = useState(false);
  const [showPassword, setShowPassword] = useState(false);
  const [errors, setErrors] = useState({});
  const [touched, setTouched] = useState({});
  const [shake, setShake] = useState(false);
  const [successAnim, setSuccessAnim] = useState(false);

  const navigate = useNavigate();

  const validate = () => {
    const e = {};
    if (!email) e.email = "Email is required";
    else if (!/^([^\s@]+@[^\s@]+\.[^\s@]+)$/.test(email)) e.email = "Invalid email format";
    if (!password) e.password = "Password is required";
    else if (password.length < 6) e.password = "Minimum 6 characters";
    return e;
  };

  const handleBlur = (field) => {
    setTouched((prev) => ({ ...prev, [field]: true }));
    const e = validate();
    setErrors((prev) => ({ ...prev, [field]: e[field] }));
  };

  const triggerShake = () => {
    setShake(true);
    setTimeout(() => setShake(false), 600);
  };

  const handleEmailLogin = async () => {
    setTouched({ email: true, password: true });
    const e = validate();
    setErrors(e);
    if (Object.keys(e).length > 0) { triggerShake(); return; }
  };

```

```

    setLoading(true);
    try {
        const userCredential = await signInWithEmailAndPassword(auth, email,
password);
        const user = userCredential.user;

        // 🔥 GET ROLE FROM FIRESTORE
        const docRef = doc(db, "users", user.uid);
        const docSnap = await getDoc(docRef);

        if (!docSnap.exists()) {
            throw new Error("User role not found");
        }

        const savedRole = docSnap.data().role;

        // 🔥 CHECK ROLE MATCH
        if (savedRole !== role) {
            setErrors({ general: "You are not allowed to login as this role
❌" });
            triggerShake();
            setLoading(false);
            return;
        }

        // ✅ Navigate based on REAL role
        setSuccessAnim(true);
        setTimeout(() => navigate(savedRole === "admin" ? "/admin" :
"/student"), 800);
    } catch (error) {
        const msgs = {
            "auth/user-not-found": "No account found with this email",
            "auth/wrong-password": "Incorrect password",
            "auth/too-many-requests": "Too many attempts – try later",
            "auth/invalid-credential": "Invalid credentials",
        };
        setErrors({ general: msgs[error.code] || error.message || "Login
failed. Please try again." });
        triggerShake();
    } finally {

```

```

        setLoading(false);
    }
};

const handleGoogleLogin = async () => {
    setLoading(true);
    try {
        const provider = new GoogleAuthProvider();
        const result = await signInWithPopup(auth, provider);
        const user = result.user;

        const docRef = doc(db, "users", user.uid);
        const docSnap = await getDoc(docRef);

        if (!docSnap.exists()) {
            setErrors({ general: "No role assigned to this account ❌" });
            triggerShake();
            setLoading(false);
            return;
        }

        const savedRole = docSnap.data().role;

        if (savedRole !== role) {
            setErrors({ general: "Wrong role selected ❌" });
            triggerShake();
            setLoading(false);
            return;
        }

        setSuccessAnim(true);
        setTimeout(() => navigate(savedRole === "admin" ? "/admin" :
"/student"), 800);
    } catch (error) {
        const msgs = {
            "auth/popup-closed-by-user": "Sign-in popup was closed",
            "auth/unknown-domain": "Domain not authorized in Firebase",
        };
        setErrors({ general: msgs[error.code] || "Google sign-in failed."
});
    }
};

```

```

        triggerShake();
    } finally {
        setLoading(false);
    }
};

const handleKeyDown = (e) => {
    if (e.key === "Enter") handleEmailLogin();
};

return (
    <div className="auth-bg split-layout">
        {/* Particles */}
        <div className="particles" aria-hidden="true">
            {PARTICLES.map((p) => (
                <span
                    key={p.id}
                    className="particle"
                    style={{
                        left: `${p.x}%`,
                        top: `${p.y}%`,
                        width: p.size,
                        height: p.size,
                        animationDuration: `${p.duration}s`,
                        animationDelay: `${p.delay}s`,
                    }}
                />
            ))}
        </div>

        {/* — LEFT PANEL — */}
        <div className="split-left">
            {/* Brand */}
            <div className="split-brand">
                <div className="split-brand-icon">
                    <svg width="36" height="36" viewBox="0 0 32 32" fill="none">
                        <path d="M16 2L28 9v14L16 30 4 23V9L16 2z" fill="url(#hexL)" />

                        <path d="M16 8l6 3.5v7L16 22l-6-3.5v-7L16 8z"
                    fill="rgba(255,255,255,0.2)" />
                </div>
            </div>
        </div>
    </div>

```

```

        <circle cx="16" cy="16" r="3" fill="white" />
        <defs>
            <linearGradient id="hexL" x1="4" y1="2" x2="28" y2="30"
gradientUnits="userSpaceOnUse">
                <stop stopColor="#6ee7b7" />
                <stop offset="1" stopColor="#3b82f6" />
            </linearGradient>
        </defs>
    </svg>
</div>
<div>
    <h1 className="split-brand-name">DisasterEdu</h1>
    <p className="split-brand-tagline">From Awareness to
Action</p>
</div>
</div>

{/* Hero Text */}
<div className="split-hero">
    <h2 className="split-hero-title">
        From Awareness<br />
        to <span className="split-hero-accent">Action</span>
    </h2>
    <p className="split-hero-desc">
        Digital Disaster Education Platform preparing students,
teachers,
        and administrators through interactive modules, virtual
drills,
        and real-time analytics.
    </p>
</div>

{/* Feature Grid */}
<div className="split-features">
    {FEATURES.map((f, i) => (
        <div
            key={i}
            className="split-feature-card"
            style={{ animationDelay: `${0.1 + i * 0.1}s` }}
        >

```

```

        <div className="split-feature-icon" style={{ color: f.color,
background: f.color + "22" }}>
            {f.icon}
        </div>
        <div>
            <p className="split-feature-title">{f.title}</p>
            <p className="split-feature-desc">{f.desc}</p>
        </div>
    </div>
    )})
</div>
</div>

{/* — RIGHT PANEL (Form) — */}
<div className="split-right">
    <div className={`auth-card split-card ${shake ? "shake" : ""}
${successAnim ? "success-pulse" : ""}`}>
        {/* Header */}
        <div className="auth-header">
            <h2 className="brand-title" style={{ fontSize: 22 }}>Welcome
Back</h2>
            <p className="brand-sub">Choose your role and sign in to
continue</p>
        </div>

        {/* Role Toggle */}
        <div className="role-toggle" role="group" aria-label="Select
your role">
            <button
                className={`role-btn ${role === "student" ? "active" : ""}`}
                onClick={() => setRole("student")}
                type="button"
            >
                <svg width="15" height="15" viewBox="0 0 24 24"
fill="currentColor">
                    <path d="M12 3L1 9l11 6 9-4.91V17h2V9L12 3zM5 13.18v4L12
21l7-3.82v-4L12 17l-7-3.82z" />
                </svg>
                Student
            </button>

```

```

        <button
          className={`role-btn ${role === "admin" ? "active" : ""}`}
          onClick={() => setRole("admin")}
          type="button"
        >
          <svg width="15" height="15" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M12 1L3 5v6c0 5.55 3.84 10.74 9 12 5.16-1.26
9-6.45 9-12V5l-9-4z" />
          </svg>
          Teacher / Admin
        </button>
      </div>

      {/* General Error */}
      {errors.general && (
        <div className="alert-error" role="alert">
          <svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M12 2C6.48 2 2 6.48 2 12s4.48 10 10 10 10-4.48
10-10S17.52 2 12 2zm1 15h-2v-2h2v2zm0-4h-2V7h2v6z" />
          </svg>
          {errors.general}
        </div>
      )}

      {/* Email Field */}
      <div className={`field-group ${touched.email && errors.email ?
"has-error" : touched.email && email && !errors.email ? "has-success" :
""}`}>
        <label className="field-label" htmlFor="email">
          {role === "student" ? "Student ID / Email" : "Email
address"}
        </label>
        <div className="field-wrap">
          <svg className="field-icon" width="16" height="16"
viewBox="0 0 24 24" fill="currentColor">
            <path d="M20 4H4c-1.1 0-2 .9-2 2v12c0 1.1.9 2 2 2h16c1.1 0
2-.9 2-2V6c0-1.1-.9-2-2-2zm0 4l-8 5-8-5V6l8 5 8-5v2z" />
          </svg>

```



```

        <input
          id="email"
          type="email"
          placeholder={role === "student" ? "Enter your student ID"
: "you@school.edu"}
          value={email}
          autoComplete="email"
          onChange={(e) => {
            setEmail(e.target.value);
            if (touched.email) {
              const er = validate();
              setErrors((p) => ({ ...p, email: er.email }));
            }
          }}
          onBlur={() => handleBlur("email")}
          onKeyDown={handleKeyDown}
          disabled={loading}
          className="field-input"
        />
        {touched.email && !errors.email && email && (
          <svg className="field-icon-right success" width="15"
height="15" viewBox="0 0 24 24" fill="currentColor">
            <path d="M9 16.17L4.83 12l-1.42 1.41L9 19 21
7l-1.41-1.41L9 16.17z" />
          </svg>
        )}
      </div>
      {touched.email && errors.email && <p
className="field-error">{errors.email}</p>}
    </div>

    {/* Password Field */}
    <div className={`field-group ${touched.password &&
errors.password ? "has-error" : touched.password && password &&
!errors.password ? "has-success" : ""}`}>
      <label className="field-label"
htmlFor="password">Password</label>
      <div className="field-wrap">
        <svg className="field-icon" width="16" height="16"
viewBox="0 0 24 24" fill="currentColor">

```

```

        <path d="M18 8h-1V6c0-2.76-2.24-5-5-5S7 3.24 7 6v2H6c-1.1
0-2 .9-2 2v10c0 1.1.9 2 2 2h12c1.1 0 2-.9 2-2V10c0-1.1-.9-2-2-2zm-6 9c-1.1
0-2-.9-2-2s.9-2 2-2 2 .9 2 2-.9 2-2 2zm3.1-9H8.9V6c0-1.71 1.39-3.1 3.1-3.1
1.71 0 3.1 1.39 3.1 3.1v2z" />
    </svg>
    <input
      id="password"
      type={showPassword ? "text" : "password"}
      placeholder="Enter your password"
      value={password}
      autoComplete="current-password"
      onChange={(e) => {
        setPassword(e.target.value);
        if (touched.password) {
          const er = validate();
          setErrors((p) => ({ ...p, password: er.password }));
        }
      }}
      onBlur={() => handleBlur("password")}
      onKeyDown={handleKeyDown}
      disabled={loading}
      className="field-input"
    />
    <button
      type="button"
      className="toggle-pw"
      onClick={() => setShowPassword((v) => !v)}
      aria-label={showPassword ? "Hide password" : "Show
password"}
    >
      {showPassword ? (
        <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
          <path d="M12 7c2.76 0 5 2.24 5 5 0 .65-.13 1.26-.36
1.8312.92 2.92c1.51-1.26 2.7-2.89 3.43-4.75-1.73-4.39-6-7.5-11-7.5-1.4
0-2.74.25-3.98.712.16 2.16C10.74 7.13 11.35 7 12 7zM2 4.2712.28
2.28.46.46A11.8 11.8 0 001 12c1.73 4.39 6 7.5 11 7.5 1.55 0 3.03-.3
4.38-.841.42.42L19.73 22 21 20.73 3.27 3 2 4.27zM7.53 9.811.55
1.55c-.05.21-.08.43-.08.65 0 1.66 1.34 3 3 .22 0 .44-.03.65-.0811.55
1.55c-.67.33-1.41.53-2.2.53-2.76 0-5-2.24-5-5

```

```

0-.79.2-1.53.53-2.2zm4.31-.7813.15 3.15.02-.16c0-1.66-1.34-3-3-3l-.17.01z"
/>

    </svg>
  ) : (
    <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
      <path d="M12 4.5C7 4.5 2.73 7.61 1 12c1.73 4.39 6 7.5
11 7.5s9.27-3.11 11-7.5c-1.73-4.39-6-7.5-11-7.5zM12 17c-2.76
0-5-2.24-5-5s2.24-5 5-5 2.24 5 5-2.24 5-5 5zm0-8c-1.66 0-3 1.34-3 3s1.34
3 3 3 3-1.34 3-3-1.34-3-3-3z" />
    </svg>
  )}
</button>
</div>
{touched.password && errors.password && <p
className="field-error">{errors.password}</p>}
</div>

{/* Login Button */}
<button
  onClick={handleEmailLogin}
  disabled={loading}
  className={`btn-primary ${loading ? "loading" : ""}`}
  type="button"
>
  {loading ? <span className="spinner" /> : "Login"}
</button>

{/* Forgot + Divider + Google */}
<div className="forgot-row" style={{ marginTop: -8 }}>
  <Link to="/forgot-password" className="forgot-link">Forgot
your password?</Link>
</div>

<div className="divider"><span>or continue with</span></div>

<button
  onClick={handleGoogleLogin}
  disabled={loading}
  className="btn-google"

```

```

        type="button"
      >
        <svg width="18" height="18" viewBox="0 0 24 24">
          <path d="M22.56
12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21
3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z" fill="#4285F4" />
          <path d="M12 23c2.97 0 5.46-.98
7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.86
0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z" fill="#34A853"
/>

          <path d="M5.84
14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55 1
10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z" fill="#FBBC05" />
          <path d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45
2.09 14.97 1 12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53
6.16-4.53z" fill="#EA4335" />
        </svg>
        Sign in with Google
      </button>

      <p className="auth-footer-text">
        Don't have an account?{" "}
        <Link to="/signup" className="auth-link">Create one
here</Link>
      </p>
    </div>
  </div>
</div>
);
}

export default Login;

/* =====
Quiz.css - DisasterEdu AI Quiz Generator
Matches dark glassmorphism theme from Auth.css / Dashboard.css
===== */

@import
url('https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600;70
0&display=swap');

```

```
/* — Root wrapper — */
.qz-root {
  min-height: 100vh;
  background: #0a0f1e;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  padding: 32px 20px;
  font-family: 'Sora', sans-serif;
  position: relative;
  overflow: hidden;
}

/* Ambient glow */
.qz-root::before {
  content: '';
  position: fixed;
  inset: 0;
  background:
    radial-gradient(ellipse 70% 50% at 15% 25%, rgba(59,130,246,0.10) 0%,
transparent 60%),
    radial-gradient(ellipse 60% 40% at 85% 75%, rgba(110,231,183,0.08) 0%,
transparent 60%);
  pointer-events: none;
}

/* — Back button — */
.qz-back {
  position: fixed;
  top: 24px;
  left: 28px;
  display: flex;
  align-items: center;
  gap: 7px;
  color: rgba(255,255,255,0.38);
  font-size: 13px;
  font-weight: 500;
  cursor: pointer;
}
```

```

    transition: color 0.2s ease;
    z-index: 20;
}
.qz-back:hover { color: #6ee7b7; }

/* — Error — */
.qz-error {
    display: flex;
    align-items: center;
    gap: 8px;
    background: rgba(239,68,68,0.12);
    border: 1px solid rgba(239,68,68,0.25);
    border-radius: 10px;
    padding: 11px 14px;
    color: #fca5a5;
    font-size: 13px;
    margin-bottom: 20px;
}

/* ===== */
      SETUP CARD
/* ===== */

.qz-setup-card {
    position: relative;
    z-index: 10;
    width: 100%;
    max-width: 520px;
    background: rgba(255,255,255,0.04);
    border: 1px solid rgba(255,255,255,0.10);
    border-radius: 24px;
    padding: 40px;
    backdrop-filter: blur(24px);
    -webkit-backdrop-filter: blur(24px);
    box-shadow: 0 32px 64px rgba(0,0,0,0.5);
    animation: cardIn 0.5s cubic-bezier(0.22,1,0.36,1) both;
}

@keyframes cardIn {
    from { opacity: 0; transform: translateY(24px) scale(0.97); }
    to   { opacity: 1; transform: none; }
}

```

```
}

/* Setup Header */
.qz-setup-header {
  display: flex;
  align-items: center;
  gap: 16px;
  margin-bottom: 32px;
}

.qz-bolt-icon {
  width: 56px;
  height: 56px;
  border-radius: 16px;
  background: linear-gradient(135deg, rgba(110,231,183,0.2),
rgba(59,130,246,0.2));
  border: 1px solid rgba(110,231,183,0.25);
  display: flex;
  align-items: center;
  justify-content: center;
  color: #6ee7b7;
  flex-shrink: 0;
}

.qz-setup-title {
  font-size: 22px;
  font-weight: 700;
  color: #f1f5f9;
  margin: 0 0 4px;
  letter-spacing: -0.3px;
}

.qz-setup-sub {
  font-size: 13px;
  color: rgba(255,255,255,0.38);
  margin: 0;
}

/* Fields */
.qz-field {
```

```
margin-bottom: 24px;
}

.qz-label {
  display: block;
  font-size: 11.5px;
  font-weight: 600;
  color: rgba(255,255,255,0.45);
  text-transform: uppercase;
  letter-spacing: 0.6px;
  margin-bottom: 10px;
}

.qz-input {
  width: 100%;
  padding: 13px 16px;
  background: rgba(255,255,255,0.05);
  border: 1px solid rgba(255,255,255,0.10);
  border-radius: 12px;
  color: #f1f5f9;
  font-family: 'Sora', sans-serif;
  font-size: 15px;
  outline: none;
  box-sizing: border-box;
  transition: border-color 0.2s ease, box-shadow 0.2s ease;
}

.qz-input::placeholder { color: rgba(255,255,255,0.2); }
.qz-input:focus {
  border-color: rgba(110,231,183,0.45);
  box-shadow: 0 0 0 3px rgba(110,231,183,0.09);
}

/* Topic Pills */
.qz-topic-pills {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
}
```



```
.qz-pill {
  padding: 8px 14px;
  border-radius: 30px;
  border: 1px solid rgba(255,255,255,0.10);
  background: rgba(255,255,255,0.04);
  color: rgba(255,255,255,0.5);
  font-family: 'Sora', sans-serif;
  font-size: 13px;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.2s ease;
}

.qz-pill:hover {
  border-color: rgba(110,231,183,0.3);
  color: #6ee7b7;
}

.qz-pill.active {
  background: rgba(110,231,183,0.15);
  border-color: rgba(110,231,183,0.4);
  color: #6ee7b7;
}

/* Generate Button */
.qz-generate-btn {
  width: 100%;
  padding: 15px;
  background: linear-gradient(135deg, #6ee7b7 0%, #3b82f6 100%);
  border: none;
  border-radius: 12px;
  color: #0a0f1e;
  font-family: 'Sora', sans-serif;
  font-size: 15px;
  font-weight: 700;
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 9px;
}
```

```

    transition: opacity 0.2s ease, transform 0.15s ease, box-shadow 0.2s
ease;
    box-shadow: 0 6px 24px rgba(59,130,246,0.35);
    margin-top: 8px;
}

.qz-generate-btn:hover:not(:disabled) {
    opacity: 0.9;
    transform: translateY(-2px);
    box-shadow: 0 12px 32px rgba(59,130,246,0.45);
}

.qz-generate-btn:disabled { opacity: 0.6; cursor: not-allowed; }

/* Spinner */
.qz-spinner {
    display: inline-block;
    width: 17px;
    height: 17px;
    border: 2.5px solid rgba(10,15,30,0.3);
    border-top-color: #0a0f1e;
    border-radius: 50%;
    animation: spin 0.7s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }

/* =====
    QUIZ CARD
===== */

.qz-quiz-card {
    position: relative;
    z-index: 10;
    width: 100%;
    max-width: 600px;
    background: rgba(255,255,255,0.04);
    border: 1px solid rgba(255,255,255,0.10);
    border-radius: 24px;
    padding: 36px;
    backdrop-filter: blur(24px);
    -webkit-backdrop-filter: blur(24px);

```

```
    box-shadow: 0 32px 64px rgba(0,0,0,0.5);
    animation: cardIn 0.4s ease both;
}

/* Top bar */
.qz-top-bar {
    display: flex;
    align-items: center;
    justify-content: space-between;
    margin-bottom: 16px;
}

.qz-meta {
    display: flex;
    flex-direction: column;
    gap: 4px;
}

.qz-topic-tag {
    font-size: 11px;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.6px;
    color: #6ee7b7;
    background: rgba(110,231,183,0.12);
    border: 1px solid rgba(110,231,183,0.2);
    border-radius: 20px;
    padding: 3px 10px;
    width: fit-content;
}

.qz-qnum {
    font-size: 13px;
    color: rgba(255,255,255,0.38);
    font-weight: 500;
}

/* Timer ring */
.qz-timer-wrap {
    position: relative;
```

```
width: 52px;
height: 52px;
display: flex;
align-items: center;
justify-content: center;
}

.qz-timer-num {
  position: absolute;
  font-size: 14px;
  font-weight: 700;
  line-height: 1;
}

/* Progress track */
.qz-progress-track {
  height: 3px;
  background: rgba(255,255,255,0.07);
  border-radius: 3px;
  margin-bottom: 28px;
  overflow: hidden;
}

.qz-progress-fill {
  height: 100%;
  background: linear-gradient(90deg, #6ee7b7, #3b82f6);
  border-radius: 3px;
  transition: width 0.4s ease;
}

/* Question text */
.qz-question {
  font-size: 18px;
  font-weight: 600;
  color: #f1f5f9;
  line-height: 1.55;
  margin: 0 0 24px;
  letter-spacing: -0.2px;
}
```

```
/* Options */
.qz-options {
  display: flex;
  flex-direction: column;
  gap: 10px;
  margin-bottom: 24px;
}

.qz-option {
  display: flex;
  align-items: center;
  gap: 14px;
  padding: 14px 18px;
  border-radius: 12px;
  border: 1px solid rgba(255,255,255,0.09);
  background: rgba(255,255,255,0.04);
  color: #cbd5e1;
  font-family: 'Sora', sans-serif;
  font-size: 14.5px;
  cursor: pointer;
  text-align: left;
  transition: all 0.18s ease;
  width: 100%;
}

.qz-option:hover:not(.dim):not(.correct):not(.wrong) {
  border-color: rgba(110,231,183,0.3);
  background: rgba(110,231,183,0.07);
  color: #f1f5f9;
}

.qz-option.chosen {
  border-color: rgba(59,130,246,0.5);
  background: rgba(59,130,246,0.12);
  color: #93c5fd;
}

.qz-option.correct {
  border-color: rgba(16,185,129,0.5);
  background: rgba(16,185,129,0.12);
}
```

```
    color: #6ee7b7;
}

.qz-option.wrong {
    border-color: rgba(239,68,68,0.4);
    background: rgba(239,68,68,0.09);
    color: #f87171;
}

.qz-option.dim {
    opacity: 0.38;
    cursor: default;
}

.qz-opt-letter {
    width: 26px;
    height: 26px;
    border-radius: 7px;
    background: rgba(255,255,255,0.07);
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 12px;
    font-weight: 700;
    flex-shrink: 0;
}

.qz-opt-text { flex: 1; }

.qz-opt-check { margin-left: auto; flex-shrink: 0; }

/* Next button */
.qz-next-btn {
    width: 100%;
    padding: 13px;
    background: linear-gradient(135deg, #6ee7b7 0%, #3b82f6 100%);
    border: none;
    border-radius: 12px;
    color: #0a0f1e;
    font-family: 'Sora', sans-serif;
}
```

```

    font-size: 14.5px;
    font-weight: 700;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
    transition: opacity 0.2s ease, transform 0.15s ease;
    box-shadow: 0 4px 18px rgba(59,130,246,0.3);
}

.qz-next-btn:hover:not(:disabled) {
    opacity: 0.9;
    transform: translateY(-1px);
}

.qz-next-btn:disabled {
    opacity: 0.35;
    cursor: not-allowed;
    box-shadow: none;
}

/* Live score */
.qz-live-score {
    text-align: center;
    margin: 14px 0 0;
    font-size: 13px;
    color: rgba(255,255,255,0.32);
}

/* =====
   RESULT CARD
   ===== */

.qz-result-card {
    position: relative;
    z-index: 10;
    width: 100%;
    max-width: 600px;
    background: rgba(255,255,255,0.04);
    border: 1px solid rgba(255,255,255,0.10);

```

```

border-radius: 24px;
padding: 40px 36px;
backdrop-filter: blur(24px);
-webkit-backdrop-filter: blur(24px);
box-shadow: 0 32px 64px rgba(0,0,0,0.5);
animation: cardIn 0.5s ease both;
}

/* Score hero */
.qz-score-hero {
  text-align: center;
  margin-bottom: 36px;
}

.qz-score-ring {
  display: inline-flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  width: 130px;
  height: 130px;
  border-radius: 50%;
  border: 3px solid var(--ring-color, #6ee7b7);
  background: rgba(255,255,255,0.03);
  box-shadow: 0 0 32px color-mix(in srgb, var(--ring-color, #6ee7b7) 25%,
transparent);
  margin-bottom: 16px;
  gap: 2px;
}

.qz-score-emoji {
  font-size: 28px;
  line-height: 1;
}

.qz-score-num {
  font-size: 26px;
  font-weight: 800;
  line-height: 1;
  letter-spacing: -1px;
}

```



```
}

.qz-score-frac {
  font-size: 12px;
  color: rgba(255,255,255,0.4);
}

.qz-grade-label {
  font-size: 22px;
  font-weight: 700;
  margin: 0 0 6px;
  letter-spacing: -0.3px;
}

.qz-result-sub {
  font-size: 13px;
  color: rgba(255,255,255,0.38);
  margin: 0;
}

/* Review list */
.qz-review-list {
  display: flex;
  flex-direction: column;
  gap: 12px;
  margin-bottom: 28px;
  max-height: 380px;
  overflow-y: auto;
  padding-right: 4px;
}

.qz-review-title {
  font-size: 14px;
  font-weight: 600;
  color: rgba(255,255,255,0.45);
  text-transform: uppercase;
  letter-spacing: 0.6px;
  margin: 0 0 4px;
}
```

```
.qz-review-item {
  border-radius: 12px;
  border: 1px solid;
  padding: 14px 16px;
}

.qz-review-item.correct {
  border-color: rgba(16,185,129,0.25);
  background: rgba(16,185,129,0.05);
}

.qz-review-item.wrong {
  border-color: rgba(239,68,68,0.22);
  background: rgba(239,68,68,0.05);
}

.qz-review-q {
  display: flex;
  align-items: flex-start;
  gap: 10px;
  margin-bottom: 8px;
}

.qz-review-num {
  font-size: 11px;
  font-weight: 700;
  color: rgba(255,255,255,0.3);
  background: rgba(255,255,255,0.07);
  border-radius: 5px;
  padding: 2px 6px;
  flex-shrink: 0;
  margin-top: 2px;
}

.qz-review-question {
  font-size: 13.5px;
  color: #cbd5e1;
  margin: 0;
  line-height: 1.5;
}
```

```
.qz-review-ans {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  padding-left: 32px;
}

.qz-ans-label {
  font-size: 12.5px;
  font-weight: 500;
  border-radius: 6px;
  padding: 3px 10px;
}

.correct-label {
  background: rgba(16,185,129,0.12);
  color: #6ee7b7;
}

.wrong-label {
  background: rgba(239,68,68,0.1);
  color: #f87171;
}

/* Result action buttons */
.qz-result-actions {
  display: flex;
  gap: 12px;
}

.qz-retry-btn {
  flex: 1;
  padding: 13px;
  border: 1px solid rgba(110,231,183,0.3);
  border-radius: 12px;
  background: rgba(110,231,183,0.08);
  color: #6ee7b7;
  font-family: 'Sora', sans-serif;
  font-size: 14px;
}
```

```
font-weight: 600;
cursor: pointer;
display: flex;
align-items: center;
justify-content: center;
gap: 7px;
transition: all 0.2s ease;
}

.qz-retry-btn:hover {
  background: rgba(110,231,183,0.15);
  transform: translateY(-1px);
}

.qz-dash-btn {
  flex: 1;
  padding: 13px;
  background: linear-gradient(135deg, #6ee7b7 0%, #3b82f6 100%);
  border: none;
  border-radius: 12px;
  color: #0a0f1e;
  font-family: 'Sora', sans-serif;
  font-size: 14px;
  font-weight: 700;
  cursor: pointer;
  transition: opacity 0.2s ease, transform 0.15s ease;
  box-shadow: 0 4px 16px rgba(59,130,246,0.3);
}

.qz-dash-btn:hover {
  opacity: 0.9;
  transform: translateY(-1px);
}

/* Scrollbar */
.qz-review-list::-webkit-scrollbar { width: 4px; }
.qz-review-list::-webkit-scrollbar-track { background: transparent; }
.qz-review-list::-webkit-scrollbar-thumb { background:
rgba(255,255,255,0.1); border-radius: 4px; }
```

```
/* — Review options (shown in result screen) — */
```

```
.qz-review-options {  
  display: flex;  
  flex-direction: column;  
  gap: 6px;  
  margin-top: 10px;  
  padding-left: 32px;  
}
```

```
.qz-review-opt {  
  display: flex;  
  align-items: center;  
  gap: 10px;  
  padding: 9px 12px;  
  border-radius: 9px;  
  border: 1px solid transparent;  
  font-size: 13px;  
}
```

```
.qz-review-opt.rv-correct {  
  background: rgba(16,185,129,0.10);  
  border-color: rgba(16,185,129,0.3);  
  color: #6ee7b7;  
}
```

```
.qz-review-opt.rv-wrong {  
  background: rgba(239,68,68,0.08);  
  border-color: rgba(239,68,68,0.25);  
  color: #f87171;  
}
```

```
.qz-review-opt.rv-neutral {  
  background: rgba(255,255,255,0.03);  
  border-color: rgba(255,255,255,0.06);  
  color: rgba(255,255,255,0.38);  
}
```

```
.qz-rv-letter {  
  width: 22px;  
  height: 22px;
```

```

border-radius: 6px;
background: rgba(255,255,255,0.07);
display: flex;
align-items: center;
justify-content: center;
font-size: 11px;
font-weight: 700;
flex-shrink: 0;
}

.qz-rv-text { flex: 1; line-height: 1.4; }
import React, { useState, useEffect, useRef, useCallback } from "react";
import axios from "axios";
import { useNavigate } from "react-router-dom";
import "../Quiz.css";

const TOPIC_PRESETS = [
  { label: "🔥 Fire", value: "Fire" },
  { label: "🌊 Flood", value: "Flood" },
  { label: "🌋 Earthquake", value: "Earthquake" },
  { label: "🌀 Cyclone", value: "Cyclone" },
  { label: "⚡ Lightning", value: "Lightning" },
  { label: "🏥 First Aid", value: "First Aid" },
];

function QuizGenerator() {
  const navigate = useNavigate();
  const [classLevel, setClassLevel] = useState("");
  const [topic, setTopic] = useState("");
  const [quiz, setQuiz] = useState([]);
  const [current, setCurrent] = useState(0);
  const [selected, setSelected] = useState("");
  const [score, setScore] = useState(0);
  const [showResult, setShowResult] = useState(false);
  const [answers, setAnswers] = useState([]);
  const [timeLeft, setTimeLeft] = useState(60);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const timerRef = useRef(null);

```

```

// Use refs to always have fresh state inside the timer callback
const currentRef = useRef(current);
const selectedRef = useRef(selected);
const answersRef = useRef(answers);
const scoreRef = useRef(score);
const quizRef = useRef(quiz);

useEffect(() => { currentRef.current = current; }, [current]);
useEffect(() => { selectedRef.current = selected; }, [selected]);
useEffect(() => { answersRef.current = answers; }, [answers]);
useEffect(() => { scoreRef.current = score; }, [score]);
useEffect(() => { quizRef.current = quiz; }, [quiz]);

// — KEY FIX 1: Normalize answer for comparison —
// Your API might return answer as "A", "B", "C", "D" (letter)
// or as the full option text. This helper handles both.
const getCorrectOptionText = (q) => {
  const ans = q.answer;
  // If answer is a single letter A/B/C/D, map to option text
  if (/^[A-D]$/i.test(ans)) {
    const idx = ans.toUpperCase().charCodeAt(0) - 65;
    return q.options[idx] ?? ans;
  }
  // Otherwise answer is already the full text
  return ans;
};

// — KEY FIX 2: finishQuiz defined with useCallback so it's stable —
const finishQuiz = useCallback((finalAnswers, finalScore, topicVal,
classVal, totalLen) => {
  clearInterval(timerRef.current);
  setShowResult(true);
  const result = {
    score: finalScore,
    total: totalLen,
    date: new Date().toLocaleString(),
    topic: topicVal,
    classLevel: classVal,
  };
  localStorage.setItem("quizResult", JSON.stringify(result));

```

```

}, []);

// — KEY FIX 3: Auto-submit uses refs to avoid stale closure —
const handleAutoSubmit = useCallback(() => {
  clearInterval(timerRef.current);
  const q = quizRef.current[currentRef.current];
  if (!q) return;

  const correctText = getCorrectOptionText(q);
  const sel = selectedRef.current || ""; // no answer selected
  const isCorrect = sel === correctText;
  const newScore = isCorrect ? scoreRef.current + 1 : scoreRef.current;

  const newAnswers = [
    ...answersRef.current,
    {
      question: q.question,
      correct: correctText,
      selected: sel || "(no answer)",
      isCorrect,
      options: q.options,
    },
  ];

  setAnswers(newAnswers);
  if (isCorrect) setScore(newScore);
  finishQuiz(newAnswers, newScore, topic, classLevel,
quizRef.current.length);
}, [finishQuiz, topic, classLevel]);

// — KEY FIX 4: Timer effect re-runs on `current` change, properly
resets —
useEffect(() => {
  if (quiz.length === 0 || showResult) return;

  setTimeLeft(60); // reset timer for each new question
  clearInterval(timerRef.current);

  timerRef.current = setInterval(() => {
    setTimeLeft((t) => {

```



```

        if (t <= 1) {
            clearInterval(timerRef.current);
            handleAutoSubmit();
            return 0;
        }
        return t - 1;
    });
}, 1000);

return () => clearInterval(timerRef.current);
}, [quiz.length, current, showResult, handleAutoSubmit]);

const generateQuiz = async () => {
    if (!classLevel || !topic) {
        setError("Please enter both class level and topic.");
        return;
    }
    setError("");
    setLoading(true);
    try {
        const res = await
axios.post("http://localhost:5000/api/quiz/generate-quiz", {
            classLevel,
            topic,
        });
        setQuiz(res.data.questions);
        setCurrent(0);
        setScore(0);
        setShowResult(false);
        setTimeLeft(60);
        setAnswers([]);
        setSelected("");
    } catch (err) {
        setError("Failed to generate quiz. Please check the server.");
    } finally {
        setLoading(false);
    }
};

const handleOptionClick = (opt) => {

```

```

    if (selected) return;
    setSelected(opt);
  };

  const handleNext = () => {
    if (!selected) return;
    clearInterval(timerRef.current);

    const q = quiz[current];
    const correctText = getCorrectOptionText(q); // ← FIX: normalize
answer
    const isCorrect = selected === correctText;
    const newScore = isCorrect ? score + 1 : score;

    const newAnswers = [
      ...answers,
      {
        question: q.question,
        correct: correctText,    // ← always store the full option text
        selected,
        isCorrect,
        options: q.options,
      },
    ];

    if (isCorrect) setScore(newScore);
    setAnswers(newAnswers);
    setSelected("");

    if (current + 1 < quiz.length) {
      setCurrent(current + 1);
      // timer resets via useEffect dependency on `current`
    } else {
      finishQuiz(newAnswers, newScore, topic, classLevel, quiz.length);
    }
  };

  const resetQuiz = () => {
    clearInterval(timerRef.current);
    setQuiz([]);
  };

```

```

    setCurrent(0);
    setScore(0);
    setShowResult(false);
    setAnswers([]);
    setSelected("");
    setTimeLeft(60);
    setTopic("");
    setClassLevel("");
  };

  const timerPct = (timeLeft / 60) * 100;
  const timerColor = timeLeft > 30 ? "#6ee7b7" : timeLeft > 10 ? "#f59e0b"
: "#ef4444";

  // — SETUP SCREEN —
  if (quiz.length === 0 && !showResult) {
    return (
      <div className="qz-root">
        <div className="qz-back" onClick={() => navigate(-1)}>
          <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M20 11H7.83l5.59-5.59L12 4l-8 8 8 8 1.41-1.41L7.83
13H20v-2z" />
          </svg>
          Back to Dashboard
        </div>

        <div className="qz-setup-card">
          <div className="qz-setup-header">
            <div className="qz-bolt-icon">
              <svg width="28" height="28" viewBox="0 0 24 24"
fill="currentColor">
                <path d="M7 2v11h3v9l7-12h-4l4-8z" />
              </svg>
            </div>
            <div>
              <h1 className="qz-setup-title">AI Quiz Generator</h1>
              <p className="qz-setup-sub">Generate disaster-preparedness
quizzes instantly</p>
            </div>
          </div>
        </div>
      </div>
    );
  }

```

```

    </div>

    {error && (
      <div className="qz-error">
        <svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor">
          <path d="M12 2C6.48 2 2 6.48 2 12s4.48 10 10 10 10-4.48
10-10s17.52 2 12 2zm1 15h-2v-2h2v2zm0-4h-2V7h2v6z" />
        </svg>
        {error}
      </div>
    )}

    <div className="qz-field">
      <label className="qz-label">Class / Grade Level</label>
      <input
        className="qz-input"
        type="number"
        min="1"
        max="12"
        placeholder="e.g. 8"
        value={classLevel}
        onChange={(e) => setClassLevel(e.target.value)}
      />
    </div>

    <div className="qz-field">
      <label className="qz-label">Disaster Topic</label>
      <div className="qz-topic-pills">
        {TOPIC_PRESETS.map((t) => (
          <button
            key={t.value}
            className={`qz-pill ${topic === t.value ? "active" :
""}`}
            onClick={() => setTopic(t.value)}
            type="button"
          >
            {t.label}
          </button>
        ))}
      </div>
    </div>

```

```

        </div>
        <input
          className="qz-input"
          style={{ marginTop: 10 }}
          placeholder="Or type a custom topic..."
          value={topic}
          onChange={(e) => setTopic(e.target.value)}
        />
      </div>

      <button className="qz-generate-btn" onClick={generateQuiz}
disabled={loading}>
        {loading ? (
          <><span className="qz-spinner" />Generating Quiz...</>
        ) : (
          <>
            <svg width="18" height="18" viewBox="0 0 24 24"
fill="currentColor">
              <path d="M7 2v11h3v9l7-12h-4l4-8z" />
            </svg>
            Generate AI Quiz
          </>
        )}
      </button>
    </div>
  </div>
);
}

// — QUIZ SCREEN —
if (quiz.length > 0 && !showResult) {
  const q = quiz[current];
  const progress = (current / quiz.length) * 100;

  return (
    <div className="qz-root">
      <div className="qz-quiz-card">
        <div className="qz-top-bar">
          <div className="qz-meta">
            <span className="qz-topic-tag">{topic}</span>

```

```

        <span className="qz-qnum">Question {current + 1} /
{quiz.length}</span>
    </div>
    <div className="qz-timer-wrap">
        <svg width="52" height="52" viewBox="0 0 52 52">
            <circle cx="26" cy="26" r="22" fill="none"
stroke="rgba(255,255,255,0.07)" strokeWidth="4" />
            <circle
                cx="26" cy="26" r="22"
                fill="none"
                stroke={timerColor}
                strokeWidth="4"
                strokeDasharray={`$ {2 * Math.PI * 22}`}
                strokeDashoffset={`$ {2 * Math.PI * 22 * (1 - timerPct /
100)}`}

                strokeLinecap="round"
                style={{ transition: "stroke-dashoffset 1s linear,
stroke 0.3s ease", transform: "rotate(-90deg)", transformOrigin: "center"
}}
            />
        </svg>
        <span className="qz-timer-num" style={{ color: timerColor
}}>{timeLeft}</span>
    </div>
</div>

<div className="qz-progress-track">
    <div className="qz-progress-fill" style={{ width:
`${progress}%` }} />
</div>

<h2 className="qz-question">{q.question}</h2>

<div className="qz-options">
    {q.options.map((opt, i) => {
        let cls = "qz-option";
        if (selected === opt) cls += " chosen";
        else if (selected && selected !== opt) cls += " dim";
        return (

```

```

        <button key={i} className={cls} onClick={() =>
handleOptionClick(opt)} type="button">
            <span className="qz-opt-letter">{String.fromCharCode(65
+ i)}</span>
            <span className="qz-opt-text">{opt}</span>
        </button>
    );
    }}}
</div>

    <button className="qz-next-btn" onClick={handleNext}
disabled={!selected} type="button">
        {current + 1 === quiz.length ? "Finish Quiz" : "Next
Question"}
        <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M8 5v14l11-7z" />
        </svg>
    </button>

    <p className="qz-live-score">
        Score so far: <strong style={{ color: "#6ee7b7"
}}>{score}</strong> / {current}
    </p>
</div>
</div>
);
}

// — RESULT SCREEN —
if (showResult) {
    const pct = Math.round((score / quiz.length) * 100);
    const grade = pct >= 85 ? { label: "Excellent!", color: "#10b981",
emoji: "🏆" }
        : pct >= 65 ? { label: "Good Job!", color: "#3b82f6", emoji: "👍" }
        : pct >= 40 ? { label: "Keep Practicing", color: "#f59e0b", emoji:
"📚" }
        : { label: "Needs Work", color: "#ef4444", emoji: "💪" };

    return (

```

```

<div className="qz-root">
  <div className="qz-result-card">
    <div className="qz-score-hero">
      <div className="qz-score-ring" style={{ "--ring-color":
grade.color }}>
        <span className="qz-score-emoji">{grade.emoji}</span>
        <span className="qz-score-num" style={{ color: grade.color
}}>{pct}%</span>
        <span className="qz-score-frac">{score}/{quiz.length}</span>
      </div>
      <h2 className="qz-grade-label" style={{ color: grade.color
}}>{grade.label}</h2>
      <p className="qz-result-sub">Topic: <strong>{topic}</strong>
Class {classLevel}</p>
    </div>

    <div className="qz-review-list">
      <h3 className="qz-review-title">Answer Review</h3>
      {answers.map((a, i) => (
        <div key={i} className={`qz-review-item ${a.isCorrect ?
"correct" : "wrong"}`}>
          <div className="qz-review-q">
            <span className="qz-review-num">Q{i + 1}</span>
            <p className="qz-review-question">{a.question}</p>
          </div>
          <div className="qz-review-options">
            {(a.options || []).map((opt, oi) => {
              const isCorrectOpt = opt === a.correct;
              const isSelectedOpt = opt === a.selected;
              let optCls = "qz-review-opt";
              if (isCorrectOpt) optCls += " rv-correct";
              else if (isSelectedOpt && !a.isCorrect) optCls += "
rv-wrong";
              else optCls += " rv-neutral";
              return (
                <div key={oi} className={optCls}>
                  <span
className="qz-rv-letter">{String.fromCharCode(65 + oi)}</span>
                  <span className="qz-rv-text">{opt}</span>
                  {isCorrectOpt && (

```



```

        <svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor" style={{ marginLeft: "auto", color: "#10b981",
flexShrink: 0 }}>
            <path d="M9 16.17L4.83 12l-1.42 1.41L9 19 21
7l-1.41-1.41L9 16.17z" />
        </svg>
    ))
    {isSelectedOpt && !a.isCorrect && !isCorrectOpt &&
(
        <svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor" style={{ marginLeft: "auto", color: "#ef4444",
flexShrink: 0 }}>
            <path d="M19 6.41L17.59 5 12 10.59 6.41 5 5
6.41 10.59 12 5 17.59 6.41 19 12 13.41 17.59 19 19 17.59 13.41 12z" />
        </svg>
    ))
</div>
);
}))
</div>
<div className="qz-review-ans" style={{ marginTop: 8 }}>
    {a.isCorrect
        ? <span className="qz-ans-label correct-label">✔
Correct!</span>
        : (
            <>
                <span className="qz-ans-label wrong-label">✘ Your
answer: {a.selected}</span>
                <span className="qz-ans-label correct-label">✔
Correct: {a.correct}</span>
            </>
        )
    }
</div>
</div>
))}
</div>

<div className="qz-result-actions">

```

```

        <button className="qz-retry-btn" onClick={resetQuiz}
type="button">
            <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
                <path d="M17.65 6.35C16.2 4.9 14.21 4 12 4c-4.42 0-7.99
3.58-7.99 8s3.57 8 7.99 8c3.73 0 6.84-2.55 7.73-6h-2.08c-.82 2.33-3.04
4-5.65 4-3.31 0-6-2.69-6-6s2.69-6 6-6c1.66 0 3.14.69 4.22 1.78L13
11h7V4l-2.35 2.35z" />
            </svg>
            Try Again
        </button>
        <button className="qz-dash-btn" onClick={() => navigate(-1)}
type="button">
            Back to Dashboard
        </button>
    </div>
</div>
</div>
);
}

return null;
}

export default QuizGenerator;
import React, { useState } from "react";
import { auth } from "../../firebase";
import { createUserWithEmailAndPassword } from "firebase/auth";
import { useNavigate, Link } from "react-router-dom";
import "../../Auth.css";
import { db } from "../../firebase";
import { doc, setDoc } from "firebase/firestore";
import axios from "axios"; // ← ADDED

const BASE = "http://localhost:5000";

const FEATURES = [
    {
        icon: (
            <svg width="22" height="22" viewBox="0 0 24 24" fill="currentColor">

```

```

        <path d="M21 5c-1.11-.35-2.33-.5-3.5-.5-1.95 0-4.05.4-5.5
1.5-1.45-1.1-3.55-1.5-5.5-1.5S2.45 4.9 1 6v14.65c0 .25.25.5.5.5.1 0
.15-.05.25-.05C3.1 20.45 5.05 20 6.5 20c1.95 0 4.05.4 5.5 1.5 1.35-.85
3.8-1.5 5.5-1.5 1.65 0 3.35.3 4.75 1.05.1.05.15.05.25.05.25 0
.5-.25.5-.5V6c-.6-.45-1.25-.75-2-1z" />
    </svg>
  ),
  color: "#f59e0b",
  title: "Interactive Learning",
  desc: "Modules & Quizzes",
},
{
  icon: (
    <svg width="22" height="22" viewBox="0 0 24 24" fill="currentColor">
      <path d="M12 1L3 5v6c0 5.55 3.84 10.74 9 12 5.16-1.26 9-6.45
9-12V5l-9-4z" />
    </svg>
  ),
  color: "#6ee7b7",
  title: "Virtual Drills",
  desc: "Practice Scenarios",
},
{
  icon: (
    <svg width="22" height="22" viewBox="0 0 24 24" fill="currentColor">
      <path d="M19 3H5c-1.1 0-2 .9-2 2v14c0 1.1.9 2 2 2h14c1.1 0 2-.9
2-2V5c0-1.1-.9-2-2-2z" />
    </svg>
  ),
  color: "#3b82f6",
  title: "Real-time Analytics",
  desc: "Track Progress",
},
{
  icon: (
    <svg width="22" height="22" viewBox="0 0 24 24" fill="currentColor">
      <path d="M12 22c1.1 0 2-.9 2-2h-4c0 1.1.9 2 2
2zm6-6v-5c0-3.07-1.64-5.64-4.5-6.32V4c0-.83-.67-1.5-1.5-1.5s-1.5-.67-1.5
1.5v.68C7.63 5.36 6 7.92 6 11v5l-2 2v1h16v-11-2-2z" />
    </svg>
  ),

```

```

    ),
    color: "#f87171",
    title: "SOS Alerts",
    desc: "Emergency Response",
  },
];

function Signup() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [role, setRole] = useState("student");
  const [loading, setLoading] = useState(false);
  const [showPassword, setShowPassword] = useState(false);
  const [showConfirm, setShowConfirm] = useState(false);
  const [errors, setErrors] = useState({});
  const [touched, setTouched] = useState({});
  const [shake, setShake] = useState(false);
  const [success, setSuccess] = useState(false);

  const navigate = useNavigate();

  const getPasswordStrength = (pw) => {
    if (!pw) return { score: 0, label: "", color: "" };
    let score = 0;
    if (pw.length >= 8) score++;
    if (/[A-Z]/.test(pw)) score++;
    if (/[0-9]/.test(pw)) score++;
    if (/^[A-Za-z0-9]/.test(pw)) score++;
    const levels = [
      { score: 1, label: "Weak", color: "#ef4444" },
      { score: 2, label: "Fair", color: "#f59e0b" },
      { score: 3, label: "Good", color: "#3b82f6" },
      { score: 4, label: "Strong", color: "#10b981" },
    ];
    return levels[score - 1] || { score: 0, label: "", color: "" };
  };

  const validate = () => {
    const e = {};

```

```

    if (!email) e.email = "Email is required";
    else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(email)) e.email = "Invalid
email format";
    if (!password) e.password = "Password is required";
    else if (password.length < 6) e.password = "Minimum 6 characters";
    if (!confirmPassword) e.confirmPassword = "Please confirm your
password";
    else if (password !== confirmPassword) e.confirmPassword = "Passwords
do not match";
    return e;
  };

const handleBlur = (field) => {
  setTouched((p) => ({ ...p, [field]: true }));
  const e = validate();
  setErrors((p) => ({ ...p, [field]: e[field] }));
};

const triggerShake = () => {
  setShake(true);
  setTimeout(() => setShake(false), 600);
};

// — ONLY THIS FUNCTION WAS CHANGED —————
const handleSignup = async () => {
  setTouched({ email: true, password: true, confirmPassword: true });
  const e = validate();
  setErrors(e);
  if (Object.keys(e).length > 0) { triggerShake(); return; }
  setLoading(true);
  try {
    // 1. Firebase signup
    const userCredential = await createUserWithEmailAndPassword(auth,
email, password);
    const user = userCredential.user;

    // 2. Save role in Firestore (unchanged from your original)
    await setDoc(doc(db, "users", user.uid), {
      email:    user.email,
      role:     role,

```

```

        createdAt: new Date().toISOString(),
    });

    // 3.  NEW: Also save to MySQL so admin dashboard shows real
students
    await axios.post(`${BASE}/api/auth/register`, {
        firebase_uid: user.uid,
        email: user.email,
        display_name: user.email.split("@")[0],
        role: role,
    });

    setSuccess(true);
    setTimeout(() => navigate(role === "admin" ? "/admin" : "/student"),
1200);

    } catch (error) {
        const msgs = {
            "auth/email-already-in-use": "An account with this email already
exists",
            "auth/weak-password": "Password is too weak",
            "auth/invalid-email": "Invalid email address",
        };
        setErrors({ general: msgs[error.code] || "Signup failed. Please try
again." });
        triggerShake();
    } finally {
        setLoading(false);
    }
};

// _____

const handleKeyDown = (e) => {
    if (e.key === "Enter") handleSignup();
};

const strength = getPasswordStrength(password);

return (
    <div className="auth-bg split-layout">

```

```

    { /* Particles */ }
    <div className="particles" aria-hidden="true">
      {Array.from({ length: 16 }, (_, i) => (
        <span
          key={i}
          className="particle"
          style={{
            left:          `${Math.random() * 100}%`,
            top:           `${Math.random() * 100}%`,
            width:         2 + Math.random() * 4,
            height:        2 + Math.random() * 4,
            animationDuration: `${6 + Math.random() * 10}s`,
            animationDelay:  `${Math.random() * 5}s`,
          }}
        />
      ))}
    </div>

    { /* — LEFT PANEL — */ }
    <div className="split-left">
      <div className="split-brand">
        <div className="split-brand-icon">
          <svg width="36" height="36" viewBox="0 0 32 32" fill="none">
            <path d="M16 2L28 9v14L16 30 4 23V9L16 2z" fill="url(#hexS)"
            />

            <path d="M16 8L16 3.5v7L16 22l-6-3.5v-7L16 8z"
            fill="rgba(255,255,255,0.2)" />
            <circle cx="16" cy="16" r="3" fill="white" />
            <defs>
              <linearGradient id="hexS" x1="4" y1="2" x2="28" y2="30"
              gradientUnits="userSpaceOnUse">
                <stop stopColor="#6ee7b7" />
                <stop offset="1" stopColor="#3b82f6" />
              </linearGradient>
            </defs>
          </svg>
        </div>
        <div>
          <h1 className="split-brand-name">DisasterEdu</h1>

```

```

        <p className="split-brand-tagline">From Awareness to
Action</p>
    </div>
</div>

<div className="split-hero">
    <h2 className="split-hero-title">
        From Awareness<br />
        to <span className="split-hero-accent">Action</span>
    </h2>
    <p className="split-hero-desc">
        Join thousands of students and educators building resilience
        through interactive modules, virtual drills, and real-time
        disaster preparedness analytics.
    </p>
</div>

<div className="split-features">
    {FEATURES.map((f, i) => (
        <div
            key={i}
            className="split-feature-card"
            style={{ animationDelay: `${0.1 + i * 0.1}s` }}
        >
            <div className="split-feature-icon" style={{ color: f.color,
background: f.color + "22" }}>
                {f.icon}
            </div>
            <div>
                <p className="split-feature-title">{f.title}</p>
                <p className="split-feature-desc">{f.desc}</p>
            </div>
        </div>
    ))}
</div>
</div>

{/* — RIGHT PANEL (Form) — */}
<div className="split-right">

```



```

    <div className={`auth-card split-card ${shake ? "shake" : ""}
    ${success ? "success-pulse" : ""}`}>
      <div className="auth-header">
        <h2 className="brand-title" style={{ fontSize: 22 }}>Create
Account</h2>
        <p className="brand-sub">Join DisasterEdu - it's free</p>
      </div>

      {/* Role Toggle */}
      <div className="role-toggle" role="group" aria-label="Select
your role">
        <button
          className={`role-btn ${role === "student" ? "active" : ""}`}
          onClick={() => setRole("student")}
          type="button"
        >
          <svg width="15" height="15" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M12 3L1 9l11 6 9-4.91V17h2V9L12 3zM5 13.18v4L12
21l7-3.82v-4L12 17l-7-3.82z" />
          </svg>
          Student
        </button>
        <button
          className={`role-btn ${role === "admin" ? "active" : ""}`}
          onClick={() => setRole("admin")}
          type="button"
        >
          <svg width="15" height="15" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M12 1L3 5v6c0 5.55 3.84 10.74 9 12 5.16-1.26
9-6.45 9-12V5l-9-4z" />
          </svg>
          Teacher / Admin
        </button>
      </div>

      {errors.general && (
        <div className="alert-error" role="alert">

```

```

        <svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor">
        <path d="M12 2C6.48 2 2 6.48 2 12s4.48 10 10 10 10-4.48
10-10S17.52 2 12 2zm1 15h-2v-2h2v2zm0-4h-2V7h2v6z" />
        </svg>
        {errors.general}
      </div>
    )}

    {success && (
      <div className="alert-success" role="alert">
        <svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor">
        <path d="M9 16.17L4.83 12l-1.42 1.41L9 19 21
7l-1.41-1.41L9 16.17z" />
        </svg>
        Account created! Redirecting...
      </div>
    )}

    {/* Email */}
    <div className={`field-group ${touched.email && errors.email ?
"has-error" : touched.email && email && !errors.email ? "has-success" :
""}`}>
      <label className="field-label" htmlFor="su-email">Email
address</label>
      <div className="field-wrap">
        <svg className="field-icon" width="16" height="16"
viewBox="0 0 24 24" fill="currentColor">
        <path d="M20 4H4c-1.1 0-2 .9-2 2v12c0 1.1.9 2 2 2h16c1.1 0
2-.9 2-2V6c0-1.1-.9-2-2-2zm0 4l-8 5-8-5V6l8 5 8-5v2z" />
        </svg>
        <input
          id="su-email"
          type="email"
          placeholder="you@example.com"
          value={email}
          autoComplete="email"
          onChange={(e) => {
            setEmail(e.target.value);

```

```

        if (touched.email) {
            const er = validate();
            setErrors((p) => ({ ...p, email: er.email }));
        }
    }}
    onBlur={() => handleBlur("email")}
    onKeyDown={handleKeyDown}
    disabled={loading}
    className="field-input"
  />
</div>
{touched.email && errors.email && <p
className="field-error">{errors.email}</p>}
</div>

{/* Password */}
<div className={`field-group ${touched.password &&
errors.password ? "has-error" : touched.password && password &&
!errors.password ? "has-success" : ""}`}>
  <label className="field-label"
htmlFor="su-password">Password</label>
  <div className="field-wrap">
    <svg className="field-icon" width="16" height="16"
viewBox="0 0 24 24" fill="currentColor">
      <path d="M18 8h-1v6c0-2.76-2.24-5-5-5s-7 3.24 7 6v2H6c-1.1
0-2 .9-2 2v10c0 1.1.9 2 2 2h12c1.1 0 2-.9 2-2V10c0-1.1-.9-2-2-2zm-6 9c-1.1
0-2-.9-2-2s.9-2 2-2 2 .9 2 2-.9 2-2 2m3.1-9H8.9V6c0-1.71 1.39-3.1 3.1-3.1
1.71 0 3.1 1.39 3.1 3.1v2z" />
    </svg>
    <input
      id="su-password"
      type={showPassword ? "text" : "password"}
      placeholder="Min. 6 characters"
      value={password}
      autoComplete="new-password"
      onChange={(e) => {
        setPassword(e.target.value);
        if (touched.password) {
            const er = validate();
            setErrors((p) => ({ ...p, password: er.password }));

```

```

    }
  }}
  onBlur={() => handleBlur("password")}
  onKeyDown={handleKeyDown}
  disabled={loading}
  className="field-input"
/>
<button type="button" className="toggle-pw" onClick={() =>
setShowPassword((v) => !v)}>
  {showPassword ? (
    <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
      <path d="M12 7c2.76 0 5 2.24 5 5 0 .65-.13 1.26-.36
1.8312.92 2.92c1.51-1.26 2.7-2.89 3.43-4.75-1.73-4.39-6-7.5-11-7.5-1.4
0-2.74.25-3.98.712.16 2.16C10.74 7.13 11.35 7 12 7zM2 4.2712.28
2.28.46.46A11.8 11.8 0 001 12c1.73 4.39 6 7.5 11 7.5 1.55 0 3.03-.3
4.38-.841.42.42L19.73 22 21 20.73 3.27 3 2 4.27z" />
    </svg>
  ) : (
    <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
      <path d="M12 4.5C7 4.5 2.73 7.61 1 12c1.73 4.39 6 7.5
11 7.5s9.27-3.11 11-7.5c-1.73-4.39-6-7.5-11-7.5zM12 17c-2.76
0-5-2.24-5-5s2.24-5 5-5 5-2.24 5-5zM0-8c-1.66 0-3 1.34-3 3s1.34
3 3 3 3-1.34 3-3-1.34-3-3-3z" />
    </svg>
  )}
</button>
</div>
{touched.password && errors.password && <p
className="field-error">{errors.password}</p>}
{password && (
  <div className="strength-meter">
    <div className="strength-bars">
      {[1, 2, 3, 4].map((level) => (
        <div
          key={level}
          className="strength-bar"
          style={{ background: strength.score >= level ?
strength.color : "rgba(255,255,255,0.1)" }}

```

```

        />
    )))
</div>
    {strength.label && (
        <span className="strength-label" style={{ color:
strength.color }}>{strength.label}</span>
    )}
</div>
    )}
</div>

    { /* Confirm Password */}
    <div className={`field-group ${touched.confirmPassword &&
errors.confirmPassword ? "has-error" : touched.confirmPassword &&
confirmPassword && !errors.confirmPassword ? "has-success" : ""}`}>
        <label className="field-label" htmlFor="su-confirm">Confirm
password</label>
        <div className="field-wrap">
            <svg className="field-icon" width="16" height="16"
viewBox="0 0 24 24" fill="currentColor">
                <path d="M12 1L3 5v6c0 5.55 3.84 10.74 9 12 5.16-1.26
9-6.45 9-12V5l-9-4z" />
            </svg>
            <input
                id="su-confirm"
                type={showConfirm ? "text" : "password"}
                placeholder="Re-enter password"
                value={confirmPassword}
                autoComplete="new-password"
                onChange={(e) => {
                    setConfirmPassword(e.target.value);
                    if (touched.confirmPassword) {
                        const er = validate();
                        setErrors((p) => ({ ...p, confirmPassword:
er.confirmPassword }));
                    }
                }}
                onBlur={() => handleBlur("confirmPassword")}
                onKeyDown={handleKeyDown}
                disabled={loading}

```

```

        className="field-input"
      />
      <button type="button" className="toggle-pw" onClick={() =>
setShowConfirm((v) => !v)}>
        {showConfirm ? (
          <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M12 7c2.76 0 5 2.24 5 5 0 .65-.13 1.26-.36
1.8312.92 2.92c1.51-1.26 2.7-2.89 3.43-4.75-1.73-4.39-6-7.5-11-7.5-1.4
0-2.74.25-3.98.712.16 2.16C10.74 7.13 11.35 7 12 7zM2 4.2712.28
2.28.46.46A11.8 11.8 0 001 12c1.73 4.39 6 7.5 11 7.5 1.55 0 3.03-.3
4.38-.841.42.42L19.73 22 21 20.73 3.27 3 2 4.27z" />
          </svg>
        ) : (
          <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M12 4.5C7 4.5 2.73 7.61 1 12c1.73 4.39 6 7.5
11 7.5s9.27-3.11 11-7.5c-1.73-4.39-6-7.5-11-7.5zM12 17c-2.76
0-5-2.24-5-5s2.24-5 5-5 2.24 5 5-2.24 5-5 5zM0-8c-1.66 0-3 1.34-3 3s1.34
3 3 3 3-1.34 3-3-1.34-3-3-3z" />
          </svg>
        )}
      </button>
    </div>
    {touched.confirmPassword && errors.confirmPassword && (
      <p className="field-error">{errors.confirmPassword}</p>
    )}
  </div>

  <button
    onClick={handleSignup}
    disabled={loading || success}
    className={`btn-primary ${loading ? "loading" : ""}`}
    type="button"
  >
    {loading ? <span className="spinner" /> : success ? "Account
Created!" : "Create Account"}
  </button>

  <p className="auth-footer-text" style={{ marginTop: 16 }}>

```

```

        Already have an account?{" "}
        <Link to="/login" className="auth-link">Sign in here</Link>
      </p>
    </div>
  </div>
</div>
);
}

export default Signup;

// src/components/pages/StudentAssignments.jsx
// Add this component to your student dashboard / route
import React, { useState, useEffect, useCallback } from "react";
import axios from "axios";
import { auth } from "../../firebase";

const BASE = "http://localhost:5000";

export default function StudentAssignments() {
  const [assignments, setAssignments] = useState([]);
  const [loading, setLoading] = useState(true);
  const [userId, setUserId] = useState(null);
  const [updating, setUpdating] = useState(null); // assignment id
  // being updated

  // Get current user's DB id
  useEffect(() => {
    const unsub = auth.onAuthStateChanged(async (firebaseUser) => {
      if (!firebaseUser) return;
      try {
        // Adjust this endpoint to match how you look up DB user by
        // firebase uid
        const res = await axios.get(`${BASE}/api/users/me`, {
          headers: { Authorization: `Bearer ${await
firebaseUser.getIdToken()}` }
        });
        setUserId(res.data.id);
      } catch (e) {

```

```

        console.error("Could not get user id:", e.message);
    }
});
return unsub;
}, []);

const fetchMyAssignments = useCallback(async () => {
    if (!userId) return;
    setLoading(true);
    try {
        const res = await
axios.get(`${BASE}/api/assignments/student/${userId}`);
        setAssignments(res.data);
    } catch (e) {
        console.error("fetchMyAssignments:", e.message);
    } finally {
        setLoading(false);
    }
}, [userId]);

useEffect(() => { fetchMyAssignments(); }, [fetchMyAssignments]);

const updateProgress = async (assignmentId, progress, status) => {
    setUpdating(assignmentId);
    try {
        await axios.patch(
            `${BASE}/api/assignments/${assignmentId}/student/${userId}`,
            { progress, status }
        );
        await fetchMyAssignments();
    } catch (e) {
        alert("Update failed: " + e.message);
    } finally {
        setUpdating(null);
    }
};

const markDone = (assignmentId) => updateProgress(assignmentId, 100,
"completed");

```



```

    const startTask = (assignmentId) => updateProgress(assignmentId, 10,
    "in_progress");

    const statusStyle = (status) => {
      if (status === "completed") return { bg:"#10b98122",
color:"#10b981", label:"✅ Completed" };
      if (status === "in_progress") return { bg:"#3b82f622",
color:"#3b82f6", label:"⌚ In Progress" };
      return { bg:"#f59e0b22", color:"#f59e0b", label:"🔴 Not Started" };
    };

    if (loading) return (
      <div style={{textAlign:"center",padding:40,color:"#64748b"}}>Loading
your assignments...</div>
    );

    return (
      <div style={{padding:24}}>
        <h2
style={{color:"#f1f5f9",fontWeight:700,fontSize:22,marginBottom:20}}>
          📋 My Assignments ({assignments.length})
        </h2>

        {assignments.length === 0 ? (
          <div style={{textAlign:"center",padding:"60px
20px",color:"#64748b"}}>
            <div style={{fontSize:52,marginBottom:12}}>🚫</div>
            <p style={{fontSize:16}}>No assignments yet. Check back
later!</p>
          </div>
        ) : (
          <div style={{display:"flex",flexDirection:"column",gap:16}}>
            {assignments.map(asn => {
              const ss = statusStyle(asn.status);
              const isUpdating = updating === asn.assignment_id;
              return (
                <div key={asn.assignment_id} style={{
                  background:"#0f1117",border:"1px solid
#1e2535",borderRadius:14,padding:20
                }}>

```

```

        { /* Title + status */ }
        <div
style={{display:"flex",justifyContent:"space-between",alignItems:"flex-start",marginBottom:12}}>
            <div>
                <h3
style={{color:"#f1f5f9",fontWeight:700,fontSize:16,margin:0}}>{asn.title}<
/h3>
                {asn.description && (
                    <p style={{color:"#64748b",fontSize:13,margin:"5px 0 0 0"}}>{asn.description}</p>
                ) }
                {asn.due_date && (
                    <p style={{color:"#f59e0b",fontSize:12,margin:"4px 0 0 0"}}>
                        17 Due: {new
Date(asn.due_date).toLocaleDateString()}
                    </p>
                ) }
            </div>
            <span
style={{background:ss.bg,color:ss.color,fontSize:12,fontWeight:700,
padding:"4px 10px",borderRadius:12,flexShrink:0,marginLeft:10}}>
                {ss.label}
            </span>
        </div>

        { /* Attachments */ }
        {asn.file_name && (
            <a href={` ${BASE} ${asn.file_url} `} target="_blank"
rel="noreferrer"
style={{
display:"inline-flex",alignItems:"center",gap:6,
background:"#1e2535",border:"1px solid
#2d3748",borderRadius:8,
padding:"7px 14px",color:"#3b82f6",fontSize:13,textDecoration:"none",
marginBottom:12
}}>

```

```

        {asn.file_name.endsWith(".pdf") ? "📄" :
asn.file_name.match(/\.\pptx?$/i) ? "📊" : "✏️"}
        &nbsp;{asn.file_name}
        <svg width="12" height="12" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M19 19H5V5h7V3H5a2 2 0 00-2 2v14a2 2 0 002
2h14a2 2 0 002-2v-7h-2v7zM14 3v2h3.59L7.76 14.83l1.41 1.41L19
5.41V9h2V3h-7z"/>
        </svg>
    </a>
    )}

    {/* Progress bar */}
    <div style={{marginBottom:12}}>
        <div
style={{display:"flex",justifyContent:"space-between",marginBottom:4}}>
            <span style={{color:"#94a3b8",fontSize:12}}>Your
Progress</span>
            <span
style={{color:"#f1f5f9",fontSize:12,fontWeight:700}}>{asn.progress||0}%</s
pan>
        </div>
        <div
style={{background:"#1e2535",borderRadius:6,height:8}}>
            <div style={{
                width:`${asn.progress||0}%`,
                background:"linear-gradient(90deg,#3b82f6,#10b981)",
                borderRadius:6,height:8,transition:"width 0.4s"
            }}/>
        </div>
    </div>

    {/* Progress slider (student can drag) */}
    {asn.status !== "completed" && (
        <div style={{marginBottom:12}}>
            <label
style={{color:"#64748b",fontSize:12,display:"block",marginBottom:4}}>
                Update your progress:
            </label>
            <input type="range" min={0} max={100} step={5}

```

```

                defaultValue={asn.progress||0}
                onMouseUp={e => updateProgress(asn.assignment_id,
Number(e.target.value),
Number(e.target.value)===100?"completed":"in_progress")}
                onTouchEnd={e => updateProgress(asn.assignment_id,
Number(e.target.value),
Number(e.target.value)===100?"completed":"in_progress")}
                style={{width:"100%",accentColor:"#3b82f6"}}
            />
        </div>
    )}

    {/* Action buttons */}
    <div style={{display:"flex",gap:8}}>
        {asn.status === "assigned" && (
            <button onClick={()=>startTask(asn.assignment_id)}
disabled={isUpdating}
                style={{
flex:1,padding:"9px",background:"#3b82f622",border:"1px solid #3b82f644",
borderRadius:8,color:"#3b82f6",fontWeight:700,cursor:"pointer",fontSize:13
                }}>
                {isUpdating ? "..." : "▶ Start Task"}
            </button>
        )}
        {asn.status !== "completed" && (
            <button onClick={()=>markDone(asn.assignment_id)}
disabled={isUpdating}
                style={{
flex:2,padding:"9px",
background:"linear-gradient(135deg,#10b981,#059669)",
border:"none",borderRadius:8,color:"#fff",fontWeight:700,cursor:"pointer",
fontSize:13
                }}>
                {isUpdating ? "Saving..." : "✅ Mark as Completed"}
            </button>
        )}
    </div>

```

```

        {asn.status === "completed" && (
            <div
style={{flex:1,padding:"9px",background:"#10b98122",border:"1px solid
#10b98144",
            borderRadius:8,color:"#10b9/*
=====
    TsunamiModule.css — Tsunami Learning Slides
    Matches dark glassmorphism theme from Dashboard.css
    ===== */

@import
url('https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600;70
0&display=swap');

/* — Overlay — */
.tsm-overlay {
    position: fixed;
    inset: 0;
    background: rgba(0, 0, 0, 0.75);
    backdrop-filter: blur(8px);
    -webkit-backdrop-filter: blur(8px);
    z-index: 1000;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 20px;
    animation: fadeOverlay 0.3s ease;
}

@keyframes fadeOverlay {
    from { opacity: 0; }
    to   { opacity: 1; }
}

/* — Modal — */
.tsm-modal {
    width: 100%;
    max-width: 760px;
    max-height: 92vh;
    background: rgba(10, 15, 30, 0.97);

```

```

border: 1px solid rgba(255, 255, 255, 0.10);
border-radius: 28px;
display: flex;
flex-direction: column;
overflow: hidden;
box-shadow: 0 40px 80px rgba(0, 0, 0, 0.7);
animation: modalIn 0.4s cubic-bezier(0.22, 1, 0.36, 1);
font-family: 'Sora', sans-serif;
}

@keyframes modalIn {
  from { opacity: 0; transform: scale(0.92) translateY(20px); }
  to   { opacity: 1; transform: none; }
}

/* — Header — */
.tsm-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 18px 24px 14px;
  border-bottom: 1px solid rgba(255, 255, 255, 0.07);
  flex-shrink: 0;
}

.tsm-header-left {
  display: flex;
  align-items: center;
  gap: 12px;
}

.tsm-module-badge {
  font-size: 13px;
  font-weight: 600;
  color: #6ee7b7;
  background: rgba(110, 231, 183, 0.12);
  border: 1px solid rgba(110, 231, 183, 0.25);
  border-radius: 20px;
  padding: 4px 12px;
}

```

```
.tsm-slide-count {
  font-size: 13px;
  color: rgba(255, 255, 255, 0.35);
}

.tsm-header-right {
  display: flex;
  align-items: center;
  gap: 14px;
}

.tsm-progress-label {
  font-size: 12px;
  color: rgba(255, 255, 255, 0.35);
}

.tsm-close-btn {
  width: 30px;
  height: 30px;
  border-radius: 50%;
  border: 1px solid rgba(255, 255, 255, 0.15);
  background: transparent;
  color: rgba(255, 255, 255, 0.5);
  cursor: pointer;
  font-size: 14px;
  display: flex;
  align-items: center;
  justify-content: center;
  transition: all 0.2s;
}

.tsm-close-btn:hover { background: rgba(255,255,255,0.1); color: #fff; }

/* — Progress Track — */
.tsm-progress-track {
  height: 3px;
  background: rgba(255, 255, 255, 0.07);
  flex-shrink: 0;
}
```

```
.tsm-progress-fill {
  height: 100%;
  border-radius: 3px;
  transition: width 0.5s ease, background 0.5s ease;
}

/* — Slide Dots — */
.tsm-dots {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 7px;
  padding: 12px 24px 8px;
  flex-shrink: 0;
  flex-wrap: wrap;
}

.tsm-dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
  border: 1px solid rgba(255, 255, 255, 0.2);
  background: rgba(255, 255, 255, 0.08);
  cursor: pointer;
  transition: all 0.2s ease;
  padding: 0;
}

.tsm-dot:hover { border-color: rgba(255,255,255,0.5); transform:
scale(1.2); }
.tsm-dot.active { width: 20px; border-radius: 4px; }
.tsm-dot.done { background: rgba(255,255,255,0.2); border-color:
rgba(255,255,255,0.3); }

/* — Slide — */
.tsm-slide {
  flex: 1;
  overflow-y: auto;
  padding: 20px 32px 16px;
  border: 1px solid transparent;
```



```
margin: 0 16px;
border-radius: 20px;
transition: background 0.4s ease, border-color 0.4s ease;
}

.tsm-slide::-webkit-scrollbar { width: 4px; }
.tsm-slide::-webkit-scrollbar-track { background: transparent; }
.tsm-slide::-webkit-scrollbar-thumb { background: rgba(255,255,255,0.1);
border-radius: 4px; }

/* Slide Animations */
@keyframes slideInForward {
  from { opacity: 0; transform: translateX(40px); }
  to   { opacity: 1; transform: none; }
}

@keyframes slideInBackward {
  from { opacity: 0; transform: translateX(-40px); }
  to   { opacity: 1; transform: none; }
}

.tsm-anim-forward { animation: slideInForward 0.35s
cubic-bezier(0.22,1,0.36,1); }
.tsm-anim-backward { animation: slideInBackward 0.35s
cubic-bezier(0.22,1,0.36,1); }

/* — Slide Content — */
.tsm-slide-tag {
  font-size: 10.5px;
  font-weight: 700;
  letter-spacing: 1px;
  text-transform: uppercase;
  border: 1px solid;
  border-radius: 20px;
  padding: 3px 10px;
  display: inline-block;
  margin-bottom: 12px;
}

.tsm-slide-icon {
```

```
    font-size: 42px;
    margin-bottom: 8px;
    line-height: 1;
}

.tsm-slide-title {
    font-size: 22px;
    font-weight: 700;
    color: #f1f5f9;
    margin: 0 0 6px;
    letter-spacing: -0.4px;
}

.tsm-slide-subtitle {
    font-size: 13.5px;
    color: rgba(255,255,255,0.45);
    margin: 0 0 18px;
    font-weight: 400;
}

.tsm-body-text {
    font-size: 15px;
    color: #cbd5e1;
    line-height: 1.7;
    margin: 0 0 16px;
}

/* — Causes Grid — */
.tsm-causes-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 12px;
    margin-bottom: 16px;
}

.tsm-cause-card {
    background: rgba(255,255,255,0.03);
    border: 1px solid;
    border-radius: 14px;
    padding: 16px 14px;
```

```
text-align: center;
animation: cardPop 0.4s ease both;
}

@keyframes cardPop {
  from { opacity: 0; transform: scale(0.9) translateY(10px); }
  to   { opacity: 1; transform: none; }
}

.tsm-cause-icon { font-size: 30px; display: block; margin-bottom: 8px; }
.tsm-cause-label { font-size: 13px; font-weight: 600; display: block;
margin-bottom: 6px; }
.tsm-cause-desc { font-size: 12px; color: rgba(255,255,255,0.45); margin:
0; line-height: 1.5; }

/* — Steps — */
.tsm-steps {
  display: flex;
  flex-direction: column;
  gap: 0;
  margin-bottom: 16px;
}

.tsm-step {
  display: flex;
  flex-direction: column;
  animation: cardPop 0.4s ease both;
}

.tsm-step-body {
  display: flex;
  align-items: flex-start;
  gap: 14px;
  background: rgba(255,255,255,0.03);
  border: 1px solid rgba(255,255,255,0.07);
  border-radius: 12px;
  padding: 14px;
  margin-bottom: 4px;
}
```

```
.tsm-step-num {
  width: 28px;
  height: 28px;
  border-radius: 50%;
  color: #0a0fle;
  font-weight: 700;
  font-size: 13px;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
}

.tsm-step-icon { font-size: 24px; flex-shrink: 0; }

.tsm-step-body strong { font-size: 14px; display: block; margin-bottom:
3px; }
.tsm-step-body p { font-size: 12.5px; color: rgba(255,255,255,0.45);
margin: 0; line-height: 1.5; }

.tsm-step-arrow {
  text-align: center;
  color: rgba(255,255,255,0.25);
  font-size: 16px;
  line-height: 1;
  margin-bottom: 4px;
}

/* — Signs Grid — */
.tsm-signs-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 12px;
  margin-bottom: 16px;
}

.tsm-sign-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid;
  border-radius: 14px;
```

```
padding: 16px;
text-align: center;
animation: cardPop 0.4s ease both;
}

.tsm-sign-icon { font-size: 28px; display: block; margin-bottom: 6px; }
.tsm-sign-card strong { font-size: 13px; display: block; margin-bottom: 6px; }
.tsm-sign-card p { font-size: 12px; color: rgba(255,255,255,0.4); margin: 0; line-height: 1.5; }

/* — Timeline — */
.tsm-timeline {
  display: flex;
  flex-direction: column;
  gap: 10px;
  margin-bottom: 16px;
}

.tsm-event {
  display: flex;
  align-items: flex-start;
  gap: 14px;
  animation: cardPop 0.4s ease both;
}

.tsm-event-year {
  font-size: 12px;
  font-weight: 700;
  border: 1px solid;
  border-radius: 8px;
  padding: 4px 10px;
  white-space: nowrap;
  flex-shrink: 0;
}

.tsm-event-body {
  background: rgba(255,255,255,0.03);
  border: 1px solid rgba(255,255,255,0.07);
  border-radius: 10px;
```

```
padding: 10px 14px;
flex: 1;
}

.tsm-event-body strong { font-size: 13px; color: #f1f5f9; display: block;
margin-bottom: 3px; }
.tsm-event-body p { font-size: 12px; color: rgba(255,255,255,0.45);
margin: 0; }

/* — Effects List — */
.tsm-effects-list {
display: flex;
flex-direction: column;
gap: 10px;
margin-bottom: 16px;
}

.tsm-effect-row {
display: flex;
align-items: flex-start;
gap: 14px;
background: rgba(255,255,255,0.03);
border: 1px solid rgba(255,255,255,0.07);
border-radius: 12px;
padding: 12px 14px;
animation: cardPop 0.4s ease both;
}

.tsm-effect-icon { font-size: 24px; flex-shrink: 0; }
.tsm-effect-row strong { font-size: 13.5px; display: block; margin-bottom:
3px; }
.tsm-effect-row p { font-size: 12.5px; color: rgba(255,255,255,0.42);
margin: 0; line-height: 1.5; }

/* — Tips Grid — */
.tsm-tips-grid {
display: grid;
grid-template-columns: 1fr 1fr;
gap: 10px;
margin-bottom: 16px;
}
```

```
}

.tsm-tip-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid rgba(255,255,255,0.07);
  border-radius: 12px;
  padding: 14px;
  display: flex;
  align-items: flex-start;
  gap: 10px;
  animation: cardPop 0.4s ease both;
}

.tsm-tip-icon { font-size: 22px; flex-shrink: 0; }
.tsm-tip-card p { font-size: 12.5px; color: rgba(255,255,255,0.55);
margin: 0; line-height: 1.5; }

/* — During Cols — */
.tsm-during-cols {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 14px;
  margin-bottom: 16px;
}

.tsm-do-header {
  font-size: 13px;
  font-weight: 700;
  margin-bottom: 10px;
  letter-spacing: 0.3px;
}

.tsm-do-item {
  font-size: 12.5px;
  color: rgba(255,255,255,0.5);
  line-height: 1.5;
  margin-bottom: 8px;
  display: flex;
  gap: 8px;
  animation: cardPop 0.4s ease both;
}
```

```

}

/* — Measures Grid — */
.tsm-measures-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
  margin-bottom: 16px;
}

.tsm-measure-card {
  background: rgba(255,255,255,0.03);
  border: 1px solid;
  border-radius: 14px;
  padding: 14px 12px;
  text-align: center;
  animation: cardPop 0.4s ease both;
}

.tsm-measure-icon { font-size: 26px; display: block; margin-bottom: 8px; }
.tsm-measure-card strong { font-size: 12px; display: block; margin-bottom: 5px; }
.tsm-measure-card p { font-size: 11.5px; color: rgba(255,255,255,0.4); margin: 0; line-height: 1.5; }

/* — Fact Badge — */
.tsm-fact-badge {
  display: inline-flex;
  align-items: center;
  gap: 8px;
  border: 1px solid;
  border-radius: 30px;
  padding: 7px 16px;
  font-size: 13px;
  font-weight: 500;
  margin-top: 4px;
}

/* — Navigation — */
.tsm-nav {

```



```
display: flex;
align-items: center;
justify-content: space-between;
padding: 14px 24px 20px;
flex-shrink: 0;
border-top: 1px solid rgba(255,255,255,0.07);
}

.tsm-nav-btn {
padding: 10px 22px;
border-radius: 10px;
border: 1px solid rgba(255,255,255,0.15);
background: transparent;
color: rgba(255,255,255,0.6);
font-family: 'Sora', sans-serif;
font-size: 14px;
font-weight: 600;
cursor: pointer;
transition: all 0.2s ease;
}

.tsm-nav-btn:disabled { opacity: 0.3; cursor: not-allowed; }
.tsm-nav-btn:hover:not(:disabled) { background: rgba(255,255,255,0.08);
color: #fff; }

.tsm-nav-btn.tsm-next {
color: #0a0f1e;
}

.tsm-nav-btn.tsm-next:hover:not(:disabled) {
opacity: 0.88;
transform: translateY(-1px);
}

.tsm-nav-center {
flex: 1;
display: flex;
align-items: center;
justify-content: center;
}
```

```

.tsm-nav-hint {
  font-size: 11.5px;
  color: rgba(255,255,255,0.25);
  text-transform: uppercase;
  letter-spacing: 0.5px;
}

.tsm-finish-btn {
  padding: 10px 24px;
  background: linear-gradient(135deg, #6ee7b7, #3b82f6);
  border: none;
  border-radius: 10px;
  color: #0a0f1e;
  font-family: 'Sora', sans-serif;
  font-size: 14px;
  font-weight: 700;
  cursor: pointer;
  transition: all 0.2s ease;
  box-shadow: 0 4px 16px rgba(59, 130, 246, 0.3);
}

.tsm-finish-btn:hover {
  opacity: 0.9;
  transform: translateY(-2px);
}

/* — Responsive — */
@media (max-width: 600px) {
  .tsm-causes-grid { grid-template-columns: 1fr; }
  .tsm-signs-grid { grid-template-columns: 1fr; }
  .tsm-tips-grid { grid-template-columns: 1fr; }
  .tsm-during-cols { grid-template-columns: 1fr; }
  .tsm-measures-grid { grid-template-columns: 1fr 1fr; }
  .tsm-slide { padding: 16px; }
}

81",fontWeight:700,fontSize:13,textAlign:"center"}}>
      ☒ Completed {asn.completed_at ? `on ${new
Date(asn.completed_at).toLocaleDateString()}` : ""}
    </div>
  )}

```

```

        </div>
      </div>
    );
  }
}
</div>
) }
</div>
);
}
import React, { useState, useEffect } from "react";
import { auth } from "../../firebase";
import { signOut } from "firebase/auth";
import { useNavigate } from "react-router-dom";
import axios from "axios";
import TsunamiModule from "./TsunamiModule";
import EarthquakeModule from "./EarthquakeModule";
import DesertificationModule from "./DesertificationModule";
import "./Dashboard.css";

// — Static quiz & alert data —————
const QUIZZES = [
  { id: 1, title: "Earthquake Quiz", score: 92, date: "May 1" },
  { id: 2, title: "Fire Safety Quiz", score: 88, date: "Apr 28" },
  { id: 3, title: "First Aid Basics", score: 75, date: "Apr 20" },
];

const ALERTS = [
  { id: 1, type: "info", message: "New module available: Cyclone Safety", time: "2h ago" },
  { id: 2, type: "success", message: "You earned the 'Fire Safety' badge!", time: "1d ago" },
  { id: 3, type: "warning", message: "Quiz deadline: Flood Safety – May 10", time: "2d ago" },
];

// — Hardcoded static modules (always visible) —————
const BASE_MODULES = [
  { id: 1, title: "Earthquake Preparedness", icon: "🏠", lessons: 12, color: "#f59e0b" },
  { id: 5, title: "Tsunami Awareness", icon: "🌊", lessons: 10, color: "#06b6d4", isTsunami: true },

```

```

    { id: 7, title: "Desertification",          icon: "🌵", lessons: 8,
color: "#10b981" },
];

const BASE_URL = "http://localhost:5000";

// — Helper: tag label & class for static modules —————
function getStaticTag(id) {
    if (id === 5) return { label: "NEW",          cls: "tag-new"          };
    if (id === 1) return { label: "ESSENTIAL",    cls: "tag-essential" };
    return          { label: "ENVIRONMENTAL", cls: "tag-env"          };
}

function getStaticMeta(id) {
    if (id === 5) return "10 interactive slides · Causes, warning signs,
safety protocols";
    if (id === 1) return "12 lessons · Drop, Cover & Hold On — seismic
survival skills";
    return          "8 lessons · Causes, impacts, and land degradation
solutions";
}

// —————
function StudentDashboard() {
    const navigate = useNavigate();

    // — State —————
    const [user,          setUser]          = useState(null);
    const [activeTab,      setActiveTab]     = useState("overview");
    const [greeting,       setGreeting]      = useState("Good day");
    const [quizHover,      setQuizHover]     = useState(false);
    const [tsunamiOpen,    setTsunamiOpen]   = useState(false);
    const [earthquakeOpen, setEarthquakeOpen] = useState(false);
    const [desertOpen,      setDesertOpen]   = useState(false);
    const [moduleProgress, setModuleProgress] = useState({});
    const [loadingProgress, setLoadingProgress] = useState(false);

    // — NEW: DB published modules —————
    const [dbModules,      setDbModules]     = useState([]);
    const [loadingModules, setLoadingModules] = useState(false);

```

```

// — Opened DB module viewer —————
const [openDbModule, setOpenDbModule] = useState(null); // holds
full module object

// — Fetch published DB modules —————
const fetchDbModules = async () => {
  setLoadingModules(true);
  try {
    // No ?role=admin → backend returns ONLY is_published = 1
    const res = await axios.get(`${BASE_URL}/api/modules`);
    setDbModules(res.data);
  } catch (err) {
    console.warn("Could not load DB modules:", err.message);
  } finally {
    setLoadingModules(false);
  }
};

// — Fetch student progress —————
const fetchProgress = async (uid) => {
  setLoadingProgress(true);
  try {
    const res = await
    axios.get(`${BASE_URL}/api/modules/progress/${uid}`);
    const map = {};
    res.data.forEach((row) => {
      map[row.module_id] = { progress: row.progress, completed:
row.completed };
    });
    setModuleProgress(map);
  } catch (err) {
    // Fallback: localStorage
    const tsunamiPct =
parseInt(localStorage.getItem("tsunami_progress") || "0", 10);
    const earthquakePct =
parseInt(localStorage.getItem("earthquake_progress") || "0", 10);
    const desertPct =
parseInt(localStorage.getItem("desert_progress") || "0", 10);
    setModuleProgress({
      5: { progress: tsunamiPct, completed: tsunamiPct >= 100 },

```

```

        1: { progress: earthquakePct, completed: earthquakePct >= 100 },
        7: { progress: desertPct,      completed: desertPct      >= 100 },
    });
  } finally {
    setLoadingProgress(false);
  }
};

```

// — On mount —————

```

useEffect(() => {
  // Greeting
  const hour = new Date().getHours();
  if (hour < 12)      setGreeting("Good morning");
  else if (hour < 17) setGreeting("Good afternoon");
  else               setGreeting("Good evening");

  // Auth listener
  const unsub = auth.onAuthStateChanged(async (u) => {
    setUser(u);
    if (u) await fetchProgress(u.uid);
  });

```

```

  //  Fetch published modules from DB
  fetchDbModules();

```

```

  return () => unsub();
}, []);

```

// — Static module close handlers —————

```

const saveProgress = async (moduleId, localKey) => {
  const pct = parseInt(localStorage.getItem(localKey) || "0", 10);
  setModuleProgress((prev) => ({
    ...prev,
    [moduleId]: { progress: pct, completed: pct >= 100 },
  }));
  try {
    const uid = auth.currentUser?.uid;
    if (uid) {
      await axios.post(`${BASE_URL}/api/modules/progress`, {
        userId: uid, moduleId, progress: pct, completed: pct >= 100,

```

```

    });
  }
} catch (err) {
  console.warn("Progress save failed:", err.message);
}
};

const handleTsunamiClose = async () => { setTsunamiOpen(false);
await saveProgress(5, "tsunami_progress"); };
const handleEarthquakeClose = async () => { setEarthquakeOpen(false);
await saveProgress(1, "earthquake_progress"); };
const handleDesertClose = async () => { setDesertOpen(false);
await saveProgress(7, "desert_progress"); };

// — Save progress for DB modules —————
const handleDbModuleClose = async (moduleId, pct) => {
  setOpenDbModule(null);
  setModuleProgress((prev) => ({
    ...prev,
    [moduleId]: { progress: pct, completed: pct >= 100 },
  }));
  try {
    const uid = auth.currentUser?.uid;
    if (uid) {
      await axios.post(`${BASE_URL}/api/modules/progress`, {
        userId: uid, moduleId, progress: pct, completed: pct >= 100,
      });
    }
  } catch (err) {
    console.warn("DB module progress save failed:", err.message);
  }
};

// — Build merged module list —————
const getAllModules = () => {
  // 1. Static hardcoded modules
  const statics = BASE_MODULES.map((m) => {
    const p = moduleProgress[m.id];
    const progress = p ? p.progress : 0;
    const completed = Math.round((progress / 100) * m.lessons);

```

```

    const badge    = progress === 100 ? "Completed" : progress > 0 ? "In
Progress" : "Not Started";
    return { ...m, progress, completed, badge, isStatic: true };
  });

// 2.  DB published modules (exclude any that clash with static IDs)
const dynamics = dbModules
  .filter((dm) => !BASE_MODULES.find((sm) => sm.id === dm.id))
  .map((dm) => {
    const p      = moduleProgress[dm.id];
    const progress = p ? p.progress : 0;
    const lessons = dm.lessons_count || 0;
    const completed = Math.round((progress / 100) * lessons);
    const badge    = progress === 100 ? "Completed" : progress > 0 ?
    "In Progress" : "Not Started";
    return {
      ...dm,
      lessons,
      progress,
      completed,
      badge,
      isDynamic: true,
      icon: dm.icon || ,
      color: dm.color || "#3b82f6",
    };
  });

return [...statics, ...dynamics];
};

const MODULES = getAllModules();

// — Misc —————
const handleLogout = async () => {
  await signOut(auth);
  localStorage.removeItem("role");
  navigate("/login");
};

```



```

    const firstName = user?.displayName?.split(" ")[0] ||
user?.email?.split("@")[0] || "Student";

    const overallProgress =
      MODULES.length > 0
        ? Math.round(MODULES.reduce((a, m) => a + m.progress, 0) /
MODULES.length)
        : 0;

    // — Open handler for any module —————
    const openModule = (m) => {
      if (m.isStatic) {
        if (m.id === 1) setEarthquakeOpen(true);
        if (m.id === 5) setTsunamiOpen(true);
        if (m.id === 7) setDesertOpen(true);
      } else {
        setOpenDbModule(m); // shows inline DB viewer
      }
    };

    const NAV_ITEMS = [
      { id: "overview", label: "Overview",
        icon: "M3 13h8V3H3v10zm0 8h8v-6H3v6zm10
0h8V11h-8v10zm0-18v6h8V3h-8z" },
      { id: "modules", label: "Modules",
        icon: "M21 5c-1.11-.35-2.33-.5-3.5-.5-1.95 0-4.05.4-5.5
1.5-1.45-1.1-3.55-1.5-5.5-1.5S2.45 4.9 1 6v14.65c0 .25.25.5.5.1 0
.15-.05.25-.05C3.1 20.45 5.05 20 6.5 20c1.95 0 4.05.4 5.5 1.5 1.35-.85
3.8-1.5 5.5-1.5 1.65 0 3.35.3 4.75 1.05.1.05.15.05.25.05.25 0
.5-.25.5-.5V6c-.6-.45-1.25-.75-2-1z" },
      { id: "quizzes", label: "Quizzes",
        icon: "M9 21c0 .55.45 1 1 1h4c.55 0 1-.45 1-1v-1H9v1zm3-19C8.14 2 5
5.14 5 9c0 2.38 1.19 4.47 3 5.74V17c0 .55.45 1 1 1h6c.55 0 1-.45
1-1v-2.26c1.81-1.27 3-3.36 3-5.74 0-3.86-3.14-7-7-7z" },
      { id: "alerts", label: "Alerts",
        icon: "M12 22c1.1 0 2-.9 2-2h-4c0 1.1.9 2 2
2zm6-6v-5c0-3.07-1.64-5.64-4.5-6.32V4c0-.83-.67-1.5-1.5-1.5s-1.5.67-1.5
1.5v.68C7.63 5.36 6 7.92 6 11v5l-2 2v1h16v-11-2-2z" },
    ];

```

```

// — DB Module inline viewer —————
if (openDbModule) {
  return (
    <DbModuleViewer
      module={openDbModule}
      progress={moduleProgress[openDbModule.id]?.progress || 0}
      onClose={ (pct) => handleDbModuleClose(openDbModule.id, pct) }
    />
  );
}

// — Main render —————
return (
  <div className="dash-root">
    { /* Static module modals */ }
    {tsunamiOpen    && <TsunamiModule
onClose={handleTsunamiClose}    moduleId={5} />}
    {earthquakeOpen && <EarthquakeModule
onClose={handleEarthquakeClose} />}
    {desertOpen     && <DesertificationModule
onClose={handleDesertClose}     />}

    { /* — Sidebar — */ }
    <aside className="dash-sidebar">
      <div className="dash-logo">
        <svg width="28" height="28" viewBox="0 0 32 32" fill="none">
          <path d="M16 2L28 9v14L16 30 4 23V9L16 2z"
fill="url(#hexGradD)" />
          <circle cx="16" cy="16" r="3" fill="white" />
          <defs>
            <linearGradient id="hexGradD" x1="4" y1="2" x2="28" y2="30"
gradientUnits="userSpaceOnUse">
              <stop stopColor="#6ee7b7" />
              <stop offset="1" stopColor="#3b82f6" />
            </linearGradient>
          </defs>
        </svg>
        <span>DisasterReady</span>
      </div>

```

```

<nav className="dash-nav">
  {NAV_ITEMS.map((item) => (
    <button
      key={item.id}
      className={`dash-nav-item ${activeTab === item.id ? "active"
: ""}`}`
      onClick={() => setActiveTab(item.id)}
    >
      <svg width="18" height="18" viewBox="0 0 24 24"
fill="currentColor">
        <path d={item.icon} />
      </svg>
      {item.label}
      {item.id === "alerts" && ALERTS.length > 0 && (
        <span className="nav-badge">{ALERTS.length}</span>
      )}
    </button>
  ))}
</nav>

{/* Progress ring */}
<div className="sidebar-progress-card">
  <p className="sidebar-progress-label">Overall Progress</p>
  <div className="sidebar-progress-ring">
    <svg width="60" height="60" viewBox="0 0 60 60">
      <circle cx="30" cy="30" r="24" fill="none"
stroke="rgba(255,255,255,0.07)" strokeWidth="5" />
      <circle
        cx="30" cy="30" r="24" fill="none" stroke="#6ee7b7"
strokeWidth="5"
        strokeDasharray={`$ {2 * Math.PI * 24}`}
        strokeDashoffset={`$ {2 * Math.PI * 24 * (1 -
overallProgress / 100)}`}
        strokeLinecap="round"
        style={{ transform: "rotate(-90deg)", transformOrigin:
"center", transition: "stroke-dashoffset 1s ease" }}
      />
    </svg>
    <span
className="sidebar-progress-pct">{overallProgress}%</span>

```

```

        </div>
    </div>

    <div className="dash-sidebar-footer">
        <div className="dash-user-pill">
            <div
className="dash-avatar">{firstName.charAt(0).toUpperCase()}</div>
            <div className="dash-user-info">
                <span className="dash-user-name">{firstName}</span>
                <span className="dash-user-role">Student</span>
            </div>
        </div>
        <button className="dash-logout" onClick={handleLogout}
title="Logout">
            <svg width="18" height="18" viewBox="0 0 24 24"
fill="currentColor">
                <path d="M17 7l-1.41 1.41L18.17 11H8v2h10.17l-2.58 2.58L17
17l5-5zM4 5h8V3H4c-1.1 0-2 .9-2 2v14c0 1.1 9 2 2 2h8v-2H4V5z" />
            </svg>
        </button>
    </div>
</aside>

{/* — Main — */}
<main className="dash-main">
    {/* Topbar */}
    <div className="dash-topbar">
        <div>
            <h2 className="dash-welcome">{greeting}, {firstName} 🙌</h2>
            <p className="dash-date">
                {new Date().toLocaleDateString("en-US", { weekday: "long",
month: "long", day: "numeric" })}
            </p>
        </div>
        <div className="dash-topbar-actions">
            <div className="dash-search">
                <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
                    <path d="M15.5 14h-.79l-.28-.27A6.471 6.471 0 0016 9.5 6.5
6.5 0 109.5 16c1.61 0 3.09-.59 4.23-1.57l.27.28v.79l5 4.99L20.49

```

```

191-4.99-5zm-6 0C7.01 14 5 11.99 5 9.5S7.01 5 9.5 5 14 7.01 14 9.5 11.99
14 9.5 14z" />
    </svg>
    <input placeholder="Search modules, quizzes..." />
  </div>
</div>
</div>

{/* ===== OVERVIEW TAB ===== */}
{activeTab === "overview" && (
  <div className="dash-content">
    {/* AI Quiz Banner */}
    <div
      className={`quiz-banner-student ${quizHover ? "hovered" :
""}` `}
      onClick={() => navigate("/quiz-generator")}
      onMouseEnter={() => setQuizHover(true)}
      onMouseLeave={() => setQuizHover(false)}
    >
      <div className="qb-orb qb-orb-1" />
      <div className="qb-orb qb-orb-2" />
      <div className="qb-orb qb-orb-3" />
      <div className="qb-left">
        <div className="qb-icon-wrap">
          <svg width="26" height="26" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M7 2v11h3v9l7-12h-4l4-8z" />
          </svg>
          <div className="qb-icon-ping" />
        </div>
        <div className="qb-text">
          <div className="qb-badge">AI POWERED</div>
          <h3 className="qb-title">Take an AI Quiz</h3>
          <p className="qb-desc">Test your disaster-preparedness
knowledge with personalized AI-generated questions</p>
        </div>
      </div>
      <div className="qb-right">
        <div className="qb-stats">

```

```

        <div className="qb-stat"><span
className="qb-stat-num">10</span><span
className="qb-stat-label">Questions</span></div>
        <div className="qb-stat-divider" />
        <div className="qb-stat"><span
className="qb-stat-num">60s</span><span className="qb-stat-label">Per
Q</span></div>
    </div>
    <div className="qb-cta-btn">
        Start Quiz
        <svg width="15" height="15" viewBox="0 0 24 24"
fill="currentColor"><path d="M8 5v14l11-7z" /></svg>
    </div>
</div>

{/* Stat Cards */}
<div className="stat-grid">
    {[
        { label: "Overall Progress",    value:
`${overallProgress}%`,
color: "#3b82f6", icon: "M13 2.05v2.02c3.95.49 7 3.85 7 7.93 0 3.21-1.81
6-4.72 7.72L13 17v5h5l-1.22-1.22C19.91 19.07 22 15.76 22
12c0-5.18-3.95-9.45-9-9.95zM11 2.05C5.95 2.55 2 6.82 2 12c0 3.76 2.09 7.07
5.22 8.78L6 22h5V2.05z" },
        { label: "Completed Modules",  value: `${MODULES.filter(m
=> m.progress === 100).length}/${MODULES.length}`,
color:
"#10b981", icon: "M9 16.17L4.83 12l-1.42 1.41L9 19 21 7l-1.41-1.41L9
16.17z" },
        { label: "Avg. Quiz Score",    value:
`${Math.round QUIZZES.reduce((a, q) => a + q.score, 0) /
QUIZZES.length}%`, color: "#8b5cf6", icon: "M19 3H5c-1.1 0-2 .9-2
2v14c0 1.1.9 2 2 2h14c1.1 0 2-.9 2-2V5c0-1.1-.9-2-2-2zm-7 3c1.93 0 3.5
1.57 3.5 3.5S13.93 13 12 13s-3.5-1.57-3.5-3.5S10.07 6 12 6zm7
13H5v-.23c0-.62.28-1.2.76-1.58C7.47 15.82 9.64 15 12 15s4.53.82 6.24
2.19c.48.38.76.97.76 1.58V19z" },
        { label: "Active Alerts",      value: ALERTS.length,
color: "#f59e0b", icon: "M12 22c1.1 0 2-.9 2-2h-4c0 1.1.9 2 2
2zm6-6v-5c0-3.07-1.64-5.64-4.5-6.32V4c0-.83-.67-1.5-1.5-1.5s-1.5.67-1.5
1.5v.68C7.63 5.36 6 7.92 6 11v5l-2 2v1h16v-11-2-2z" },
    ]}

```

```

        ].map((stat) => (
            <div key={stat.label} className="stat-card">
                <div className="stat-icon" style={{ background:
stat.color + "22", color: stat.color }}>
                    <svg width="20" height="20" viewBox="0 0 24 24"
fill="currentColor"><path d={stat.icon} /></svg>
                </div>
                <div>
                    <p className="stat-label">{stat.label}</p>
                    <p className="stat-value">{stat.value}</p>
                </div>
            </div>
        )))
    </div>

    { /* Learning Modules list */ }
    <div className="section-card">
        <div className="mod-section-header">
            <h3 className="section-title" style={{ margin: 0
}}>Learning Modules</h3>
            <span className="mod-count-pill">{MODULES.length}
modules</span>
        </div>

        {loadingModules && (
            <p style={{ color: "#94a3b8", fontSize: 13, padding: "8px
0" }}>Loading modules...</p>
        )}

        <div className="mod-cards-list">
            {MODULES.map((m) => {
                const badgeClass = m.progress === 100 ? "completed" :
m.progress > 0 ? "inprogress" : "notstarted";
                const badgeLabel = m.progress === 100 ? "Completed" :
m.progress > 0 ? "In Progress" : "Not Started";
                const tag = m.isStatic ? getStaticTag(m.id) : {
label: m.category || "MODULE", cls: "tag-new" };
                const metaText = m.isStatic
                    ? getStaticMeta(m.id)
                    : (m.description || `${m.lessons} lessons`);
            })}
        </div>
    </div>

```

```

        return (
          <div
            key={m.id}
            className="mod-list-card"
            style={{ "--accent": m.color, "--icon-bg": m.color +
"1a" }}

            onClick={() => openModule(m)}
          >
            <div className="mod-list-icon">{m.icon}</div>
            <div className="mod-list-body">
              <div className="mod-list-top">
                <p className="mod-list-title">{m.title}</p>
                <span className={`mod-list-tag
${tag.cls}`}>{tag.label}</span>
              </div>
              <p className="mod-list-meta">{metaText}</p>
              <div className="mod-list-progress-row">
                <div className="progress-bar-bg">
                  <div className="progress-bar-fill" style={{
width: `${m.progress}%`, background: m.color }} />
                </div>
                <span className="mod-list-pct" style={{ color:
m.progress > 0 ? m.color : "rgba(255,255,255,0.25)" }}>
                  {m.progress}%
                </span>
                <span className={`mod-list-badge
${badgeClass}`}>{badgeLabel}</span>
              </div>
            </div>
            <span className="mod-list-arrow">></span>
          </div>
        );
      })}
    </div>
  </div>

  {/* Recent Quizzes */}
  <div className="section-card">
    <h3 className="section-title">Recent Quiz Results</h3>

```



```

        {QUIZZES.map((q) => (
          <div key={q.id} className="quiz-row">
            <div>
              <p className="quiz-name">{q.title}</p>
              <p className="quiz-date">{q.date}</p>
            </div>
            <div className="quiz-score" style={{ color: q.score >=
85 ? "#10b981" : q.score >= 70 ? "#f59e0b" : "#ef4444" }}>
              {q.score}%
            </div>
          </div>
        ))}
      </div>
    </div>
  )}

  { /* ===== MODULES TAB ===== */
  {activeTab === "modules" && (
    <div className="dash-content">
      {loadingModules && (
        <p style={{ color: "#94a3b8", fontSize: 13, marginBottom: 12
}}>Loading modules...</p>
      )}
      <div className="modules-grid">
        {MODULES.map((m) => (
          <div
            key={m.id}
            className="module-card"
            onClick={() => openModule(m)}
            style={{ cursor: "pointer", ...(m.isTsunami ? { border:
`1px solid ${m.color}44` } : {}) }}
          >
            <div className="module-card-header" style={{ background:
m.color + "22" }}>
              <span style={{ fontSize: 36 }}>{m.icon}</span>
              <div style={{ display: "flex", flexDirection:
"column", alignItems: "flex-end", gap: 4 }}>
                {m.isDynamic && (
                  <span style={{ fontSize: 10, color: m.color,
background: m.color + "22", border: `1px solid ${m.color}44`,

```



```

    })

    { /* ===== QUIZZES TAB ===== */
    {activeTab === "quizzes" && (
      <div className="dash-content">
        <div className="quiz-tab-cta" onClick={() =>
navigate("/quiz-generator")}>
          <svg width="18" height="18" viewBox="0 0 24 24"
fill="currentColor"><path d="M7 2v11h3v9l7-12h-4l4-8z" /></svg>
          Generate a new AI Quiz
          <svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor" style={{ marginLeft: "auto" }}><path d="M8 5v14l11-7z"
/></svg>
        </div>
        <div className="section-card">
          <h3 className="section-title">All Quiz Results</h3>
          {QUIZZES.map((q) => (
            <div key={q.id} className="quiz-row" style={{ padding:
"18px 0" }}>
              <div>
                <p className="quiz-name" style={{ fontSize: 16
}}>{q.title}</p>
                <p className="quiz-date">{q.date}</p>
              </div>
              <div className="quiz-score-bar">
                <div className="score-track">
                  <div className="score-fill" style={{ width:
`${q.score}%`, background: q.score >= 85 ? "#10b981" : q.score >= 70 ?
"#f59e0b" : "#ef4444" }} />
                </div>
                <span className="quiz-score" style={{ color: q.score
>= 85 ? "#10b981" : q.score >= 70 ? "#f59e0b" : "#ef4444"
}}>{q.score}%</span>
              </div>
            </div>
          ))}
        </div>
      </div>
    )}
  )}

```

```

    { /* ===== ALERTS TAB ===== */
    {activeTab === "alerts" && (
      <div className="dash-content">
        <div className="section-card">
          <h3 className="section-title">Notifications</h3>
          {ALERTS.map((a) => (
            <div key={a.id} className={`alert-row alert-${a.type}`}>
              <div className="alert-dot" />
              <div className="alert-body">
                <p className="alert-msg">{a.message}</p>
                <p className="alert-time">{a.time}</p>
              </div>
            </div>
          ))}
        </div>
      </div>
    )}
  </main>
</div>
);
}

// -----
// DB Module Viewer — shown when a student opens a dynamic module
// -----

function DbModuleViewer({ module: mod, progress: initialProgress, onClose
}) {
  const [sections, setSections] = useState([]);
  const [quizzes, setQuizzes] = useState([]);
  const [loading, setLoading] = useState(true);
  const [step, setStep] = useState(0); // 0 = intro, 1..N
= sections, N+1 = quiz
  const [quizAnswers, setQuizAnswers] = useState({});
  const [quizDone, setQuizDone] = useState(false);
  const [score, setScore] = useState(0);

  useEffect(() => {
    axios.get(`http://localhost:5000/api/modules/${mod.id}`)
      .then(res => {
        setSections(res.data.sections || []);

```

```

        setQuizzes(res.data.quizzes || []);
    })
    .catch(err => console.warn("Module load error:", err.message))
    .finally(() => setLoading(false));
}, [mod.id]);

const totalSteps = 1 + sections.length + (quizzes.length > 0 ? 1 : 0);
const currentPct = Math.round(((step) / Math.max(totalSteps - 1, 1)) *
100);
const isQuizStep = step === 1 + sections.length && quizzes.length > 0;
const isIntro = step === 0;
const sectionIndex = step - 1; // valid when 1 <= step <=
sections.length

const handleAnswer = (qIdx, ans) => setQuizAnswers(prev => ({ ...prev,
[qIdx]: ans }));

const submitQuiz = () => {
    let correct = 0;
    quizzes.forEach((q, i) => {
        if ((quizAnswers[i] || "").toUpperCase() ===
q.correct_answer.toUpperCase()) correct++;
    });
    setScore(Math.round((correct / quizzes.length) * 100));
    setQuizDone(true);
};

const handleFinish = () => onClose(100);

if (loading) {
    return (
        <div style={overlayStyle}>
            <div style={panelStyle}>
                <p style={{ color: "#94a3b8", textAlign: "center", paddingTop:
60 }}>Loading module...</p>
            </div>
        </div>
    );
}

```

```

return (
  <div style={overlayStyle}>
    <div style={panelStyle}>
      {/* Header */}
      <div style={{ display: "flex", alignItems: "center",
justifyContent: "space-between", padding: "20px 24px 0" }}>
        <div style={{ display: "flex", alignItems: "center", gap: 12 }}>
          <span style={{ fontSize: 28 }}>{mod.icon}</span>
          <div>
            <h2 style={{ margin: 0, fontSize: 18, color: "#f1f5f9"
}}>{mod.title}</h2>
            <p style={{ margin: 0, fontSize: 12, color: "#64748b"
}}>{mod.category}</p>
          </div>
        </div>
        <button onClick={() => onClose(currentPct)}
style={closeBtnStyle}>X</button>
      </div>

      {/* Progress bar */}
      <div style={{ padding: "12px 24px 0" }}>
        <div style={{ height: 4, background: "#1e293b", borderRadius: 4
}}>
          <div style={{ height: "100%", width: `${currentPct}%`,
background: mod.color, borderRadius: 4, transition: "width .4s ease" }} />
        </div>
        <p style={{ margin: "4px 0 0", fontSize: 11, color: "#64748b"
}}>
          {step === 0 ? "Introduction" : isQuizStep ? "Quiz" : `Section
${sectionIndex + 1} of ${sections.length}`} · {currentPct}%
        </p>
      </div>

      {/* Body */}
      <div style={{ padding: "20px 24px", overflowY: "auto", flex: 1 }}>
        {/* INTRO */}
        {isIntro && (
          <div>
            <h3 style={{ color: "#f1f5f9", marginTop: 0 }}>About this
module</h3>

```

```

        <p style={{ color: "#94a3b8", lineHeight: 1.7
}}>{mod.description || "No description provided."}</p>
        <div style={{ marginTop: 16, display: "flex", gap: 12,
flexWrap: "wrap" }}>
            <span style={chipStyle(mod.color)}>📖 {sections.length}
sections</span>
            {quizzes.length > 0 && <span
style={chipStyle("#8b5cf6")}>🧠 {quizzes.length} quiz questions</span>}
            <span style={chipStyle("#10b981")}>📁
{mod.category}</span>
        </div>
    </div>
    )}

    {/* SECTION */}
    {!isIntro && !isQuizStep && sections[sectionIndex] && (() => {
        const sec = sections[sectionIndex];
        return (
            <div>
                {sec.tag && <span style={chipStyle(sec.color ||
mod.color)}>{sec.tag}</span>}
                <h3 style={{ color: "#f1f5f9", marginTop: sec.tag ? 10 : 0
}}>{sec.title}</h3>
                {sec.image_url && (
                    <img src={sec.image_url} alt={sec.title} style={{ width:
"100%", borderRadius: 10, marginBottom: 16, objectFit: "cover", maxHeight:
220 }} />
                )}
                <p style={{ color: "#94a3b8", lineHeight: 1.8, whiteSpace:
"pre-wrap" }}>{sec.content}</p>
            </div>
        );
    }) () }

    {/* QUIZ */}
    {isQuizStep && !quizDone && (
        <div>
            <h3 style={{ color: "#f1f5f9", marginTop: 0 }}>Module
Quiz</h3>
            {quizzes.map((q, i) => (

```

```

        <div key={q.id} style={{ marginBottom: 28 }}>
          <p style={{ color: "#e2e8f0", fontWeight: 600,
marginBottom: 10 }}>
            Q{i + 1}. {q.question}
          </p>
          {[ "A", "B", "C", "D" ].map((opt) => {
            const text = q[`option_${opt.toLowerCase()}`];
            if (!text) return null;
            const selected = quizAnswers[i] === opt;
            return (
              <div
                key={opt}
                onClick={() => handleAnswer(i, opt)}
                style={{
                  padding: "10px 14px", borderRadius: 8,
marginBottom: 6, cursor: "pointer",
                  border: `1px solid ${selected ? mod.color :
"#334155"}`,
                  background: selected ? mod.color + "22" :
"#1e293b",
                  color: selected ? mod.color : "#94a3b8",
fontSize: 14, transition: "all .2s",
                }}
              >
                <strong>{opt}</strong> {text}
              </div>
            );
          })}
          </div>
        <button
          onClick={submitQuiz}
          disabled={Object.keys(quizAnswers).length <
quizzes.length}
          style={{ ...primaryBtn(mod.color), opacity:
Object.keys(quizAnswers).length < quizzes.length ? 0.5 : 1 }}
        >
          Submit Quiz
        </button>
      </div>

```



```

    ))

    { /* QUIZ RESULT */
    {isQuizStep && quizDone && (
        <div style={{ textAlign: "center", paddingTop: 20 }}>
            <div style={{ fontSize: 56 }}>{score} >= 70 ? "🎉" :
"📖"</div>
            <h3 style={{ color: "#f1f5f9" }}>Quiz Complete!</h3>
            <p style={{ fontSize: 32, fontWeight: 700, color: score >=
70 ? "#10b981" : "#f59e0b" }}>{score}%</p>
            <p style={{ color: "#64748b" }}>{score} >= 70 ? "Great job!
Module completed." : "Keep practising – you can retake the quiz."</p>
            <button onClick={handleFinish}
style={primaryBtn(mod.color)}>Finish Module ✓</button>
        </div>
    ))
</div>

    { /* Navigation */
    {!(isQuizStep && quizDone) && (
        <div style={{ display: "flex", justifyContent: "space-between",
padding: "16px 24px", borderTop: "1px solid #1e293b" }}>
            <button
                onClick={() => setStep(s => Math.max(0, s - 1))}
                disabled={step === 0}
                style={{ ...secondaryBtn, opacity: step === 0 ? 0.4 : 1 }}
            >
                ← Back
            </button>

            {isQuizStep ? (
                <button onClick={submitQuiz} style={primaryBtn(mod.color)}
                disabled={Object.keys(quizAnswers).length <
quizzes.length}
                >
                    Submit Quiz
                </button>
            ) : step < totalSteps - 1 ? (
                <button onClick={() => setStep(s => s + 1)}
style={primaryBtn(mod.color)}>

```

```

                {step === sections.length && quizzes.length === 0 ?
"Finish ✓" : "Next →"}
                </button>
            ) : (
                <button onClick={handleFinish}
style={primaryBtn(mod.color)}>Finish ✓</button>
            )}
        </div>
    )}
</div>
</div>
);
}

// — Inline styles for DbModuleViewer —————
const overlayStyle = {
    position: "fixed", inset: 0, background: "rgba(0,0,0,0.75)",
    display: "flex", alignItems: "center", justifyContent: "center",
    zIndex: 1000, padding: 16,
};
const panelStyle = {
    background: "#0f172a", borderRadius: 16, width: "100%", maxWidth: 680,
    maxHeight: "90vh", display: "flex", flexDirection: "column",
    border: "1px solid #1e293b", boxShadow: "0 25px 60px rgba(0,0,0,0.6)",
};
const closeBtnStyle = {
    background: "transparent", border: "none", color: "#64748b",
    fontSize: 18, cursor: "pointer", padding: "4px 8px", borderRadius: 6,
};
const chipStyle = (color) => ({
    fontSize: 12, color, background: color + "22",
    border: `1px solid ${color}44`, borderRadius: 20,
    padding: "3px 10px", fontWeight: 600,
});
const primaryBtn = (color) => ({
    background: color, color: "#fff", border: "none",
    borderRadius: 8, padding: "10px 22px", fontWeight: 700,
    fontSize: 14, cursor: "pointer",
});
const secondaryBtn = {

```

```

    background: "transparent", color: "#94a3b8",
    border: "1px solid #334155", borderRadius: 8,
    padding: "10px 22px", fontWeight: 600, fontSize: 14, cursor: "pointer",
  };

export default StudentDashboard;
import React, { useState } from "react";
import { auth } from "../firebase";
import { sendPasswordResetEmail } from "firebase/auth";
import { Link } from "react-router-dom";
import "../Auth.css";

const PARTICLES = Array.from({ length: 14 }, (_, i) => ({
  id: i,
  x: Math.random() * 100,
  y: Math.random() * 100,
  size: 2 + Math.random() * 4,
  duration: 6 + Math.random() * 10,
  delay: Math.random() * 5,
})));

function ForgotPassword() {
  const [email, setEmail] = useState("");
  const [loading, setLoading] = useState(false);
  const [sent, setSent] = useState(false);
  const [error, setError] = useState("");
  const [touched, setTouched] = useState(false);
  const [shake, setShake] = useState(false);

  const validate = () => {
    if (!email) return "Email is required";
    if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(email)) return "Invalid email
format";
    return "";
  };

  const triggerShake = () => {
    setShake(true);
    setTimeout(() => setShake(false), 600);
  };

```

```

const handleSubmit = async () => {
  setTouched(true);
  const err = validate();
  if (err) { setError(err); triggerShake(); return; }
  setLoading(true);
  setError("");
  try {
    await sendPasswordResetEmail(auth, email);
    setSent(true);
  } catch (e) {
    const msgs = {
      "auth/user-not-found": "No account found with this email address",
      "auth/invalid-email": "Invalid email address",
      "auth/too-many-requests": "Too many attempts – please try again
later",
    };
    setError(msgs[e.code] || "Something went wrong. Please try again.");
    triggerShake();
  } finally {
    setLoading(false);
  }
};

const handleKeyDown = (e) => {
  if (e.key === "Enter") handleSubmit();
};

return (
  <div className="auth-bg split-layout">
    {/* Particles */}
    <div className="particles" aria-hidden="true">
      {PARTICLES.map((p) => (
        <span
          key={p.id}
          className="particle"
          style={{
            left: `${p.x}%`,
            top: `${p.y}%`,
            width: p.size,

```

```

        height: p.size,
        animationDuration: `${p.duration}s`,
        animationDelay: `${p.delay}s`,
      }}
    />
  ))}
</div>

{/* — LEFT PANEL — */}
<div className="split-left">
  <div className="split-brand">
    <div className="split-brand-icon">
      <svg width="36" height="36" viewBox="0 0 32 32" fill="none">
        <path d="M16 2L28 9v14L16 30 4 23V9L16 2z"
fill="url(#hexFP)" />
        <path d="M16 8l6 3.5v7L16 22l-6-3.5v-7L16 8z"
fill="rgba(255,255,255,0.2)" />
        <circle cx="16" cy="16" r="3" fill="white" />
      <defs>
        <linearGradient id="hexFP" x1="4" y1="2" x2="28" y2="30"
gradientUnits="userSpaceOnUse">
          <stop stopColor="#6ee7b7" />
          <stop offset="1" stopColor="#3b82f6" />
        </linearGradient>
      </defs>
    </svg>
  </div>
  <div>
    <h1 className="split-brand-name">DisasterEdu</h1>
    <p className="split-brand-tagline">From Awareness to
Action</p>
  </div>
</div>

<div className="split-hero">
  <h2 className="split-hero-title">
    Reset Your<br />
    <span className="split-hero-accent">Password</span>
  </h2>
  <p className="split-hero-desc">

```

```

No worries — it happens to everyone. Enter your registered
email

    and we'll send you a secure link to create a new password.
</p>
</div>

{/* Steps */}
<div className="fp-steps">
    [
        { num: "1", title: "Enter your email", desc: "Provide the
email linked to your account", color: "#6ee7b7" },
        { num: "2", title: "Check your inbox", desc: "We'll send a
secure reset link instantly", color: "#3b82f6" },
        { num: "3", title: "Set new password", desc: "Click the link
and choose a strong password", color: "#8b5cf6" },
    ].map((step) => (
        <div key={step.num} className="fp-step-card" style={{
animationDelay: `${parseInt(step.num) * 0.12}s` }}>
            <div className="fp-step-num" style={{ background: step.color
+ "22", color: step.color, border: `1px solid ${step.color}44` }}>
                {step.num}
            </div>
            <div>
                <p className="fp-step-title">{step.title}</p>
                <p className="fp-step-desc">{step.desc}</p>
            </div>
        </div>
    )))
</div>

{/* — RIGHT PANEL — */}
<div className="split-right">
    <div className={`auth-card split-card ${shake ? "shake" : ""}
${sent ? "success-pulse" : ""}`}>

        {!sent ? (
            <>
                {/* Header */}
                <div className="auth-header">

```

```

        <div className="fp-icon-wrap">
            <svg width="28" height="28" viewBox="0 0 24 24"
fill="none">
                <path d="M20 4H4c-1.1 0-2 .9-2 2v12c0 1.1.9 2 2
2h16c1.1 0 2-.9 2-2V6c0-1.1-.9-2-2-2zm0 4l-8 5-8-5V6l8 5 8-5v2z"
fill="url(#mailGrad)" />
                <defs>
                    <linearGradient id="mailGrad" x1="2" y1="4" x2="22"
y2="20" gradientUnits="userSpaceOnUse">
                        <stop stopColor="#6ee7b7" />
                        <stop offset="1" stopColor="#3b82f6" />
                    </linearGradient>
                </defs>
            </svg>
        </div>
        <h2 className="brand-title" style={{ fontSize: 22
}}>Forgot Password?</h2>
        <p className="brand-sub">Enter your email to receive a
reset link</p>
    </div>

    { /* Error */ }
    { error && (
        <div className="alert-error" role="alert">
            <svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor">
                <path d="M12 2C6.48 2 2 6.48 2 12s4.48 10 10 10
10-4.48 10-10S17.52 2 12 2zm1 15h-2v-2h2v2zm0-4h-2V7h2v6z" />
            </svg>
            {error}
        </div>
    ) }

    { /* Email field */ }
    <div className={`field-group ${touched && error ?
"has-error" : touched && email && !error ? "has-success" : ""}`}>
        <label className="field-label" htmlFor="fp-email">Email
address</label>
        <div className="field-wrap">

```

```

        <svg className="field-icon" width="16" height="16"
viewBox="0 0 24 24" fill="currentColor">
            <path d="M20 4H4c-1.1 0-2 .9-2 2v12c0 1.1.9 2 2
2h16c1.1 0 2-.9 2-2V6c0-1.1-.9-2-2-2zm0 4l-8 5-8-5V6l8 5 8-5v2z" />
        </svg>
        <input
            id="fp-email"
            type="email"
            placeholder="you@school.edu"
            value={email}
            autoComplete="email"
            onChange={(e) => {
                setEmail(e.target.value);
                if (touched) setError(validate());
            }}
            onBlur={() => { setTouched(true);
setError(validate()); }}
            onKeyDown={handleKeyDown}
            disabled={loading}
            className="field-input"
        />
        {touched && !error && email && (
            <svg className="field-icon-right success" width="15"
height="15" viewBox="0 0 24 24" fill="currentColor">
                <path d="M9 16.17L4.83 12l-1.42 1.41L9 19 21
7l-1.41-1.41L9 16.17z" />
            </svg>
        )}
    </div>
    {touched && error && <p
className="field-error">{error}</p>}
</div>

    {/* Info note */}
    <div className="fp-info-note">
        <svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor">
            <path d="M12 2C6.48 2 2 6.48 2 12s4.48 10 10 10-4.48
10-10S17.52 2 12 2zm1 15h-2v-6h2v6zm0-8h-2V7h2v2z" />
        </svg>

```


The reset link will expire in **15 minutes**.
Check your spam folder if you don't see it.

```
</div>

{/* Submit button */}
<button
  onClick={handleSubmit}
  disabled={loading}
  className={`btn-primary ${loading ? "loading" : ""}`}
  type="button"
>
  {loading ? <span className="spinner" /> : (
    <>
      <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
        <path d="M2.01 21L23 12 2.01 3 2 10 15 2-15 2z" />
      </svg>
      Send Reset Link
    </>
  )}
</button>

<p className="auth-footer-text">
  Remember your password?{" "}
  <Link to="/login" className="auth-link">Back to
Login</Link>
</p>
</>
) : (
  /* — SUCCESS STATE — */
  <div className="fp-success-state">
    <div className="fp-success-icon">
      <svg width="36" height="36" viewBox="0 0 24 24"
fill="none">
        <circle cx="12" cy="12" r="11"
stroke="url(#successCircle)" strokeWidth="1.5" />
        <path d="M7 13 13.5 3.5 17 9" stroke="url(#successCheck)"
strokeWidth="2" strokeLinecap="round" strokeLinejoin="round" />
      <defs>
```

```

        <linearGradient id="successCircle" x1="1" y1="1"
x2="23" y2="23" gradientUnits="userSpaceOnUse">
            <stop stopColor="#6ee7b7" /><stop offset="1"
stopColor="#3b82f6" />
        </linearGradient>
        <linearGradient id="successCheck" x1="7" y1="9"
x2="17" y2="17" gradientUnits="userSpaceOnUse">
            <stop stopColor="#6ee7b7" /><stop offset="1"
stopColor="#3b82f6" />
        </linearGradient>
    </defs>
</svg>
</div>

<h2 className="brand-title" style={{ fontSize: 22,
marginBottom: 8 }}>Email Sent!</h2>
<p className="fp-success-desc">
    We've sent a password reset link to
</p>
<div className="fp-email-chip">
    <svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor">
        <path d="M20 4H4c-1.1 0-2 .9-2 2v12c0 1.1.9 2 2 2h16c1.1
0 2-.9 2-2V6c0-1.1-.9-2-2-2zm0 4l-8 5-8-5V6l8 5 8-5v2z" />
    </svg>
    {email}
</div>

<div className="fp-success-steps">
    {[
        "Open your email inbox",
        "Click the reset link in the email",
        "Choose a strong new password",
        "Log in with your new password",
    ]}.map((step, i) => (
        <div key={i} className="fp-success-step" style={{
animationDelay: `${0.1 + i * 0.1}s`} >
            <div className="fp-success-step-num">{i + 1}</div>
            <span>{step}</span>
        </div>
    ))

```

```

    )) }
  </div>

  <div className="fp-info-note" style={{ marginBottom: 24 }}>
    <svg width="14" height="14" viewBox="0 0 24 24"
fill="currentColor">
      <path d="M12 2C6.48 2 2 6.48 2 12s4.48 10 10 10 10-4.48
10-10S17.52 2 12 2zm1 15h-2v-6h2v6zm0-8h-2V7h2v2z" />
    </svg>
    Didn't receive it? Check your <strong>spam/junk</strong>
folder or{" "}

    <button
      className="fp-resend-btn"
      onClick={() => { setSent(false); setError("");
setTouched(false); }}
    >
      try again
    </button>
  </div>

  <Link to="/login" className="btn-primary" style={{
textDecoration: "none", display: "flex", alignItems: "center",
justifyContent: "center", gap: 8 }}>
    <svg width="16" height="16" viewBox="0 0 24 24"
fill="currentColor">
      <path d="M20 11H7.83l5.59-5.59L12 4l-8 8 8 8
1.41-1.41L7.83 13H20v-2z" />
    </svg>
    Back to Login
  </Link>
</div>

  )}
</div>
</div>
</div>
);
}

export default ForgotPassword;

```

```
import React from "react";
import { BrowserRouter as Router, Routes, Route, Navigate } from
"react-router-dom";

import Login from "../components/pages/Login";
import Signup from "../components/pages/Signup";
import ForgotPassword from "../components/Forgotpassword";
import StudentDashboard from "../components/pages/StudentDashboard";
import AdminDashboard from "../components/pages/AdminDashboard";
import QuizGenerator from "../components/pages/QuizGenerator";
import StudentAssignments from "../components/pages/StudentAssignments";

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Navigate to="/login" />} />
        <Route path="/login" element={<Login />} />
        <Route path="/signup" element={<Signup />} />
        <Route path="/forgot-password" element={<ForgotPassword />} />
        <Route path="/student" element={<StudentDashboard />} />
        <Route path="/admin" element={<AdminDashboard />} />
        <Route path="/quiz-generator" element={<QuizGenerator />} />
        <Route path="/my-assignments" element={<StudentAssignments />} />
      </Routes>
    </Router>
  );
}

export default App;
```